

University of Stuttgart

Germany

Institute of Industrial Automation and
Software Engineering

Hierarchical Imitation and Reinforcement Learning for Fault-Tolerant Manipulation

Master Thesis 4005

**Submitted at the University of Stuttgart by
Cheng Cheng**

MSc. Autonome Systeme

Examiner: Prof. Dr.-Ing. Andrey Morozov

Supervisor: Yuliang Ma

18.05.2026



Contents

List of Figures	v
List of Tables	viii
List of Abbreviations	x
Kurzfassung	xii
Abstract	xiii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Contributions	2
1.4 Thesis Organization	4
2 Related Work	5
2.1 Fault-Tolerant Robot Control (FDI/FTC)	5
2.2 POMDP Solution Methods	6
2.3 Robotic Reinforcement Learning and Residual Structure	6
2.4 Robust Reinforcement Learning and Domain Randomization	7
2.5 Imitation Learning and Human-in-the-Loop	7
2.6 Safe Reinforcement Learning and the Reward Hacking Background . . .	8
3 Encoder Bias Modeling and POMDP Formulation	10
3.1 Encoder Calibration Bias in Industrial Robots	10
3.2 Causal Chain: One Source, Three Effects	12
3.2.1 Effect 1: Initial Position Offset	14
3.2.2 Effect 2: Control Bias	14
3.2.3 Effect 3: Perception Bias	15
3.2.4 Summary of the Causal Chain	15
3.3 POMDP Formalization	16
3.3.1 The Bias-Free MDP Baseline	16
3.3.2 The POMDP with Encoder Bias	17
3.3.3 Episode-Level Static Property of the Latent Variable $\mathbf{b}$	18

Contents

3.4	Observability Analysis	18
3.4.1	Proposition 1: Non-Identifiability under the Basic Observation . .	19
3.4.2	Proposition 2: Observation Augmentation Restores Identifiability	19
3.4.3	Identifiability $\neq$ Solvability	20
3.4.4	Discussion: Noise, Configuration Information, and Correspondence with Experiments	22
3.5	Simulation Implementation and Equivalence Analysis	23
3.5.1	Sufficient Conditions for Fault Injection	23
3.5.2	Concrete Implementation in MuJoCo	24
3.5.3	External Sensor Simulation	26
3.5.4	Main Gaps between Simulation and Reality	26
4	Task Policy: Method Design and Shared Network Architecture	28
4.1	Motivation of the Method Design	28
4.2	Observation Space Design	30
4.2.1	Composition of the Observation Vector	30
4.2.2	Information Paths	31
4.2.3	Why the Corrupted Proprioception Is Retained	31
4.2.4	Generalized Treatment of Image Observation and TCP Localization	32
4.3	Randomized-Bias Training	33
4.4	Network Architecture and Training Objective	34
4.5	Reward Design	37
5	Simulation: Policy Training and Observability Validation	39
5.1	Introduction: The Simulation-Side Paradigm and Chapter Organization	39
5.2	Simulation Training of the Task Policy (HIL-SERL)	40
5.3	Simulation Training of the Backup Policy (SAC + DR)	42
5.3.1	Problem Definition and Safety Requirements	42
5.3.2	Choice of the Training Paradigm	43
5.3.3	Scenarios, Observation, and Motion Modes	45
5.3.4	Evolution of the Reward Function Design	46
5.3.5	V3→V3b Geometry Upgrade: from a Single Sphere to a 5-Sphere Whole Arm	48
5.3.6	Training Protocol and Domain Randomization	49
5.4	Simulation Experiments and Analysis	50
5.4.1	Experimental Setup	50
5.4.2	H1: The Encoder-Bias Degradation Baseline	51
5.4.3	H2 + H3: Fixed vs. Random Bias Training	51
5.4.4	Observation-Space Ablation: from 18D to 24D	52
5.4.5	Backup S1 V3b Main Survival Rate	53
5.4.6	Backup Bias Robustness Zero-Shot	53
5.4.7	Reward V1 / V2 / V3 Evolution Training Curves	54

Contents

5.4.8	Backup S2 Multi-Obstacle Scenario Extension	54
5.5	The Bridge from Simulation to the Real Robot	55
6	Real Robot: Imitation Deployment and End-to-End Validation	56
6.1	Introduction: The Real-Robot-Side Paradigm and Chapter Organization	56
6.2	Real-Robot System Architecture and Encoder-Bias Injection	57
6.2.1	Dual-Machine Hardware Architecture	57
6.2.2	B+D Dual-Injection-Point Encoder Bias	58
6.2.3	Equivalence of OSC and Cartesian Impedance Control	59
6.3	Real-Robot Deployment of the Task Policy: BC pretrain + HG-Dagger .	59
6.3.1	The Real-Robot 14-Dimensional Observation	60
6.3.2	BC pretrain	61
6.3.3	HG-Dagger Intervention Iteration	62
6.3.4	Three-Task Deployment: Shared Pipeline and Task-Level Differences	64
6.3.5	Methodological Post-Mortem of the HIL-SERL Real-Robot Failure	65
6.4	Real-Robot Deployment of the Backup Policy	67
6.4.1	Hand-Detection Selection: the MediaPipe → WiLoR Design Evolution	67
6.4.2	The Hierarchical Supervisor and the HomingController	68
6.4.3	Sim-to-Real Geometric Alignment: Route A center-dist	68
6.5	Real-Robot Experiments and Analysis	69
6.5.1	The Three-Task Joint Success-Rate Matrix	69
6.5.2	The HG-Dagger Intervention-Rate Descent Curve	71
6.5.3	End-to-End Validation of the Encoder-Bias Injection	71
6.5.4	Real-Robot Avoidance Evaluation of the Backup Policy	71
6.5.5	Data Post-Mortem of the HIL-SERL Real-Robot Negative Result .	72
6.6	Operator Safety, Measured Precision, and Multiple Rounds of Fixes . . .	72
7	Conclusion and Outlook	74
7.1	Summary of the Work	74
7.2	Limitations	75
7.3	Future Work	75
	Bibliography	77

List of Figures

3.1	Control and observation data flow in the ideal case	13
3.2	Causal chain of the encoder bias	16
3.3	Geometric illustration of identifiability. The horizontal axis $\mathbf{q}_{\text{true}}$ and the vertical axis $\mathbf{b}$ are an abstract one-dimensional representation. (a) When relying only on the basic observation, all pairs $(\mathbf{q}_{\text{true}}, \mathbf{b})$ satisfying $\mathbf{q}_{\text{true}} + \mathbf{b} = \mathbf{c}$ (red dashed line) produce the same observation $\mathbf{o}$; the translational symmetry along the direction δ makes the original problem non-identifiable. (b) After introducing an independent external position observation, by Equation (3.19), $\mathbf{q}_{\text{true}}$ is locked to $\text{FK}^{-1}(\mathbf{o}_{\text{real}})$ (blue solid line); the intersection of this constraint with the equivalence line is unique (green dot), i.e., the identifiable solution given by Proposition 3.2. When $\sigma > 0$, the blue line becomes a “band” of finite width, and uniqueness degenerates into a maximum-likelihood estimate over the posterior—this is precisely the geometric origin of the “averaging compensation” in Section 3.4.3.	21
4.1	Illustration of the information paths of the 25-dimensional observation space. In the encoder path (left, 18D), the joint angle and the TCP position derived from it are corrupted by the bias $\mathbf{b}$, while the joint velocity and the gripper opening are unaffected; the external observation path (right, 7D) contains the independent TCP and the hand information, all decoupled from the encoder. The two signal paths converge at the policy network, and in principle the policy can implicitly recover $\mathbf{b}$ from the inconsistency between $\mathbf{q} + \mathbf{b}$ and $\mathbf{o}_{\text{real}}$	32

List of Figures

4.2	Network architecture of the task policy. The three input streams (simulation 25-dimensional / real-robot 14-dimensional state + front-view and wrist 128×128 RGB) are each processed by their own encoder and then concatenated channel-wise into a 768-dimensional shared feature $\mathbf{h}$; the three heads share $\mathbf{h}$ as input but have different outputs and training losses—the continuous Actor produces a 6-dimensional Cartesian end-effector action, the Twin Critic outputs two scalar Q-values (the minimum is taken to suppress overestimation), and the discrete Gripper Critic outputs a 3-dimensional Q-value to select the gripper action. The DINOv3-S backbone remains frozen during training, and only the subsequent projection layers, the state encoder, and the three heads are updated. Simulation HIL-SERL trains all four losses, whereas the real-robot BC stage trains only the continuous Actor (the critic and the temperature parameter serve as ckpt-schema placeholders).	35
5.1	The Actor-Learner distributed architecture of HIL-SERL. The Actor is responsible for interacting with the environment, receiving HIL interventions, and writing into the online buffer; the Learner holds the buffers and the optimizer, and is responsible for mixed sampling, the SAC update, and pushing the weights back. The two sides communicate through gRPC, and environment sampling and gradient updating are fully decoupled. .	41
6.1	The real-robot dual-machine architecture. The RT PC communicates with the Franka control cabinet over a direct Ethernet link and runs the 1 kHz real-time control stack; the GPU workstation communicates with the RT PC over an independent subnet and bears policy inference, visual perception, and training. The decoupling of the two machines lets the GPU-side policy iteration and the RT PC-side real-time control evolve independently.	57
6.2	The path difference between the real-robot 14-dimensional observation and the simulation 25-dimensional observation (Figure 4.1). On the simulation side, the encoder path 18D ($\mathbf{q} + \mathbf{b}$, $\dot{\mathbf{q}}$, the gripper opening, $\text{FK}(\mathbf{q} + \mathbf{b})$) and the hand channel 4D ($\mathbf{h}_{\text{active}}$, $\mathbf{p}_{\text{hand}}$) do not enter the observation on the real robot for now (the red crosses); only the $\mathbf{o}_{\text{real}}$ path is retained and extended into a 7D true TCP pose containing the quaternion (tcp_true), which together with the end-effector 6D velocity (linear + angular, computed from $\mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}}$) and the gripper opening forms 14D. $\mathbf{o}_{\text{real}}$ is the simulation-validated “success signal path” of Proposition 3.2, provided by the <code>/getstate_true</code> external independent channel described in Section 6.2.2.	61

6.3	The HG-DAGger intervention state machine. In the IDLE state the actions are generated by the BC policy and not marked as an intervention; when the stick displacement $\ sm[: 6]\ > 0.05$ or any button is pressed, it switches to ACTIVE, the actions are then provided by the SpaceMouse and marked as an intervention; after the stick displacement < 0.02 is sustained for <code>tail_k = 10</code> frames (about 1 s @ 10 Hz), it switches back to IDLE. <code>tail_k</code> also retains the follow-through frames of the “release moment” as intervention data, capturing the corrective signal of the recovery tail. . .	63
6.4	The BC + HG-DAGger iteration workflow. Each round of iteration is a four-step “train–deploy–intervene–merge” closed loop. (1) A batch is randomly sampled from the merged dataset (the original demos + the dagger of past rounds, with <code>-intervention-only</code> making the original demos fully retained and the dagger keeping only the intervention frames), and a new policy is initialized from scratch and trained for 20,000 steps of BC pretrain; (2) the new ckpt is loaded and deployed to the real robot to execute 30 episodes; (3) the operator intervenes as needed, and all transitions are written into a pickle with an <code>is_intervention</code> label; (4) the intervention frames are merged into the dataset, triggering the next round of retraining. The stopping criterion simultaneously requires the intervention rate and the zero-intervention proportion to satisfy the thresholds; on the real robot, 1–3 rounds are measured to be enough to bring a pickup-type task into a steady state.	65

List of Tables

3.1	Five sources of encoder calibration bias and their characteristics	12
4.1	Mapping from the theoretical analysis of Chapter 3 and the practical constraints to the method design of this chapter	30
4.2	Composition of the 25-dimensional state observation of the task policy .	31
4.3	Task-policy training hyperparameters (simulation HIL-SERL vs. real-robot BC + HG-Dagger; “—” means not applicable)	37
5.1	The four obstacle motion modes in backup-policy training	46
5.2	Total expected return of typical scenarios under the V3 reward ($\gamma = 1.0$, 10-step episode)	47
5.3	Key hyperparameter differences of the backup policy relative to the task policy	49
5.4	Domain-randomization parameters in backup-policy training	50
5.5	Sim-to-real gap analysis of the backup policy	50
5.6	H1: degradation of the bias-free policy under different J1 biases	51
5.7	H2 + H3: success-rate comparison of the three policies under a bias sweep (PickCube 24D environment; the same bias grid as the H1 degradation curve of Table 5.6)	52
5.8	Observation-space ablation: a one-to-one correspondence between theoretical prediction and measurement	53
6.1	Composition of the 14-dimensional state observation of the real-robot deployment	61
6.2	Final comparison of hand-detection schemes after real-robot deployment (vs. the initial comparison during the simulation planning of Section 5.3)	68

List of Tables

6.3	The real-robot three-task $\times$ 4-column “training bias $\times$ test bias” joint success-rate matrix $\star$ the core experimental data of the thesis (each cell $N = 10$ episodes, a small-sample exploratory test). Training data without bias = the BC pretrain 50 demos + the HG-Dagger intervention frames are all collected under the bias-free condition; training data with bias = the same training protocol but all demos and intervention frames are collected under the injection condition of a J1 encoder bias $\mathbf{b} \sim \mathcal{U}[-0.2, 0.2]$ rad, corresponding to the randomized-bias training paradigm of Section 4.3; test with bias = the same J1 bias distribution is injected in real time during evaluation. Each column is evaluated with the HG-Dagger iter 2 converged ckpt of the corresponding training setting (Table 6.4). The runtime backup does not affect the task success rate because of the HOMING reset, and its separate evaluation is given in Section 6.5.4.	70
6.4	HG-Dagger three-task intervention-rate descent statistics (each iter 30 episodes). This table corresponds to the convergence process of the “training data with bias” policy used in columns 3, 4 of Table 6.3; the “training data without bias” comparison policy goes through the same iteration protocol, with a similar intervention-rate convergence trend, and is not listed repeatedly.	71

List of Abbreviations

BC	Behavior Cloning
CIC	Cartesian Impedance Control
CMDP	Constrained Markov Decision Process
D _{Agger}	Dataset Aggregation
DOF	Degrees of Freedom
DR	Domain Randomization
DRQN	Deep Recurrent Q-Network
EE	End-Effector
FCI	Franka Control Interface
FDI	Fault Detection and Isolation
FK	Forward Kinematics
FSM	Finite State Machine
FTC	Fault-Tolerant Control
HG-D _{Agger}	Human-Gated D _{Agger}
HIL	Human-in-the-Loop
HIL-SERL	Human-in-the-Loop Sample-Efficient Robotic Reinforcement Learning
IL	Imitation Learning
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
OOD	Out-of-Distribution
OSC	Operational Space Control
POMDP	Partially Observable Markov Decision Process

List of Abbreviations

RL	Reinforcement Learning
RLPD	Reinforcement Learning from Prior Data
ROI	Region of Interest
ROS	Robot Operating System
SAC	Soft Actor-Critic
SVD	Singular Value Decomposition
TCP	Tool Center Point
UTD	Update-to-Data

Kurzfassung

Eine Encoder-Kalibrierungsabweichung in Industrierobotern erzeugt einen systematischen Fehler, der gleichzeitig den Regelkreis (über eine fehlerhafte Jacobi-Matrix) und die Zustandsbeobachtung (über eine fehlerhafte Vorwärtskinematik) verfälscht und den Roboter von einem vollständig beobachtbaren Markov-Entscheidungsprozess (MDP) zu einem partiell beobachtbaren MDP (POMDP) herabstuft. Diese Arbeit untersucht, wie sich unter einer unbekannten Encoder-Kalibrierungsabweichung eine robuste Manipulationsleistung ohne Stillstandszeit für eine Neukalibrierung aufrechterhalten lässt.

Wir formalisieren das Problem als POMDP, in dem der Abweichungsvektor als episodenseitige latente Variable den wahren Gelenkzustand unbeobachtbar macht. Eine Beobachtbarkeitsanalyse zeigt, dass übliche propriozeptive Beobachtungen verschiedene Abweichungszustände nicht unterscheiden können, eine Erweiterung um ein externes Signal der Endeffektorposition die näherungsweise Beobachtbarkeit jedoch wiederherstellt — wodurch der POMDP wieder auf einen mit gängigen Reinforcement-Learning-Algorithmen lösbaren MDP reduziert wird.

Darauf aufbauend schlagen wir ein zweiteiliges System vor: (1) eine *robuste Aufgabenstrategie*, die in der Simulation mit Reinforcement Learning from Prior Data (RLPD) unter einer randomisierten Abweichung validiert wird (Erfolgsquote 92–100 % über den Bereich $[0, 0.3]$ rad einer Greifaufgabe) und auf dem realen Roboter mittels Imitationslernen (Behavior Cloning und Human-Gated DAgger) eingesetzt wird, da der geringe Datendurchsatz Online-RL instabil macht; und (2) eine *Backup-Sicherheitsstrategie*, die menschliche Bediener während des Human-in-the-Loop-Trainings durch aktive Kollisionsvermeidung schützt. Domain Randomization überbrückt die Sim-to-Real-Lücke der verdrahteten Handerkennung.

Ablationsstudien bestätigen, dass das externe Positionssignal für eine abweichungsrobuste Leistung notwendig ist. Zudem stellen wir eine Einsatzarchitektur für den realen Roboter (Franka Panda) vor, einschließlich eines Fehlerinjektionsschemas mit zwei Injektionspunkten, das die Kausalkette der Abweichung auf realer Hardware getreu reproduziert.

Stichwörter: Encoder-Kalibrierungsabweichung, POMDP, Reinforcement Learning, fehlertolerante Manipulation, Sim-to-Real-Transfer

Abstract

Encoder calibration bias in industrial robots introduces a systematic error that simultaneously corrupts both the control loop (via incorrect Jacobian computation) and the state observation (via incorrect forward kinematics), degrading the robot from a fully observable Markov Decision Process (MDP) to a Partially Observable MDP (POMDP). This thesis addresses the challenge of maintaining robust manipulation performance under unknown encoder bias without requiring downtime for recalibration.

We first formalize the encoder bias problem as a POMDP, where the bias vector acts as an episode-level latent variable that renders the true joint state unobservable. Through observability analysis, we show that standard proprioceptive observations are insufficient to distinguish different bias conditions, and that augmenting the observation with an external end-effector position signal restores approximate observability—effectively reducing the POMDP back to an MDP that can be solved with standard reinforcement learning algorithms.

Building on this insight, we propose a two-component system: (1) a *robust task policy* that is validated in simulation with Reinforcement Learning from Prior Data (RLPD) under randomized encoder bias—achieving a 92–100% success rate across the full bias range $[0, 0.3]$ rad on a block-grasping task—and deployed on the real robot via imitation learning (behavior cloning plus human-gated DAgger), since the low real-robot data throughput makes online RL unstable; and (2) a *backup safety policy* that protects human operators during human-in-the-loop training by performing active collision avoidance when a hand enters the shared workspace. Domain randomization on the backup policy bridges the sim-to-real gap for noisy hand detection.

Ablation studies confirm that the external position signal is necessary for bias-robust performance, validating the theoretical observability analysis. We further present a real-robot deployment architecture for the Franka Panda, including a dual-injection-point fault injection scheme that faithfully reproduces the encoder bias causal chain on physical hardware.

Keywords: encoder bias, POMDP, reinforcement learning, fault-tolerant manipulation, sim-to-real transfer

1 Introduction

1.1 Background and Motivation

The reliability of industrial robots in assembly, grasping, and human–robot collaboration tasks depends critically on the accuracy of their joint encoders. Whether at the level of low-level joint servoing, Cartesian impedance control, or high-level task planning, both the control law and the observation module take the encoder reading q_{measured} as feedback input, and map joint space to task space through forward kinematics and the Jacobian [1]. In real-world deployment, however, a systematic bias b between the encoder reading and the true joint angle is pervasive, such that $q_{\text{measured}} = q_{\text{true}} + b$.

This bias can be introduced by a variety of mechanisms—factory calibration residuals, temperature drift, replacement of components without recalibration, misalignment after a collision, electromagnetic interference, among others (a detailed taxonomy and typical magnitudes are given in Section 3.1). Among these, the “replacement without recalibration” and “post-collision misalignment” categories can reach magnitudes above 0.1 rad, which is sufficient to significantly degrade an otherwise reliable policy when it executes fine manipulation tasks. To cope with such bias, industrial practice typically adopts two approaches: *offline recalibration* [2], [3] (high in precision, but requiring an interruption of production that disrupts the line takt time) and *traditional fault-tolerant control* (FDI/FTC) [4], which relies on an explicit fault model and aims at “maintaining stability” rather than “completing the task,” and is therefore often powerless against a problem such as a latent bias of unknown magnitude. This thesis takes a path different from both: **without interrupting production, the policy is exposed to a variety of biases already during training, so that it can adapt online to an unknown calibration bias.** A comparison and positioning of several natural alternatives to this path—such as explicitly identifying the bias and then compensating for it (which still requires an independent external observation and an offline recomputation of the Jacobian), or meta-RL (which, under the assumption of a constant latent variable, degenerates into single-task domain randomization)—is given in Chapter 2. This idea calls for a re-examination of the formal structure of the control problem (the bias as a latent variable degrades the MDP to a POMDP), the observation design, the training paradigm, and the guarantee of human–robot safety; these questions constitute the research theme of this thesis.

1.2 Problem Statement

Once an unknown bias $\mathbf{b}$ is introduced, the original MDP breaks down at three interrelated levels.

Partial observability A single bias source produces effects along three independent paths—an initial position offset (the steady-state result of the PD control law deviates from $\mathbf{q}_{\text{home}}$), a control bias (the task-space torque is computed from an incorrect Jacobian), and a perception bias (both the joint reading and the forward-kinematics end-effector pose are corrupted simultaneously). All three effects make it impossible for the agent to directly observe the true joint state, and the control problem degrades from an MDP to a POMDP (Chapter 3 further proves that the bias is non-identifiable under the basic observation, and becomes approximately identifiable once an independent external position observation is introduced).

Deployment constraints The visual sim-to-real gap causes pure-simulation training followed by direct transfer to fail on the real robot; real-robot data collection is costly, with the number of available demonstrations on the order of 50–200. **Under the real-robot conditions of this thesis** (10 Hz control frequency, a single episode ≤ 30 s, on the order of 50 demonstrations), under a sparse reward the SAC online stage exhibits a large drift once the actor is unfrozen (a detailed post-mortem is given in Section 6.3.5). Together, these constraints determine a dual-end paradigm of “simulation-side validation plus real-robot imitation deployment”: the simulation side uses RL to validate theoretical feasibility, and the real-robot side uses imitation learning (IL; including behavior cloning, BC, and human-in-the-loop intervention correction) to carry out deployment.

Human–robot collaborative safety Real-robot deployment involves a human hand intermittently entering the workspace (operator intervention during HIL training, and a worker approaching during production deployment), which requires a backup avoidance policy that cooperates with the task policy, taking over control when a distance threshold is triggered and handing it back once the avoidance is complete. The backup policy itself must be trained without introducing additional unsafe factors; this thesis adopts a scheme of pure-simulation SAC plus domain randomization (the safety constraint during HIL training is likewise provided by this backup policy, interrelated with the previous class of constraints).

1.3 Contributions

The core contributions of this thesis revolve around its title, “Hierarchical Imitation and Reinforcement Learning for Fault-Tolerant Manipulation,” and fall into three points.

Contribution 1: POMDP formalization of encoder bias and observability analysis (Chapter 3). This thesis presents a one-source-three-effect causal-chain model and a seven-tuple POMDP formalization, and proves through rigorous propositions that $(q_{\text{true}}, \mathbf{b})$ is non-identifiable under the basic observation (Proposition 3.1) and becomes approximately identifiable once an independent external position observation is introduced (Proposition 3.2). This is *identifiability in the information-theoretic sense*, and does not directly imply the learnability of any specific policy class; Section 3.4.3 further clarifies that “identifiability $\neq$ solvability”—even when the information is sufficient, the optimal policy must still output a compensating action conditioned on $\mathbf{b}$, which in turn establishes the necessity of “randomized-bias training.”

Contribution 2: A fault-tolerant manipulation framework of hierarchical imitation and reinforcement learning (Chapter 4–Chapter 6). This thesis designs a task $\oplus$ backup dual-policy hierarchical architecture together with a dual-end paradigm split into sim/real, comprising:

1. **A shared method layer** (Chapter 4)—observation design, randomized-bias training, a shared network architecture, and a sparse reward;
2. **Simulation-side task-policy training** (Chapter 5)—the HIL-SERL framework validates the observability propositions (the H1–H3 experimental loop);
3. **Real-robot-side task-policy deployment** (Chapter 6)—BC plus HG-Dagger imitation iteration, which avoids the actor-drift problem of SAC in the small-data real-robot setting;
4. **Backup-policy simulation training and real-robot deployment**—pure-simulation SAC plus DR training (Section 5.3), with a hard switch between the task policy and the backup policy carried out on the real robot by the three-state FSM of the Hierarchical Supervisor (Section 6.4.2).

The paradigm choice is formalized by a three-axis decision principle of (observation modality $\times$ data scale $\times$ safety constraint) (see Section 4.1 and Section 5.3.2 for details); the task and backup policies are two instances of this decision principle on the system of this thesis.

Contribution 3: End-to-end validation of three real-robot tasks under injected encoder bias (Section 6.5). This thesis implements a B+D dual-injection-point encoder-bias injection (Section 6.2.2), and designs a four-column joint success-rate matrix evaluation (a 2×2 “training bias $\times$ test bias” factorial: train w/o bias & test w/o bias, train w/o bias & test w/ bias, train w/ bias & test w/o bias, train w/ bias & test w/ bias) on the three real-robot tasks pickup, wipe, and pickandplace, which constitutes the core empirical thread of this thesis (the experimental design and partial results are given in Section 6.5, and the full matrix continues to be filled in as real-robot data accumulate). It also presents a methodology-level negative result of the HIL-SERL real-robot failure (see

Section 6.3.5), which provides empirical support for the paradigm choice of switching to BC plus HG-Dagger on the real robot.

1.4 Thesis Organization

This thesis consists of seven chapters, organized in the following order:

- **Chapter 1, Introduction:** research background, problem statement, contribution claims, and organizational structure.
- Chapter 2 (Related Work): reviews the literature on FDI/FTC, POMDP solving, imitation learning and human-in-the-loop methods, and safe RL, and positions the methodological choices of this thesis.
- Chapter 3 (Encoder Bias Modeling and POMDP Analysis): the theoretical core—the one-source-three-effect causal chain, the POMDP formalization, and the observability propositions.
- Chapter 4 (Task Policy: Method Design and Shared Network Architecture): the method layer shared by simulation and the real robot.
- Chapter 5 (Simulation: Policy Training and Observability Validation): the two main threads on the simulation side (the HIL-SERL task policy and the SAC+DR backup policy) and the H1–H3 experimental loop.
- Chapter 6 (Real Robot: Imitation Deployment and End-to-End Validation): the full real-robot deployment stack and the three-task joint success-rate matrix.
- Chapter 7 (Conclusion and Outlook): summarizes the contributions and gives future work (including the two-step ablation of real-robot bias robustness).

2 Related Work

This chapter reviews the related literature according to the logical layers of this thesis’s method, echoing the three classes of challenges raised in Section 1.2: Section 2.1 and Section 2.2 correspond to “partial observability,” introducing the classical frameworks of traditional control theory and partially observable decision-making; Section 2.3, Section 2.4, and Section 2.5 correspond to “deployment constraints,” providing the methodological foundation for the real-robot deployment of this thesis along three threads—the residual structure of robotic RL, robust RL and domain randomization, and imitation learning with human-in-the-loop; Section 2.6 corresponds to “human–robot collaborative safety,” providing the background on safe RL and reward hacking, which offers conceptual support for the analysis of the backup-policy reward evolution in Section 5.3.4. This chapter focuses on the landscape of the field and the positioning of the method, and does not repeat the specific configuration details of Chapter 4–Chapter 6.

2.1 Fault-Tolerant Robot Control (FDI/FTC)

Fault Detection, Isolation, and Reconfiguration (FDI/FTC) is a classical framework for fault-tolerant robot control [4]. FDI detects faults in a system and locates the fault source through analytical redundancy, state observers, or parameter identification; FTC, once a fault is detected, maintains system stability through control-law reconfiguration. This framework has been applied successfully to problems such as sensor drift and actuator lock-up, but exhibits three key limitations when applied to the “encoder systematic bias of unknown magnitude” addressed in this thesis:

- **It requires an explicit fault model**—bias sources addressed in this thesis, such as “replacement of components without recalibration,” lack a prior model;
- **It targets “maintaining stability”**—*classical passive FTC* stops or degrades operation after detection; although active FTC pursues performance retention, it still relies on an explicit fault model, which does not match the “unknown magnitude with no fault model” subproblem of this thesis;
- **It has limited ability to detect a latent-variable bias that is constant within an episode**—parameter-identification FDI (such as RLS or UKF) can in principle estimate a constant parameter online, but in the absence of an independent external observation there exists a non-identifiable subspace (consistent with the translational symmetry given in Proposition 3.1).

Haddadin et al. [5] survey the problem of robot collision detection and isolation, which likewise focuses on the “collision event” rather than the “residual bias after a collision.” This thesis adopts a path different from FDI/FTC—it treats the bias as part of the environment distribution and lets the policy adapt online by having seen the bias during training, thereby bypassing the two-stage paradigm of “detect first, then switch.”

2.2 POMDP Solution Methods

Solving partially observable MDPs has long been a core difficulty of reinforcement learning [6] and sequential decision-making. Kaelbling et al. [7] propose an exact solution method based on belief states, which converts a POMDP into an MDP solved over the belief space; such methods have theoretical guarantees on small-scale discrete problems, and extending them to the high-dimensional continuous state space used in this thesis requires approximation (such as point-based POMDP solvers or deep variational belief models). Hausknecht and Stone [8] propose the Deep Recurrent Q-Network (DRQN), which uses an RNN/LSTM to implicitly memorize the history of observations, achieving good results on partially observable tasks such as Atari. Treating a latent-variable bias that is constant within an episode as a *static hidden parameter* or a *contextual MDP* is another thread for handling this class of problem (it shares with the randomized-bias training of this thesis the basic idea of “exposing the policy to multiple parameters during training,” but explicitly modeling the latent variable requires an additional inference module). Weighing engineering trade-offs (implementation complexity, hyperparameter sensitivity, and compatibility with the existing BC/RL training stack), this thesis chooses the path of observation augmentation plus end-to-end RL—introducing an independent external observation so that the POMDP is approximately restored to an MDP in the information-theoretic sense (see Section 3.4 for details); the other paths above are in principle solvable and are mentioned as future work in Chapter 7.

2.3 Robotic Reinforcement Learning and Residual Structure

Residual reinforcement learning (Residual RL) was proposed by Johannink et al. [9]: a hand-designed base controller first completes most of the action, and RL then learns a residual policy that makes small corrections to the base controller’s output. Residual RL can significantly reduce sample complexity and improve training stability on many robotic tasks, and is a common compromise in the sim-to-real field—it leaves the analytically modelable part to the base controller and the hard-to-model disturbances to RL.

The encoder bias of this thesis happens to corrupt the joint readings used by the base controller (the Jacobian and forward kinematics); whether Residual RL remains effective in this scenario requires a separate empirical analysis. This thesis does not conduct a

systematic comparison of Residual RL against BC plus HG-Dagger on the real robot, and leaves it as future work (see Chapter 7).

2.4 Robust Reinforcement Learning and Domain Randomization

Domain Randomization (DR) is the mainstream approach for narrowing the sim-to-real gap. Tobin et al. [10] propose visual DR on an object pose estimation task—by randomizing parameters such as textures, lighting, and camera poses, the policy is made robust to differences in visual appearance; subsequent work extends this to state-space DR (position noise, control delay, dynamics parameters), achieving sim-to-real success in scenarios such as legged robots and drones.

The “randomized-bias training” used in this thesis (see Section 4.3 for details) is formally cognate with DR—both rely on the basic idea of “letting the policy see samples of the deployment distribution during training.” It is necessary, however, to point out an essential information-theoretic difference between the two: in the original setting of Tobin et al. (pose estimation, grasping), the randomized visual parameters are *redundant variables* for the policy (they should be ignored, and what the policy learns should be an invariance); whereas the bias $\mathbf{b}$ randomized in this thesis is a latent variable on which the policy *must condition*—according to the conditional-compensation requirement of Section 3.4.3, the optimal policy must output different actions for different values of $\mathbf{b}$. In this setting the two belong to two opposite information-theoretic paradigms, invariance learning and conditional-compensation learning respectively, and are cognate only in the surface idea that “the training distribution covers the deployment distribution.” State-space DR is used as a mature approach in the backup-policy training of Section 5.3.6—low-dimensional state perturbations such as position noise, velocity jitter, and detection delay can be covered by DR alone to bridge the sim-to-real gap, which forms a contrast with the visual path of the task policy that requires BC plus HG-Dagger real-robot imitation.

2.5 Imitation Learning and Human-in-the-Loop

Imitation Learning (IL) and the Human-in-the-Loop (HIL) variants of reinforcement learning (RL) together form the methodological foundation of the real-robot deployment of this thesis. This section reviews two parallel branches—BC→Dagger→HG-Dagger in the IL direction and RLPD→HIL-SERL in the RL+HIL direction—and finally gives the rationale for the dual-end choice of this thesis.

IL branch: BC, DAgger, HG-DAgger **Behavior Cloning** (BC) is the simplest IL method—supervised learning on expert demonstration data that maps states to actions. BC is simple to implement and stable to train, but its behavior is undefined in

out-of-distribution (OOD) states (the compounding-error problem), and it usually requires a large number of demonstrations or additional regularization. **Dagger** (Dataset Aggregation) [11], proposed by Ross et al., is a classical improvement—during policy deployment it queries the expert on newly encountered states, merges the expert actions into the dataset, and retrain, iterating until convergence. The key limitation of Dagger is that the query is initiated by the policy (query-by-policy)—frequently asking the expert for actions in unexpected states while the policy is not yet stable places an undue burden on the human expert. **HG-Dagger** (Human-Gated Dagger) [12] is a human-initiated variant of Dagger—the intervention trigger is decided by the human (human-initiated): the operator intervenes only on the failure modes of the policy, and the intervention frames are merged into the next retraining round. This thesis adopts HG-Dagger as the final approach for the real-robot stage.

RL+HIL branch: RLPD, HIL-SERL RLPD [13], building on Soft Actor-Critic [14], adds a 50/50 mixed sampling of an offline demonstration buffer and an online buffer, providing demo guidance for sparse-reward tasks. **HIL-SERL** [15], on top of RLPD, makes human-in-the-loop intervention a continuous source of training data throughout the entire online stage, with intervention transitions reinjected into the offline buffer on an equal footing with the original demos. HIL-SERL achieves outstanding results on several real-robot manipulation tasks in the original work of Luo et al. *The interpretation of this thesis is the following:* the SAC online stage of HIL-SERL requires a relatively high real-robot throughput for the critic to calibrate the Q-surface online through continuous online interaction (inferred backward from the real-robot failure mode in Section 6.3.5), and this condition is not easily satisfied on a low-throughput real robot.

The dual-end choice of this thesis The simulation side adopts HIL-SERL (Section 5.2)—simulation throughput is high and bias injection is controllable, so HIL-SERL fully converges and validates H1–H3; the real-robot side switches to BC plus HG-Dagger (Section 6.3)—under the real-robot conditions of this thesis, the actor drifts substantially after being unfrozen in the SAC online stage of HIL-SERL (a detailed post-mortem is given in Section 6.3.5), whereas HG-Dagger does not depend on a critic, has a smaller tuning surface, and is markedly more stable in the small-data real-robot scenario. This choice is determined by the three axes of (observation modality $\times$ data scale $\times$ hardware throughput), and is one instance of the decision principle for choosing a paradigm in Contribution 2 of Section 1.3.

2.6 Safe Reinforcement Learning and the Reward Hacking Background

Safe reinforcement learning is concerned with guaranteeing that a system does not violate safety constraints during training or deployment. Methods such as the Constrained

MDP and Constrained Policy Optimization (CPO) handle constraints explicitly through techniques such as Lagrangian multipliers or rejection sampling. This thesis does not directly adopt the constrained MDP—the safety requirement of this thesis (human–robot collaborative avoidance) is realized through a *hierarchical structure* (task policy + backup policy + a runtime distance-triggered hard switch, see Section 5.3.2 for details) rather than constrained optimization, which better matches the need for engineering interpretability in industrial deployment scenarios.

Kiemel et al. [16] propose, in the context of safe RL for industrial robots, the reward-structure principle of *non-negative immediate rewards + a large termination penalty*, used to train backup-as-safety-layer avoidance policies. The V3 backup-policy reward of Section 5.3.4 directly inherits this principle, and over the three iterations $V1 \rightarrow V2 \rightarrow V3$ empirically demonstrates that “if the reward design does not satisfy this principle, the policy will, through reward misspecification, find an optimal solution outside the design intent.”

Reward hacking refers to a policy violating the design intent while maximizing the reward signal—a typical manifestation is that the policy finds a shortcut behavior that “raises the reward without truly completing the task.” Reward hacking is a classical problem in reward engineering, and is widely discussed in both large language model alignment and robotic RL. The two failed iterations V1 (a proximity reward) and V2 (an all-negative reward) of Section 5.3.4 provide two typical manifestations of reward misspecification (V2 additionally exhibits a termination-policy bias); the specific mechanisms and a comparison of training curves are given in Section 5.3.4.

3 Encoder Bias Modeling and POMDP Formulation

3.1 Encoder Calibration Bias in Industrial Robots

The closed-loop control of a serial-link industrial robot relies on the angular feedback of every joint. The low-level joint PD controller, the Cartesian-space impedance/admittance controller, and the high-level task planner all require the joint angle $\mathbf{q}_{\text{measured}}$ measured by the joint encoders as input, and establish the connection between joint space and task space through forward kinematics (FK) and the Jacobian matrix $J(\mathbf{q})$ [1]. In the ideal case, the encoder reading equals the true joint angle ($\mathbf{q}_{\text{measured}} = \mathbf{q}_{\text{true}}$), and the torque computation of the controller and the end-effector pose estimation of the observation module are both built on accurate state information. In a real industrial environment, however, this assumption is often broken: a systematic bias $\mathbf{b} \in \mathbb{R}^7$ may exist between the encoder reading and the true joint angle, such that

$$\mathbf{q}_{\text{measured}} = \mathbf{q}_{\text{true}} + \mathbf{b}. \quad (3.1)$$

Because the control and the observation of the robot share the same encoder reading, the bias $\mathbf{b}$ corrupts both the control loop and the state observation simultaneously, a point analyzed in detail in Section 3.2.

Five Sources of Encoder Bias

Encoder calibration bias can be caused by a variety of physical mechanisms in industrial practice. ISO 9283:1998 [17] provides standardized test methods for performance characteristics of industrial robots such as pose accuracy, repeatability, and drift, and explicitly lists indicators such as pose drift and interchangeability, reflecting the prevalence of such bias in real deployment. The following classifies these sources by time scale and physical mechanism, and gives representative magnitudes obtainable from the existing literature or from engineering practice.

1. **Factory calibration residual:** even after rigorous kinematic parameter identification, limited by the precision of the calibration equipment and the simplification of the kinematic model, a small residual bias still remains in the joint zero positions after factory calibration. As a concrete reference, Kana et al. [2] carried out kinematic recalibration on a Kinova Gen3 seven-degree-of-freedom arm and

reported zero-offset values of the seven joint angles distributed over the interval $[-0.0084, 0.0088]$ rad, roughly corresponding to the 0.5 scale. This is the *only* one of the five sources in this thesis with direct empirical magnitude support.

2. **Temperature drift:** after a robot arm has run for a long time, the motors, reducers, and links produce thermal expansion due to heating, causing a slow drift of the end-effector pose. Gong et al. [3] modeled, through inverse calibration with a laser tracker, the coupled influence of geometric, stiffness, and time-varying thermal errors in industrial robots; Vocetka et al. [18] further quantified the influence of drift on repeatability and gave a compensation method. The existing literature usually reports the thermal-drift effect as a task-space pose error (on the millimeter scale), which when traced back to joint space lies roughly in the range 10^{-3} – 10^{-2} rad, far smaller, by an order of magnitude or more, than the fault categories discussed below.
3. **Motor or reducer replacement without recalibration:** if a replaced component is not re-zeroed during maintenance (for example, KUKA’s “mastering” or Franka’s zero-position reset procedure is not performed), a fixed bias is introduced. For example, in a typical harmonic reducer (reduction ratio 100–160), *the misalignment of a single input tooth* can cause a bias of about 0.04 rad at the output, while the influence of coupler reassembly or belt-tension change can be further superimposed. Since this kind of bias depends on the specific maintenance operation, statistical reports are lacking in the academic literature, and this thesis treats it as an *engineering estimate* rather than an empirical magnitude, with a possible amplitude range on the order of 0.05–0.3 rad.
4. **Post-collision encoder misalignment:** an accidental collision may cause a sudden change in the relative position of the joint encoder, the harmonic reducer, or the coupler, producing a sudden bias. Haddadin et al. [5] provide a systematic survey of the detection, isolation, and identification of robot collisions, but the literature in this field mainly focuses on *detection and recovery* rather than the magnitude statistics of the residual bias after a collision. This thesis likewise treats the bias caused by a collision as a *design envelope*, with a possible amplitude range extending from 0.1 rad to a larger scale depending on the collision severity.
5. **Encoder noise and electromagnetic interference:** modern optical encoders, through differential line drivers and shielded designs, already suppress electromagnetic interference strongly. The residual measurement noise (including quantization noise) is about 10^{-3} rad on a typical 17–20-bit absolute encoder, and is zero-mean random, essentially different in statistical character from the first four systematic biases.

Table 3.1 summarizes the key characteristics of the above five bias sources, and distinguishes them in the table by the two categories *empirical magnitude* and *engineering*

estimate, in order to make clear the differences among the sources in their literature support.

Table 3.1: Five sources of encoder calibration bias and their characteristics

Source	Temporal character	Category	Representative magnitude (rad)	Covered
Factory calibration residual	Fixed	Empirical magnitude [2]	$\sim 10^{-3}$ – 10^{-2}	✓
Temperature drift	Slowly varying	Empirical magnitude [3], [18]	$\sim 10^{-3}$ – 10^{-2}	×
Replacement without recalibration	Fixed	Engineering estimate	$\sim 10^{-1}$	✓
Post-collision misalignment	Sudden	Design envelope [5]	$\gtrsim 10^{-1}$	✓
Encoder noise / EMI	Random	Quantization + noise	$\sim 10^{-3}$	×

Scope of This Thesis

Among the above five sources, this thesis focuses on **bias that is constant within an episode** (i.e., sources 1, 3, and 4), covering two experimental settings: *fixed bias* uses the same $\mathbf{b}$ across all episodes, corresponding to a single known fault; *random bias* samples $\mathbf{b}$ from a distribution $P(\mathbf{b})$ at the beginning of each episode, corresponding to a distribution of unknown fault magnitudes. What the two settings have in common is that $\mathbf{b}$ is invariant within an episode, and both can be modeled as a static latent variable of a POMDP (see Section 3.3); whereas temperature drift and electromagnetic interference require a time-varying latent state, constitute an essentially different POMDP structure, and are left as future work. In the subsequent experiments, the bias is sampled independently for each joint, $b_i \sim \mathcal{U}[0, 0.25 \text{ rad}]$ ($i = 1, \dots, 7$, about 14.3), deliberately exceeding the typical magnitude of the factory calibration residual (Table 3.1), so as to cover the more severe fault scenarios such as maintenance oversight and post-collision misalignment.

For such bias, existing approaches such as offline recalibration [2], [3] or the traditional fault-tolerant control framework (FDI/FTC) [4] all have obvious limitations—the former requires interrupting production, and the latter relies on an explicit fault model and targets “maintaining stability” rather than “completing the task,” with the related discussion given in Chapter 2. This thesis takes a different path: rather than detecting or switching after the fact, it lets the policy see various biases already during the training stage, so that it learns to *adapt online* to an unknown calibration bias. The subsequent chapters develop this at both the theoretical and the methodological levels—Sections 3.2 to 3.4 characterize the essential influence of the bias on the structure of the control problem, and Chapter 4 gives the concrete policy-learning method.

3.2 Causal Chain: One Source, Three Effects

In Section 3.1, we formalized the encoder bias as an additive error $\mathbf{b}$ in joint space, such that $\mathbf{q}_{\text{measured}} = \mathbf{q}_{\text{true}} + \mathbf{b}$ (see Equation (3.1)). This definition itself only characterizes “what the bias is,” but does not yet answer a more crucial question: *how does an encoder bias propagate in the closed-loop control system of a robot and ultimately affect task execution?*

Answering this question is the premise of the subsequent POMDP modeling (Section 3.3). This section proves: **a single encoder bias b produces three observable effects along three mutually independent paths, but these three effects are not independent faults—they are projections of the same latent variable onto different modules.**

To understand how the bias propagates, we first review the standard control and observation structure of a serial manipulator such as the Franka Panda in the bias-free case [1]. As shown in Figure 3.1, within one control cycle the robot has three modules that explicitly depend on the same encoder reading q_{measured} : the **low-level joint PD controller** (which for the Franka Panda runs in the 1 kHz real-time loop of its FCI [19], [20]) tracks the desired joint angle; the **task-level OSC controller** maps the task-space error to joint torques through $J(q_{\text{measured}})$ and $\text{FK}(q_{\text{measured}})$; and the **observation module** returns q_{measured} and $\text{FK}(q_{\text{measured}})$ to the high-level policy. The three modules share the same encoder reading—this is precisely the structural reason why a single b can corrupt both control and observation simultaneously¹.

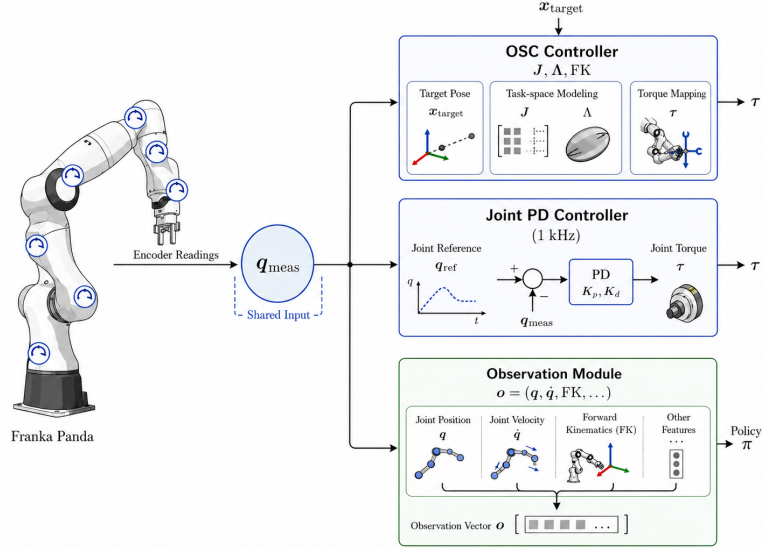


Figure 3.1: Control and observation data flow of the Franka Panda in the ideal case. The three downstream modules (low-level joint PD, task-level OSC, observation module) all take the same encoder reading q_{measured} as input. This shared dependency is the structural reason why the encoder bias can corrupt both control and observation simultaneously.

¹This section assumes that the real robot uses joint servoring executed by a low-level real-time controller; in the simulation environment this low-level loop is simplified by directly setting the initial joint position, with the concrete implementation detailed in Section 3.5.

3.2.1 Effect 1: Initial Position Offset

At the beginning of an episode, the high level usually commands the robot to move to a preset initial pose $\mathbf{q}_{\text{home}}$. The low-level joint PD controller of the Franka Panda can be approximately modeled as a PD tracking controller with gravity compensation [1]:

$$\boldsymbol{\tau}_{\text{joint}} = \mathbf{K}_p(\mathbf{q}_{\text{target}} - \mathbf{q}_{\text{measured}}) + \mathbf{K}_d(\dot{\mathbf{q}}_{\text{target}} - \dot{\mathbf{q}}_{\text{measured}}) + \mathbf{g}(\mathbf{q}_{\text{measured}}), \quad (3.2)$$

where $\mathbf{K}_p, \mathbf{K}_d \succ 0$ are the stiffness and damping matrices, and $\mathbf{g}(\cdot)$ is the gravity compensation term. In the scenario of reaching the home pose, $\mathbf{q}_{\text{target}} = \mathbf{q}_{\text{home}}$ is constant, so $\dot{\mathbf{q}}_{\text{target}} = \mathbf{0}$; furthermore, when the system reaches steady state $\dot{\mathbf{q}}_{\text{measured}} = \mathbf{0}$, and the passive torque and the gravity compensation cancel, so Equation (3.2) reduces to

$$\mathbf{K}_p(\mathbf{q}_{\text{target}} - \mathbf{q}_{\text{measured}}) = \mathbf{0} \implies \mathbf{q}_{\text{measured}} = \mathbf{q}_{\text{target}}. \quad (3.3)$$

This is precisely the basic property of a PD controller: in steady state the *measured* value equals the desired value. However, the feedback loop of the PD controller is closed on the encoder reading, and is blind to the *true* joint angle in the physical world. Substituting Equation (3.1) into Equation (3.3) gives:

$$\mathbf{q}_{\text{true}} + \mathbf{b} = \mathbf{q}_{\text{home}} \implies \boxed{\mathbf{q}_{\text{true}} = \mathbf{q}_{\text{home}} - \mathbf{b}}. \quad (3.4)$$

Physical meaning The robot *believes*, at the data level, that it has already reached $\mathbf{q}_{\text{home}}$, but physically all joints have an offset of magnitude $\mathbf{b}$ in the direction *opposite* to the bias. This effect is the first step of “the bias being projected from the digital world into the physical world”: no control command has yet been executed incorrectly, yet the system is already in a wrong physical state. All subsequent motion will start from this corrupted starting point.

3.2.2 Effect 2: Control Bias

During the episode, the task-level OSC controller converts the desired end-effector pose $\mathbf{x}_{\text{target}}$ into joint torques. The standard OSC control law [1], [21] has the form:

$$\boldsymbol{\tau}_{\text{osc}}(\mathbf{q}) = \mathbf{J}(\mathbf{q})^\top \boldsymbol{\Lambda}(\mathbf{q}) [\mathbf{K}_x \mathbf{e}_x(\mathbf{q}) + \mathbf{K}_v \dot{\mathbf{e}}_x(\mathbf{q})] + \boldsymbol{\tau}_{\text{null}}(\mathbf{q}) + \mathbf{g}(\mathbf{q}), \quad (3.5)$$

where $\mathbf{e}_x(\mathbf{q}) = \mathbf{x}_{\text{target}} - \text{FK}(\mathbf{q})$ is the task-space position error, $\boldsymbol{\Lambda}(\mathbf{q}) = (\mathbf{J}(\mathbf{q})\mathbf{M}^{-1}(\mathbf{q})\mathbf{J}(\mathbf{q})^\top)^{-1}$ is the task-space inertia matrix, $\boldsymbol{\tau}_{\text{null}}$ is the null-space term, and $\mathbf{g}$ is the gravity compensation. The control law explicitly depends on five functions of $\mathbf{q}$: $\mathbf{J}, \boldsymbol{\Lambda}, \text{FK}, \boldsymbol{\tau}_{\text{null}}, \mathbf{g}$.

When a bias is present, the controller reads $\mathbf{q}_{\text{measured}} = \mathbf{q}_{\text{true}} + \mathbf{b}$, so the torque actually applied is $\boldsymbol{\tau}^{\text{biased}} = \boldsymbol{\tau}_{\text{osc}}(\mathbf{q}_{\text{true}} + \mathbf{b})$, which differs from the ideal torque $\boldsymbol{\tau}_{\text{osc}}(\mathbf{q}_{\text{true}})$ by $\Delta\boldsymbol{\tau}(\mathbf{b})$. A first-order expansion for a small bias gives:

$$\Delta\boldsymbol{\tau}(\mathbf{b}) = \boldsymbol{\tau}_{\text{osc}}(\mathbf{q}_{\text{true}} + \mathbf{b}) - \boldsymbol{\tau}_{\text{osc}}(\mathbf{q}_{\text{true}}) \approx \left. \frac{\partial \boldsymbol{\tau}_{\text{osc}}}{\partial \mathbf{q}} \right|_{\mathbf{q}_{\text{true}}} \mathbf{b}. \quad (3.6)$$

Here $\partial \tau_{\text{osc}} / \partial q$ contains the variation of J , FK, and Λ with respect to the joint angle simultaneously, and is nonzero for any nontrivial kinematic structure, so:

$$\forall \mathbf{b} \neq \mathbf{0}: \quad \Delta \tau(\mathbf{b}) \neq \mathbf{0}. \quad (3.7)$$

Physical meaning Any nonzero encoder bias causes the controller to output a joint torque that deviates from the ideal value in both magnitude and direction; the first-order expansion Equation (3.6) shows that the magnitude of the torque bias is approximately linear in $\mathbf{b}$, and its direction is determined by the gradient of the OSC control law with respect to q . This effect originates from the difference of J and FK under different joint configurations rather than from any additive noise, so even if $\mathbf{b}$ is constant within an episode, the torque bias persists along the entire trajectory.

3.2.3 Effect 3: Perception Bias

The high-level policy π decides the next action based on the state $\mathbf{o}$ returned by the observation function. Taking a typical task of the Franka Panda as an example, the observation contains at least the following components:

$$\mathbf{o} = (q_{\text{measured}}, \dot{q}_{\text{measured}}, \mathbf{x}_{\text{ee}}^{\text{FK}}, \dots), \quad (3.8)$$

where $\mathbf{x}_{\text{ee}}^{\text{FK}} = \text{FK}(q_{\text{measured}})$ is the kinematic estimate of the end-effector position. Substituting Equation (3.1):

$$q_{\text{measured}} = q_{\text{true}} + \mathbf{b}, \quad (3.9)$$

$$\mathbf{x}_{\text{ee}}^{\text{FK}} = \text{FK}(q_{\text{true}} + \mathbf{b}) \neq \text{FK}(q_{\text{true}}). \quad (3.10)$$

Note that $\dot{q}_{\text{measured}} = \dot{q}_{\text{true}}$: because $\mathbf{b}$ is constant within an episode, it vanishes after differentiation with respect to time. Therefore, within an episode, the perception bias acts only on the position-type observations (joint angle and end-effector position), and does not affect the velocity-type observations.

Physical meaning The robot state that the policy “sees” *systematically lies in the position dimension*. The two position-type observations— q_{measured} and $\mathbf{x}_{\text{ee}}^{\text{FK}}$ —appear on the surface to be two independent information sources, but actually share the same corrupted underlying variable, so the policy *cannot* detect the bias by comparing them ($\mathbf{x}_{\text{ee}}^{\text{FK}}$ is just the forward kinematics of q_{measured} , and any “consistency check” between the two will always pass). This observation will play a central role in the observability analysis of Section 3.4.

3.2.4 Summary of the Causal Chain

Although the three effects arise via different paths (PD feedback, OSC Jacobian/FK, observation function), they *all originate from the same joint-space variable $\mathbf{b}$* —this is precisely

the direct basis for Section 3.3 modeling $\mathbf{b}$ as a *single episode-level latent variable*. Figure 3.2 summarizes this coupled structure graphically.

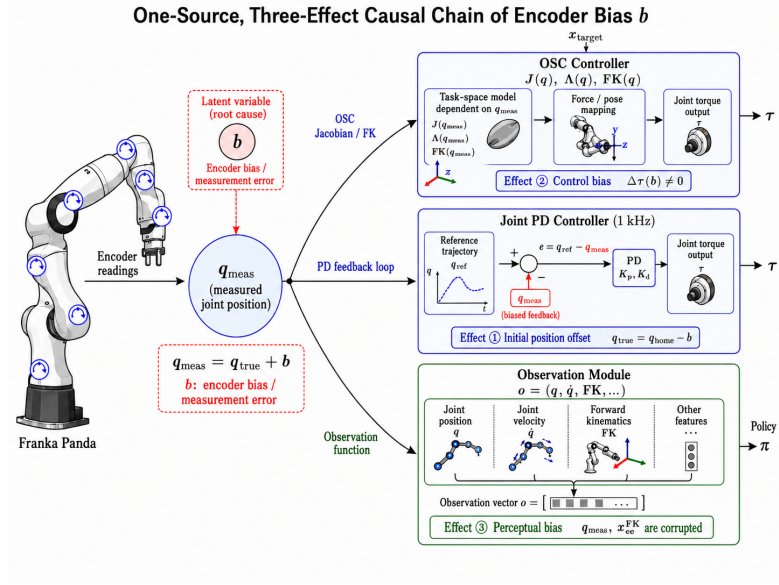


Figure 3.2: Causal chain of the encoder bias $\mathbf{b}$: the “one source, three effects” structure. A single joint-space latent variable $\mathbf{b}$ produces three observable effects simultaneously through three structurally independent paths—the feedback loop of the low-level PD controller, the Jacobian and FK in the OSC control law, and the observation function. The three effects are not independent faults, but projections of the same latent variable onto different modules.

3.3 POMDP Formalization

Section 3.2 characterized the propagation path of the encoder bias at the physical level. This section translates this physical story into the standard language of the reinforcement learning field: first reviewing the bias-free MDP baseline, then introducing the POMDP model after the bias is introduced, and focusing on the analysis of the episode-level static property of the *latent variable* $\mathbf{b}$ —a property that is the core premise of the methodology of this thesis, and also the starting point of the observability analysis in the subsequent Section 3.4.

3.3.1 The Bias-Free MDP Baseline

In the ideal case without encoder bias, a robot manipulation task can be modeled as a discrete-time Markov decision process [6]

$$\mathcal{M}_0 = (\mathcal{S}^{\text{phys}}, \mathcal{A}, T_{\text{phys}}, R, \gamma), \quad (3.11)$$

where the elements of the physical state space $\mathcal{S}^{\text{phys}}$, denoted $\mathbf{s}^{\text{phys}}$, contain the true kinematic state of the robot and the task-related environment variables:

$$\mathbf{s}^{\text{phys}} = (\mathbf{q}, \dot{\mathbf{q}}, g, \mathbf{x}_{\text{ee}}, \mathbf{x}_{\text{obj}}), \quad (3.12)$$

that is, the 7-dimensional joint angle, the joint velocity, the gripper opening, the end-effector position, and the position of the task target object. The action space $\mathcal{A} \subseteq \mathbb{R}^7$ is the control command that the high-level policy can issue (the concrete form will be defined in Chapter 4); $T_{\text{phys}}(\mathbf{s}^{\text{phys}'} | \mathbf{s}^{\text{phys}}, \mathbf{a})$ is the state transition kernel determined by the physics simulator; R is the sparse task-success reward; and $\gamma \in (0, 1)$ is the discount factor. Under this baseline, the system is *fully observable*, i.e., the observation $\mathbf{o} = \mathbf{s}^{\text{phys}}$, and the optimal policy takes the form $\pi^*(\mathbf{a} | \mathbf{s}^{\text{phys}})$.

3.3.2 The POMDP with Encoder Bias

After the encoder bias $\mathbf{b}$ is introduced, Sections 3.2.1 to 3.2.3 tell us that the true joint angle $\mathbf{q}_{\text{true}}$, the true end-effector position $\mathbf{x}_{\text{ee, true}}$, and the bias $\mathbf{b}$ itself are all *invisible* to the agent. This degradation in the information structure is precisely the situation described by a partially observable Markov decision process (POMDP) [7]. We formalize the biased problem as the seven-tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma). \quad (3.13)$$

State space On top of the physical state of the baseline MDP, an additional bias component is appended:

$$\mathbf{s} = (\mathbf{s}^{\text{phys}}, \mathbf{b}), \quad \mathbf{s}^{\text{phys}} = (\mathbf{q}_{\text{true}}, \dot{\mathbf{q}}, g, \mathbf{x}_{\text{ee, true}}, \mathbf{x}_{\text{obj}}), \quad (3.14)$$

where $\mathbf{q}_{\text{true}}$ and $\mathbf{x}_{\text{ee, true}}$ are the physically true values, and $\mathbf{b} \in \mathbb{R}^7$ is the episode-level latent variable.

Observation space and observation function The observation space Ω is determined by the perception structure given in Section 3.2.3. The observation function O is a deterministic mapping²

$$\mathbf{o} = O(\mathbf{s}) = (\underbrace{\mathbf{q}_{\text{true}} + \mathbf{b}}_{\text{corrupted}}, \dot{\mathbf{q}}, g, \underbrace{\text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b})}_{\text{corrupted}}, \mathbf{x}_{\text{obj}}). \quad (3.15)$$

Note that the two position-type components in the observation—the joint angle and the end-effector position—are both corrupted by the same $\mathbf{b}$; the velocity-type and external-perception-type components $(\dot{\mathbf{q}}, g, \mathbf{x}_{\text{obj}})$ are unaffected. The reward R and the discount

²Here only the basic observation quantities affected by the bias are included, in order to focus the theoretical analysis; the augmented observation space that introduces an external end-effector position observation $\mathbf{x}_{\text{ee, true}} + \epsilon$ will be discussed in Section 3.4.

factor γ are the same as in the baseline MDP; the concrete form of the transition kernel T will be derived in the next subsection. Since $\mathbf{o} \neq \mathbf{s}$, the policy can only rely on a partial observation, taking the form $\pi(\mathbf{a} \mid \mathbf{o})$.

3.3.3 Episode-Level Static Property of the Latent Variable $\mathbf{b}$

The core observation of this section is: **$\mathbf{b}$ is not a hidden state that evolves step by step as in a classical POMDP, but an episode-level constant**— $\mathbf{b}$ is sampled once from a distribution $P(\mathbf{b})$ at the beginning of each episode, and then remains unchanged within the episode, and no action of the agent can change its value. This is essentially different from the time-varying hidden state of a classical POMDP: the latter requires the agent to track a belief through temporal observations, whereas $\mathbf{b}$, as an episode-level “environment parameter,” does not require temporal tracking. This static property brings two corollaries crucial to the methodology of this thesis: (1) *the bias information is consistent throughout the entire episode*, so in principle no recurrent memory mechanism is needed, and an appropriate observation augmentation can approximately restore the POMDP to an MDP at the level of information structure (see Section 3.4); (2) *the optimality of the policy is defined over the distribution $P(\mathbf{b})$* ,

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\mathbf{b} \sim P(\mathbf{b}), \tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(\mathbf{s}_t, \mathbf{a}_t) \right], \quad (3.16)$$

that is, the policy must achieve expected optimality over the bias distribution, which directly motivates the *randomized-bias training* of this thesis (see Section 4.3).

3.4 Observability Analysis

Section 3.3 has modeled biased robot manipulation as a POMDP, but there is still a key theoretical gap between “modeling” and “solving”: the policy $\pi(\mathbf{a} \mid \mathbf{o})$ can only make decisions based on a *single-step* observation $\mathbf{o}$, so can the true physical state $\mathbf{q}_{\text{true}}$ and the latent variable $\mathbf{b}$ be uniquely recovered from a single $\mathbf{o}$? The answer to this question determines whether the POMDP is “solvable” in terms of information structure.

This section gives two complementary conclusions: **Proposition 3.1** proves that relying only on the basic observation given by Equation (3.15) *cannot* distinguish different $(\mathbf{q}_{\text{true}}, \mathbf{b})$; **Proposition 3.2** then proves that if an external position observation independent of the encoder is further introduced, $(\mathbf{q}_{\text{true}}, \mathbf{b})$ can be uniquely recovered. These two propositions theoretically and precisely predict the success and the failure of the observation-space ablation experiment in Section 5.4.4.

3.4.1 Proposition 1: Non-Identifiability under the Basic Observation

Proposition 3.1 (Non-identifiability under the basic observation). *Let the observation function O of the POMDP be given by Equation (3.15):*

$$\mathbf{o} = O(\mathbf{s}) = (\mathbf{q}_{\text{true}} + \mathbf{b}, \dot{\mathbf{q}}, g, \text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b}), \mathbf{x}_{\text{obj}}).$$

For any fixed observation $\mathbf{o}$, there exist infinitely many pairs $(\mathbf{q}_{\text{true}}, \mathbf{b})$ producing that observation. Therefore, O is not injective in $(\mathbf{q}_{\text{true}}, \mathbf{b})$, and $(\mathbf{q}_{\text{true}}, \mathbf{b})$ cannot be uniquely determined from $\mathbf{o}$.

Proof. Consider the two components of the observation $\mathbf{o}$ related to $(\mathbf{q}_{\text{true}}, \mathbf{b})$: the joint reading $\mathbf{q}_{\text{true}} + \mathbf{b}$ and the end-effector estimate $\text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b})$. Both depend only on the sum $\mathbf{q}_{\text{true}} + \mathbf{b}$, and not on $\mathbf{q}_{\text{true}}$ or $\mathbf{b}$ individually.

Take any $\delta \in \mathbb{R}^7$, and construct another pair $(\tilde{\mathbf{q}}, \tilde{\mathbf{b}}) = (\mathbf{q}_{\text{true}} - \delta, \mathbf{b} + \delta)$; then

$$\tilde{\mathbf{q}} + \tilde{\mathbf{b}} = (\mathbf{q}_{\text{true}} - \delta) + (\mathbf{b} + \delta) = \mathbf{q}_{\text{true}} + \mathbf{b},$$

so $\text{FK}(\tilde{\mathbf{q}} + \tilde{\mathbf{b}}) = \text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b})$. The remaining observation components $(\dot{\mathbf{q}}, g, \mathbf{x}_{\text{obj}})$ do not depend on $(\mathbf{q}_{\text{true}}, \mathbf{b})$, and therefore remain unchanged under the transformation.

In summary, for any δ , the old and the new pairs $(\mathbf{q}_{\text{true}}, \mathbf{b})$ produce exactly the same observation $\mathbf{o}$. Since δ is arbitrary, there exist infinitely many pairs $(\mathbf{q}_{\text{true}}, \mathbf{b})$ compatible with a given $\mathbf{o}$. $\square$

Proposition 3.1 shows that there is a *translational symmetry* of arbitrary direction between $(\mathbf{q}_{\text{true}}, \mathbf{b})$: adjusting the true joint angle and the bias simultaneously along the direction δ leaves the observation *exactly identical*. The fundamental reason lies in the fact revealed in Section 3.2.3—the two position-type observation components, though seemingly independent, actually share the corrupted $\mathbf{q}_{\text{measured}}$, are always self-consistent with each other, and cannot provide any additional identifying information. Therefore, any policy that relies only on a single-step basic observation is in principle unable to distinguish different $(\mathbf{q}_{\text{true}}, \mathbf{b})$, and thus unable to make a targeted response to different biases.

3.4.2 Proposition 2: Observation Augmentation Restores Identifiability

The crux of Proposition 3.1 is that all position-type observations pass through the same corrupted encoder channel. To break the translational symmetry, a position observation *independent of the encoder* must be introduced. A natural choice is an external localization system—such as OptiTrack motion capture, or an ArUco visual marker together with an Intel RealSense D455, etc.—which does not depend on the robot’s own joint encoders and directly measures the true position of the end-effector in the world coordinate frame. Such a measurement can be uniformly modeled as

$$\mathbf{o}_{\text{real}} = \text{FK}(\mathbf{q}_{\text{true}}) + \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_3), \quad (3.17)$$

where σ characterizes the precision of the localization system. Appending this observation component to the basic observation function O yields the augmented observation space Ω' and the augmented observation function:

$$\mathbf{o}' = O'(s) = (\underbrace{\mathbf{q}_{\text{true}} + \mathbf{b}, \dot{\mathbf{q}}, g, \text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b}), \mathbf{x}_{\text{obj}}, \mathbf{o}_{\text{real}}}_{\mathbf{o}}). \quad (3.18)$$

Proposition 3.2 (Identifiability under the augmented observation). *Let the augmented observation function O' be given by Equation (3.18), and let the external localization be noise-free ($\epsilon = \mathbf{0}$). If FK is injective in a neighborhood of the current configuration $\mathbf{q}_{\text{true}}$ ³, then $(\mathbf{q}_{\text{true}}, \mathbf{b})$ can be uniquely determined from $\mathbf{o}'$.*

Proof. Under the condition $\epsilon = \mathbf{0}$, by Equation (3.17), $\mathbf{o}_{\text{real}} = \text{FK}(\mathbf{q}_{\text{true}})$. By the assumption that FK is locally injective, there exists a local inverse function FK^{-1} such that

$$\mathbf{q}_{\text{true}} = \text{FK}^{-1}(\mathbf{o}_{\text{real}}). \quad (3.19)$$

Since $\mathbf{q}_{\text{true}}$ is now known, and since $\mathbf{o}$ contains the component $\mathbf{q}_{\text{true}} + \mathbf{b}$, one can directly solve for

$$\mathbf{b} = (\mathbf{q}_{\text{true}} + \mathbf{b}) - \mathbf{q}_{\text{true}}. \quad (3.20)$$

Therefore $(\mathbf{q}_{\text{true}}, \mathbf{b})$ is uniquely determined by $\mathbf{o}'$. $\square$

Compared with Proposition 3.1, the key difference of Proposition 3.2 is that $\mathbf{o}_{\text{real}}$ bypasses the corrupted encoder channel and directly provides an “independent observation” that depends only on $\mathbf{q}_{\text{true}}$ (rather than $\mathbf{q}_{\text{true}} + \mathbf{b}$), thereby *breaking* the translational symmetry used in the proof of Proposition 3.1. From an information-theoretic perspective, $\mathbf{o}_{\text{real}}$ is linearly independent of the existing position-type observations, and exactly compensates for the previously missing one-dimensional identifying direction. Its geometric intuition is shown in Figure 3.3.

3.4.3 Identifiability $\neq$ Solvability

Proposition 3.2 is a conclusion at the *information-theoretic* level: the augmented observation $\mathbf{o}'$ can in principle distinguish different $(\mathbf{q}_{\text{true}}, \mathbf{b})$. But this is not equivalent to “the problem has returned to the bias-free case”—the bias has not *disappeared*, it has only changed from *invisible* to *inferable*. The control bias proved in Section 3.2.2 and the initial position offset of Section 3.2.1 still exist: the physical transition kernel T_{phys} of the POMDP is *exactly the same* before and after the introduction of $\mathbf{o}_{\text{real}}$, and what observation augmentation changes is the *information structure*, not the *dynamics*.

³For a 7-DoF redundant manipulator such as the Franka Panda, FK is not globally injective (there is a 1-DoF null space). However, within the actual operating region and away from singular configurations, FK is locally injective; combining it with the velocity observation $\dot{\mathbf{q}}$ can further eliminate the null-space ambiguity. The discussion of this section holds under this assumption.

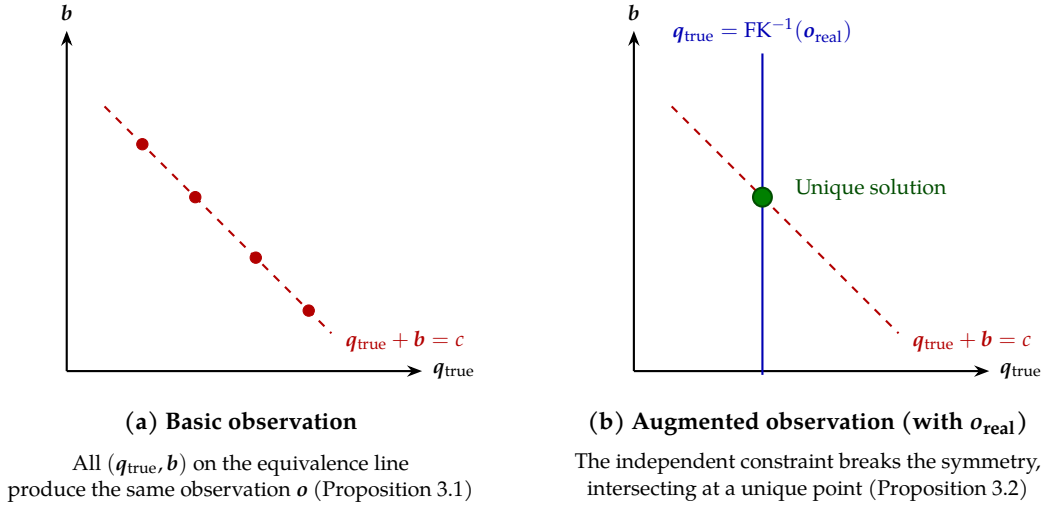


Figure 3.3: Geometric illustration of identifiability. The horizontal axis q_{true} and the vertical axis b are an abstract one-dimensional representation. (a) When relying only on the basic observation, all pairs (q_{true}, b) satisfying $q_{\text{true}} + b = c$ (red dashed line) produce the same observation o ; the translational symmetry along the direction δ makes the original problem non-identifiable. (b) After introducing an independent external position observation, by Equation (3.19), q_{true} is locked to $\text{FK}^{-1}(o_{\text{real}})$ (blue solid line); the intersection of this constraint with the equivalence line is unique (green dot), i.e., the identifiable solution given by Proposition 3.2. When $\sigma > 0$, the blue line becomes a “band” of finite width, and uniqueness degenerates into a maximum-likelihood estimate over the posterior—this is precisely the geometric origin of the “averaging compensation” in Section 3.4.3.

Therefore, the rigorous meaning of “the POMDP is approximately restored to an MDP” is: there exists a family of policies $\pi(a \mid o')$ acting on o' whose Bellman-equation solution coincides with the optimal policy of the original POMDP; the MDP state here still contains b , and the optimal policy is a function *conditioned on b* :

$$\pi^*(a \mid o') = \pi^*(a \mid q_{\text{true}}, b). \quad (3.21)$$

For different b , π^* must output *different* compensating actions. In the realistic case of $\epsilon > 0$, since the noise makes it possible for different (q_{true}, b) to produce almost the same o' , the optimal action degenerates into the expected optimum over the posterior conditioned on the observation, that is, a kind of “**averaging compensation**”—the policy outputs a compromise action over all biases compatible with the observation.

This analysis brings two insights crucial to the subsequent chapters. *First*, for a single policy to cover the entire $P(b)$, it must be exposed to the diversity of the bias distribution during training—training only under a fixed b leads to “single-point conditioning”: the

policy learns the compensation amount under a specific bias, and *over-compensates* on other biases (especially the fault-free case $\mathbf{b} = \mathbf{0}$). This is precisely the theoretical basis of the *randomized-bias training* of Section 4.3. *Second*, observation augmentation provides the *feasibility* of training but does not guarantee its *sufficiency*—whether “information-theoretically identifiable” can be turned into “actually learnable” still depends on the data distribution, the reward density, and the optimization capability. The fixed-bias experiment of Section 5.4.3 will provide direct empirical support: a policy trained under a fixed $b_i = 0.2$ rad on all joints has a significantly lower success rate in the zero-bias test than a policy obtained by randomized-bias training, which is precisely the over-compensation phenomenon caused by single-point conditioning (the concrete numbers are given in Section 5.4.3).

3.4.4 Discussion: Noise, Configuration Information, and Correspondence with Experiments

Influence of localization noise Proposition 3.2 assumes $\epsilon = 0$. In the practical case of $\sigma > 0$, $\mathbf{q}_{\text{true}}$ and $\mathbf{b}$ can only be *approximately* recovered: as $\sigma \rightarrow 0$ the identification precision tends to be perfect; as σ increases the identification error increases accordingly; in the extreme, if σ is so large that the signal-to-noise ratio of $\mathbf{o}_{\text{real}}$ falls below the typical amplitude of $\mathbf{b}$ itself, the external observation loses its effect, and the system degenerates back to the non-identifiable case described by Proposition 3.1. The $\sigma = 5$ mm adopted in this thesis corresponds to the localization precision of an ArUco visual marker together with a D455 depth camera at a typical working distance, which is both easy to realize in practice and sufficient to support the robust response of the policy to the bias in Chapter 4.

Why the biased proprioception is still retained The biased $\mathbf{q}_{\text{measured}}$ still encodes the joint configuration of the arm—singularity proximity, joint-limit margin, the null-space direction in the 7-DoF redundancy, etc.—and this information cannot be provided by the three-dimensional $\mathbf{o}_{\text{real}}$ alone, which constitutes the necessity of retaining the proprioception channel. The complementary design-level reasons (information completeness and runtime fault tolerance) are given in Section 4.2.3.

Correspondence with the ablation experiment The two propositions of this section are not purely theoretical deductions; they directly predict the results of the observation-space ablation experiment in Section 5.4.4. The ablation experiment contrasts the full 24-dimensional observation used in Chapter 4 with an 18-dimensional variant that *removes* the independent external TCP $\mathbf{o}_{\text{real}}$ —the former corresponds to the identifiable case of Proposition 3.2, where the policy should remain robust over the full bias range; the latter corresponds to the non-identifiable case of Proposition 3.1, where the policy performance is expected to degrade severely under bias. This correspondence both provides a verifiable theoretical basis for the observation-space design of this thesis, and

conversely confirms that the two propositions are not a mere formal exercise, but an accurate characterization of the essence of the problem.

3.5 Simulation Implementation and Equivalence Analysis

The analysis of the previous four sections has all unfolded at the abstract level: Section 3.2 gave the physical causal chain of bias propagation in control and observation, Section 3.3 formalized it as a POMDP, and Section 3.4 analyzed the identifiability at the level of information structure. The task of this section is to *ground* these abstract conclusions—to faithfully reproduce, in a MuJoCo-based simulation environment, the three effects described in Section 3.2, and to rigorously argue that the simulation implementation is equivalent to the original physical mechanism in the relevant observation quantities.

Why the `/joint_states` injection cannot simply be reused In a ROS environment, a common fault-injection practice is to directly add a bias to the joint angles published on the `/joint_states` topic [22]. This is concise and effective in the standard ROS architecture where “the controller subscribes to `/joint_states` for feedback,” because once the bias enters the topic it naturally propagates to all downstream readers. However, neither the simulation nor the real robot of this thesis *conforms* to this premise: the OSC controller in the simulation directly reads the joint angles from MuJoCo’s `data.qpos`, with no `/joint_states` topic as an intermediary at all; the impedance controller of the HIL-SERL architecture on the real robot reads the robot state through direct C++ memory access via the `FrankaStateHandle`, likewise bypassing ROS topic communication (see Chapter 6). In both cases, injecting the bias only on `/joint_states` cannot affect the controller or the observation module—an *equivalent injection scheme* decoupled from the coupling mode of the underlying data flow needs to be designed. This section first gives the implementation-independent sufficient conditions, and then describes the concrete implementation under MuJoCo on this basis.

3.5.1 Sufficient Conditions for Fault Injection

Proposition 3.3 (Sufficient conditions for fault injection). *Suppose a fault-injection implementation satisfies the following two conditions:*

- (C1) **Control condition:** *every time the control law computes the joint torque, the joint-angle value used equals $\mathbf{q}_{\text{true}} + \mathbf{b}$;*
- (C2) **Observation condition:** *every time the observation module returns the state, the joint-angle component equals $\mathbf{q}_{\text{true}} + \mathbf{b}$, and the end-effector pose component equals $\text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b})$.*

Then this implementation faithfully reproduces the three effects defined in Section 3.2—the initial position offset, the control bias, and the perception bias.

Proof. Effect 2 (control bias): the biased torque under the definition of Section 3.2.2 is $\tau^{\text{biased}} = \tau_{\text{osc}}(\mathbf{q}_{\text{true}} + \mathbf{b})$. Condition (C1) guarantees that the value used when the control law is computed is exactly $\mathbf{q}_{\text{true}} + \mathbf{b}$, so the output torque is exactly τ^{biased} , fully consistent with the definition of that subsection.

Effect 3 (perception bias): Section 3.2.3 requires the joint component in the observation $\mathbf{o}$ to be $\mathbf{q}_{\text{true}} + \mathbf{b}$ and the end-effector pose component to be $\text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b})$, which is exactly the content of condition (C2).

Effect 1 (initial position offset): the steady-state condition $\mathbf{q}_{\text{true}} = \mathbf{q}_{\text{home}} - \mathbf{b}$ of Section 3.2.1 is a direct corollary of applying condition (C1) to the low-level PD controller—when the PD controller tracks the target with $\mathbf{q}_{\text{target}} = \mathbf{q}_{\text{home}}$ and uses $\mathbf{q}_{\text{true}} + \mathbf{b}$ as feedback, the steady state must stop at $\mathbf{q}_{\text{true}} = \mathbf{q}_{\text{home}} - \mathbf{b}$. If the low-level PD controller is not explicitly run in the simulation (see Section 3.5.2), directly initializing the joint position with $\mathbf{q}_{\text{true}} = \mathbf{q}_{\text{home}} - \mathbf{b}$ achieves the same steady-state effect. $\square$

Proposition 3.3 abstracts the question of “whether the fault injection is faithful” into two *architecture-independent* conditions: regardless of whether the lower layer is implemented through a ROS topic, C++ memory access, or the simulation’s data structure, as long as (C1) and (C2) hold, the injection is equivalent. This provides a unified correctness basis for the simulation implementation of this section and the real-robot B+D dual-injection scheme of Chapter 6.

3.5.2 Concrete Implementation in MuJoCo

The simulation of this thesis is built on MuJoCo 3.0, with a robot control cycle of 10 Hz and an embedded 500 Hz physics simulation. The lower layer does not run an explicit PD servo, but directly uses the current `data.qpos` as the true joint state; the upper-level OSC controller obtains the Jacobian and the end-effector position by calling `mujoco.mj_jacSite` and `data.site_xpos`. We implement the three effects respectively according to the two conditions of Proposition 3.3.

Effect 1: Implementation of the Initial Position Offset

In the `reset_robot` stage, we do not explicitly simulate the transient of the low-level PD controller, but directly set MuJoCo’s joint position to the offset steady-state value:

$$\mathbf{q}_{\text{data.qpos}}^{\text{reset}} \leftarrow \mathbf{q}_{\text{home}} - \mathbf{b}. \quad (3.22)$$

By Equation (3.4), this is exactly the steady-state end point of the low-level PD controller when the feedback is corrupted by the bias. This simplification skips the tens to hundreds of milliseconds of PD transient on the real robot, but is strictly consistent with the real robot at the *steady-state end point*. Since this thesis is concerned with the task execution within an episode (rather than the transient during initialization), this simplification has no influence on the subsequent analysis.

Effect 2: Implementation of the Control Bias

Condition (C1) requires the OSC to use $\mathbf{q}_{\text{true}} + \mathbf{b}$ rather than $\mathbf{q}_{\text{true}}$ when computing the torque. Since `ospace()` directly reads the joint state and the Jacobian from `data.qpos` and the `mj_forward` result derived from it, we adopt a three-step pattern of “**temporary replacement—computation—restoration**”: at the beginning of each physics substep, the arm joint components in `data.qpos` are first replaced with $\mathbf{q}_{\text{true}} + \mathbf{b}$ and `mj_forward` is called to update the internal derived quantities; then `ospace()` is called to compute the torque; then `data.qpos` is restored to the true $\mathbf{q}_{\text{true}}$ and `mj_forward` is called again; and finally the computed torque is written into `data.ctrl` and the physics simulation is driven by `mj_step`. The complete procedure is given in Algorithm 1.

Algorithm 1 A single control step with encoder bias (N physics substeps)

Require: true joint position $\mathbf{q}_{\text{true}}$, desired end-effector pose $\mathbf{x}_{\text{target}}$, bias $\mathbf{b}$

- 1: **for** $k = 1$ **to** N **do**
- 2: $\mathbf{q}_{\text{save}} \leftarrow \text{data.qpos}$ ▷ back up the true joint position
- 3: $\text{data.qpos} \leftarrow \mathbf{q}_{\text{save}} + \mathbf{b}$ ▷ inject the bias
- 4: `mj_forward(model, data)` ▷ update derived quantities (Jacobian/FK)
- 5: $\boldsymbol{\tau} \leftarrow \text{ospace}(\text{data}, \mathbf{x}_{\text{target}})$ ▷ compute torque with the biased $\mathbf{q}$
- 6: $\text{data.qpos} \leftarrow \mathbf{q}_{\text{save}}$ ▷ restore the true joint position
- 7: `mj_forward(model, data)`
- 8: $\text{data.ctrl}[\text{arm}] \leftarrow \boldsymbol{\tau}$ ▷ the true system applies the biased torque
- 9: `mj_step(model, data)` ▷ advance the physics simulation
- 10: **end for**

The equivalence of this pattern relies on a simple observation: `ospace` is a *pure function* of the geometric quantities ($\mathbf{J}$, FK , $\boldsymbol{\Lambda}$) derived from `data.qpos` via `mj_forward`, and its output is fully determined by the `data.qpos` at the time of the call. Lines 3–5 of the algorithm guarantee that when `ospace` is called, the joint components are exactly $\mathbf{q}_{\text{true}} + \mathbf{b}$, so the returned torque equals exactly $\boldsymbol{\tau}_{\text{osc}}(\mathbf{q}_{\text{true}} + \mathbf{b})$ —which is precisely the requirement of condition (C1) of Proposition 3.3. The subsequent restoration operation does not change the torque value already computed (it has been saved to a local variable), and merely lets the physics simulation execute again based on the true state. Observed from the outside, this procedure corresponds exactly to the real-robot situation of “the OSC directly reads the corrupted encoder, outputs a biased torque, and the true physics evolves according to this torque.”

Effect 3: Implementation of the Perception Bias

Condition (C2) requires the observation module to return the corrupted joint angle and end-effector pose. In `get_robot_state()`, we directly construct $\mathbf{q}_{\text{measured}} = \mathbf{q}_{\text{true}} + \mathbf{b}$, and compute $\text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b})$ through one temporary replacement:

$$\mathbf{o}_{\text{joint}} = \mathbf{q}_{\text{true}} + \mathbf{b}, \quad \mathbf{o}_{\text{ee}} = \text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b}), \quad (3.23)$$

where $\text{FK}(\mathbf{q}_{\text{true}} + \mathbf{b})$ is obtained by the same trick of temporarily replacing `data.qpos` with $\mathbf{q}_{\text{true}} + \mathbf{b}$, calling `mj_forward`, reading `data.site_xpos`, and then restoring the true value, consistent with the replace-restore pattern of Algorithm 1, with correctness guaranteed by the same “pure function dependency” argument.

3.5.3 External Sensor Simulation

In addition to the bias-corrupted observation components above, Proposition 3.2 of Section 3.4 also requires the introduction of an external position observation $\mathbf{o}_{\text{real}}$ independent of the encoder. In the simulation we model it as

$$\mathbf{o}_{\text{real}} = \mathbf{x}_{\text{ee,true}} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_3), \quad \sigma = 5 \text{ mm}, \quad (3.24)$$

where $\mathbf{x}_{\text{ee,true}}$ is read directly from MuJoCo’s true FK (i.e., the value of `data.site_xpos` under the true `data.qpos`, without bias), and $\boldsymbol{\epsilon}$ is an isotropic Gaussian noise independent of the time step and the bias. As already discussed in Section 3.4, $\sigma = 5 \text{ mm}$ corresponds to the external localization precision of an ArUco visual marker together with an Intel RealSense D455 at a typical working distance. The position of the task target object (such as a block) is assumed to be measured by an overhead external camera, is not affected by the encoder bias, and is read directly from MuJoCo’s object sensor in the simulation.

3.5.4 Main Gaps between Simulation and Reality

The MuJoCo simulation described in this section is an *idealized approximation* of the real industrial robot system, with the main gaps including: (i) Effect 1 skips the transient of the low-level PD controller, whereas on the real robot the 1 kHz servo driven by `libfranka` has a finite response time of tens to hundreds of milliseconds; (ii) the simulation uses the OSC control law, whereas the real-robot HIL-SERL architecture uses a Cartesian impedance controller, and the equivalence of the two will be rigorously analyzed in Section 6.2.3; (iii) MuJoCo’s rigid-body dynamics do not include real effects such as joint friction, motor delay, and cable influence; (iv) this thesis only considers fixed and random bias, and does not yet model time-varying bias types such as temperature drift and electromagnetic noise.

These gaps *all point in the direction of making the bias effect weaker in the simulation*—friction and delay would further worsen the influence of the bias on the control precision

on the real robot, and time-varying bias would increase the identification difficulty on the real robot. Therefore the simulation can be regarded as a conservative test platform: if a method can cope with the bias in the simulation, then coping with it is *even more necessary* on the real robot, and the effect is usually *even more significant*. The complete 8-item comparison table and the concrete implementation of the real-robot injection scheme will be given in Chapter 6, and are not elaborated on here.

4 Task Policy: Method Design and Shared Network Architecture

4.1 Motivation of the Method Design

Chapter 3 characterized the structure of the encoder bias problem through the physical causal chain and the POMDP formalization, and gave, through the observability analysis, a sufficient condition for “ $\mathbf{b}$ being information-theoretically identifiable.” This section turns these theoretical conclusions into three concrete method-design choices, and briefly explains the basis of each choice and the corresponding sections.

Design choice 1: observation-space augmentation According to Proposition 3.1, relying only on the basic proprioceptive observation—in which the joint-angle reading $\mathbf{q} + \mathbf{b}$ and the TCP position $\text{FK}(\mathbf{q} + \mathbf{b})$ derived from it via forward kinematics are both corrupted by the bias, while the joint velocity (the time derivative of the encoder reading, in which the static bias $\dot{\mathbf{b}} = \mathbf{0}$ cancels out) and the gripper opening (which comes from the control command rather than the encoder) are unaffected by the bias—*cannot* distinguish different pairs $(\mathbf{q}_{\text{true}}, \mathbf{b})$; whereas Proposition 3.2 shows that introducing an external position observation $\mathbf{o}_{\text{real}}$ independent of the encoder can break the translational symmetry between $(\mathbf{q}_{\text{true}}, \mathbf{b})$ and make the system approximately identifiable. Therefore, the first design choice of this thesis is to **explicitly add an external position signal to the observation**, extending it to a 25-dimensional observation space (containing the independent external TCP and the hand state, among other components). The concrete implementation is given in Section 4.2.

Design choice 2: randomized-bias training Section 3.4.3 further points out that even when the information is sufficient, the optimal policy is still a function *conditioned on the bias*: for different $\mathbf{b}$, π^* must output different compensating actions (see Equation (3.21)). If the training process is exposed only to a single fixed $\mathbf{b}$, the policy can only learn “single-point conditioning” near that point, and produces over-compensation at other points of the bias distribution (especially at $\mathbf{b} = \mathbf{0}$). Therefore, the second design choice of this thesis is to **train by randomly sampling the bias per episode from the bias distribution $P(\mathbf{b})$** , exposing the policy to the entire bias distribution. This idea is in principle cognate with the domain randomization adopted by Tobin et al. in visual sim-to-real [10]—both “let the policy see the distribution of the future deployment environment during training.” Section 4.3 will describe randomized-bias training in detail, and the

fixed-bias baseline experiment of Section 5.4.3 will provide direct empirical evidence of the over-compensation of single-point conditioning.

Design choice 3: the dual-end paradigm of simulation validation and real-robot deployment The first two choices answer “what to observe” and “what distribution to train on,” but still leave the execution-level question of “where to train and with what algorithm.” This thesis adopts a dual-end paradigm of **validating the theory in simulation and deploying by imitation on the real robot**, jointly determined by the three factors of *observation modality, data scale, and safety constraint*.

On the simulation side, the HIL-SERL framework of Luo et al. [15] (based on the RLPD of Ball et al. [13] with human-in-the-loop intervention) is adopted to validate the observability propositions and the randomized-bias robust training (the H1–H3 experiments), with the concrete training protocol given in Section 5.2.

On the real-robot side, because under the small-data condition of a sparse reward and a 50–200 demo scale the actor drifts substantially once it is unfrozen in the SAC online stage (a thesis-level negative result, detailed in Section 6.3.5), we switch to the HG-Dagger of Kelly et al. [12] (the idea of human-intervention imitation learning originates from the query-by-policy paradigm of the DAgger of Ross et al. [11]; both HIL-SERL and HG-Dagger are human-initiated variants): an initial policy obtained by BC pretraining is then corrected by operator intervention, with the concrete training protocol given in Section 6.3. The two ends are consistent at the shared method level (25-dimensional observation + shared network architecture + sparse reward), and differ only in the training algorithm and the concrete protocol.

Simplification of the real-robot observation and future work. The real-robot deployment observation is further simplified to 14 dimensions (detailed in Section 6.3.1), in which the encoder-bias path does not yet enter the observation channel, and the TCP signal comes uniformly from an external independent channel—corresponding to the “success signal path” of Proposition 3.2 already validated in simulation. This simplification lets the first real-robot run focus on the sim-to-real deployment pipeline and the end-to-end success-rate validation under fault injection (the 4-column success-rate matrix of the three tasks jointly with the backup in Section 6.5); the bias-robustness loop at the real-robot level—feeding both the “corrupted encoder signal” and the “external position signal” into real-robot training simultaneously—is left as the two-step future-work ablation of Chapter 7.

Summary The three design choices above can be concisely mapped to the “theory–constraint–method” correspondence shown in Table 4.1. The subsequent sections develop the concrete implementation in this order.

Table 4.1: Mapping from the theoretical analysis of Chapter 3 and the practical constraints to the method design of this chapter

Source	Concrete basis	Method design	Section
Theoretical conclusion	Proposition 3.2 (identifiability)	Observation-space augmentation (25D)	Section 4.2
Theoretical conclusion	Section 3.4.3 (conditional compensation)	Randomized-bias training	Section 4.3
Practical constraint	Visual sim-to-real gap	Real-robot deployment and the IL path	Section 6.3
Practical constraint	Sparse-reward exploration difficulty (simulation side)	HIL-SERL (demo + HIL bootstrap)	Section 5.2
Practical constraint	Real-robot sparse + small-data SAC drift	BC pretrain + HG-Dagger	Section 6.3

4.2 Observation Space Design

4.2.1 Composition of the Observation Vector

The low-dimensional state observation vector $\mathbf{o}_t \in \mathbb{R}^{25}$ of the task policy is formed by the sequential concatenation of seven groups of components (Table 4.2): the first 18 dimensions come from the robot’s proprioceptive path (encoder readings and control commands), the middle 3 dimensions are the independent external TCP position, and the last 4 dimensions are the hand state, used to switch at runtime to the backup avoidance policy of Section 5.3. The noise modeling of $\mathbf{o}_{\text{real}}$ and the abstraction of the TCP localization scheme are given in Section 4.2.4.¹

Table 4.2 also annotates two orthogonal properties of each component—*whether it is corrupted by the bias $\mathbf{b}$ and its relevance to the task goal*. The latter is divided into four classes: the **directly relevant** ones ($\mathbf{o}_{\text{real}}$ and $\text{FK}(\mathbf{q} + \mathbf{b})$) directly participate in the pose decision; the **indirectly relevant** joint angle $\mathbf{q} + \mathbf{b}$ is used only when joint limits or geometric constraints are triggered; the **structural** joint velocity and gripper opening mainly serve the smoothness and state closure of the control loop; and the **runtime-safety-relevant** $\mathbf{h}_{\text{active}}$ and $\mathbf{p}_{\text{hand}}$ do not enter the task reward gradient—their role is to let the task policy and the backup avoidance policy of Section 5.3 share the same perception pipeline, and to be used at runtime by the distance criterion as the basis for policy switching (although the task policy can “see” the hand, it does not thereby learn avoidance; the avoidance behavior is borne entirely by the backup policy after the switch). This classification reveals the core design idea of this section: among the two directly task-relevant position quantities, $\text{FK}(\mathbf{q} + \mathbf{b})$ is corrupted and $\mathbf{o}_{\text{real}}$ is uncorrupted—it is precisely this set of

¹At the simulation-implementation level the environment interface is a 31-dimensional default implementation, in which the block position $\mathbf{o}_{\text{block}}$ and the target-plate position $\mathbf{o}_{\text{plate}}$ are reset per episode to deterministic positions in the experimental scenarios of this chapter, and are constant channels for the policy. To avoid giving the reader the impression that this method depends on the ground-truth object positions, and to align with the simplified real-robot observation (see Section 6.3), this thesis describes the observation space in the 25-dimensional form with the two constant channels removed—the gradient of the constant dimensions is always zero, and the quantitative results are equivalent to the original 31-dimensional implementation (provided that the input normalization treats the constant channels specially: this work guarantees, through the const-channel guard in `compute_dataset_stats.py`, that the min-max normalization does not produce a NaN on the constant channels, see Section 6.3.4).

directly relevant and mutually inconsistent position signals that provides the policy with the information source for implicitly identifying $\mathbf{b}$.

Table 4.2: Composition of the 25-dimensional state observation of the task policy

Index	Component	Dim	Source	Bias-corrupted	Task relevance
[0, 6]	Joint angle $\mathbf{q} + \mathbf{b}$	7	Encoder reading	Yes	Indirect
[7, 13]	Joint velocity $\dot{\mathbf{q}}$	7	Time difference of encoder reading	No	Structural
14	Gripper opening	1	Control command	No	Structural
[15, 17]	TCP position FK($\mathbf{q} + \mathbf{b}$)	3	Biased forward kinematics	Yes	Direct
[18, 20]	Real TCP $\mathbf{o}_{\text{real}}$	3	External localization + 5 mm noise	No	Direct
21	Hand activation h_{active}	1	Visual detection pipeline	No	Runtime safety
[22, 24]	Hand position $\mathbf{p}_{\text{hand}}$	3	Visual detection + D455 depth	No	Runtime safety

4.2.2 Information Paths

Figure 4.1 illustrates the flow of the 25-dimensional observation along two independent paths (the corruption marks are shown by the red labels in the figure). It should be noted that the 25 dimensions can be further divided by role into an *identification-relevant* part and a *runtime-safety* part: the former contains all 18 dimensions of the encoder path plus the external independent TCP $\mathbf{o}_{\text{real}}$ (21 dimensions in total), and is the entire basis of the identifiability argument required by Proposition 3.2—a single independent external position signal suffices to break the translational symmetry of $(\mathbf{q}_{\text{true}}, \mathbf{b})$; the latter contains the hand activation h_{active} and the hand position $\mathbf{p}_{\text{hand}}$, 4 dimensions in total, and serves only the distance-triggered switching between the task policy and the backup policy, not participating in the identifiability argument. The joint observation along the two paths makes the system satisfy the sufficient condition for approximate identifiability—in principle, the policy can implicitly recover $\mathbf{b}$ by comparing the inconsistency between $\mathbf{q} + \mathbf{b}$ and $\mathbf{o}_{\text{real}}$, and accordingly output the conditional compensating action required by Section 3.4.3; its quantitative error is bounded by the noise $\epsilon = 5$ mm of $\mathbf{o}_{\text{real}}$ and the condition number $\kappa(\mathbf{J})$ of the manipulator Jacobian, and the concrete upper bound is not elaborated on in this thesis. This thesis imposes no explicit constraint on the estimation structure (such as a Kalman filter or a dedicated $\mathbf{b}$ -prediction head), and leaves it to end-to-end optimization to organize on its own.

4.2.3 Why the Corrupted Proprioception Is Retained

A natural question is: since $\mathbf{q} + \mathbf{b}$ and FK($\mathbf{q} + \mathbf{b}$) are already corrupted by the bias, why not directly remove them and use only $\mathbf{o}_{\text{real}}$ as the sole source of the TCP? This thesis chooses to retain both, based on two considerations.

Information completeness. According to the analysis of Section 3.4.3, the premise for the policy to be able to identify $\mathbf{b}$ is that it simultaneously holds the “corrupted encoder signal” and the “independent external position signal”—it is the *difference* between the

Fig. 4.1 Information Pathways in the 25-D Observation Space

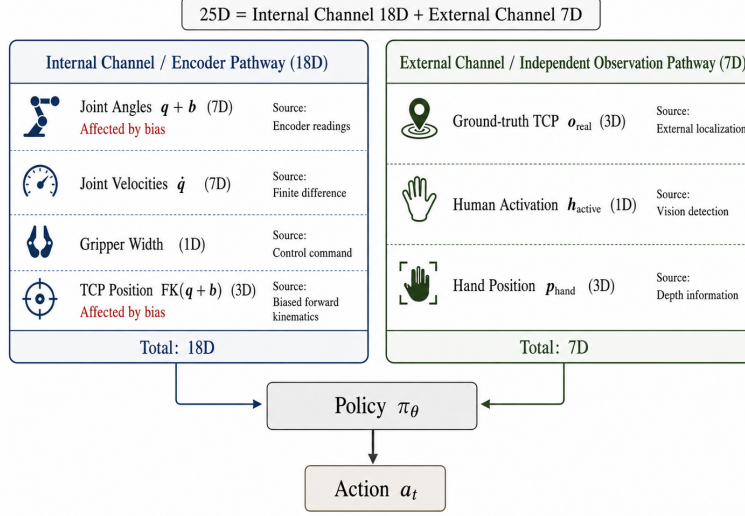


Figure 4.1: Illustration of the information paths of the 25-dimensional observation space. In the encoder path (left, 18D), the joint angle and the TCP position derived from it are corrupted by the bias b , while the joint velocity and the gripper opening are unaffected; the external observation path (right, 7D) contains the independent TCP and the hand information, all decoupled from the encoder. The two signal paths converge at the policy network, and in principle the policy can implicitly recover b from the inconsistency between $q + b$ and o_{real} .

two that carries the bias information. If the former is removed, the policy is left with only the single path o_{real} , and in fact loses the other half of the observation needed to infer b .

Runtime fault tolerance. A real-robot external localization system (ArUco/AprilTag, OptiTrack, etc.) may experience brief occlusion, marker loss, or communication jitter; retaining the proprioception path means that when o_{real} is temporarily unavailable, the policy can still fall back to biased behavior that relies on proprioception alone, rather than failing completely.

4.2.4 Generalized Treatment of Image Observation and TCP Localization

Image observation. The task policy additionally receives two RGB image streams from a front-view camera (global scene view) and a wrist camera (local gripper view), which are encoded by a *frozen* DINOv3-S (ViT-S/16, about 22 M frozen parameters, self-supervised pretrained on LVD-1689M) [23] and then share the backbone with the 25-dimensional state observation (detailed in Section 4.4). The significant gap between the simulated rendering and the real camera in texture, lighting, and noise distribution

is precisely the direct basis on which Section 4.1 argued that “real-robot deployment requires continued learning on the real robot.”

Noise modeling of real_tcp . In simulation training, $\mathbf{o}_{\text{real}}$ is obtained by superimposing $\mathcal{N}(\mathbf{0}, (0.005 \text{ m})^2 \mathbf{I})$ Gaussian noise on the MuJoCo scene ground truth. The 5 mm scale matches the order of the measured median error of the real-robot ArUco calibration of this thesis (the concrete calibration procedure is given in Chapter 6), in order to reduce the performance degradation caused by unmodeled noise during sim-to-real transfer.

Abstraction of the TCP localization scheme. This chapter does not bind $\mathbf{o}_{\text{real}}$ to a concrete implementation—the optional schemes include ArUco or AprilTag visual markers, OptiTrack optical motion capture, etc., as long as the path satisfies three minimal requirements: it is independent of the joint encoders, it can give the TCP position in metric units, and its typical noise scale is comparable to 5 mm. The concrete selection and calibration details are given in Chapter 6.

4.3 Randomized-Bias Training

Section 3.4.3 pointed out that even when the observation space already satisfies the identifiability condition, the optimal policy must still output different compensating actions conditioned on the bias $\mathbf{b}$ (Equation (3.21)). If the training process exposes the policy only to a single fixed $\mathbf{b}_0$, the policy can at most learn “single-point conditioning” near that point—once the true bias of the deployment environment deviates from $\mathbf{b}_0$ (especially when it returns to the calibrated state $\mathbf{b} = \mathbf{0}$), the policy over-corrects along the compensation direction learned during training, producing an *over-compensation* phenomenon. This thesis circumvents this problem by randomly sampling the bias per episode.

Training protocol. The bias is treated as an episode-level environment parameter: at each reset, $b_i \sim \mathcal{U}[0, b_{\text{max}}]$ ($i = 1, \dots, 7$) is sampled independently for each joint, and remains constant within the episode until the next reset; this thesis takes $b_{\text{max}} = 0.25 \text{ rad}$ (about 14°), significantly larger than the typical bias magnitude of industrial robots surveyed in Section 3.1, so as to apply a relatively aggressive stress test to the policy. The sampled $\mathbf{b}$ acts on the encoder readings and the controller reference values inside the simulation environment in the fault-injection manner described in Section 3.5, and is *not provided to the policy as any form of additional input*—the policy can only implicitly perceive the bias through the 25-dimensional observation defined in Section 4.2.

Connection with domain randomization. This practice is in principle cognate with the domain randomization proposed by Tobin et al. [10]: both rely on the basic idea of “letting the policy see samples of the target deployment distribution during training.”

But an essential difference must be pointed out: the visual parameters randomized by Tobin et al. are *redundant variables* for the policy (they should be ignored), whereas the $\mathbf{b}$ randomized in this thesis is a latent variable on which the policy *must condition*—according to the conditional-compensation requirement of Section 3.4.3, the optimal policy must output different actions for different $\mathbf{b}$. The two are opposite information-theoretic paradigms, cognate only in the surface idea of “having seen the deployment distribution during training”; the randomization goal of this thesis is to let the policy learn to implicitly recover and condition on the latent variable from the observation, rather than to acquire an invariance to it. Another difference is that randomized-bias training optimizes the *expected* return over the bias distribution rather than the worst-case return in the sense of a robust MDP—this thesis is concerned with the average performance across real-robot instances, not the guaranteed performance in the worst case.

Experimental preview of the fixed-bias baseline. To directly validate the hypothesis “single-point conditioning $\rightarrow$ zero-bias over-compensation,” Section 5.4.3 designs a controlled experiment of two training groups, fixed bias and random bias: under the same network structure and training hyperparameters, the two groups of policies are exposed respectively to $b_i = 0.2$ rad (fixed, for all $i = 1, \dots, 7$) and $b_i \sim \mathcal{U}[0, 0.25 \text{ rad}]$ (random), and are then evaluated for grasping success rate respectively at $\mathbf{b} = \mathbf{0}$ and at several bias values not seen during training, so as to characterize the generalization difference of the two training schemes over the bias distribution.

The concrete forms of the network architecture and the training hyperparameters are given uniformly in Section 4.4, applicable to both the simulation and the real-robot paths of this chapter.

4.4 Network Architecture and Training Objective

The task policy described in this chapter shares the same network architecture at both the simulation and the real-robot ends—a *shared encoder*, three *independent network heads*, and the corresponding *training-loss definitions*. This section gives these concrete forms in turn (Figure 4.2); the concrete training protocols (demo warmup, online mixed sampling, HG-Dagger iteration, etc.) are given respectively in Section 5.2 (simulation HIL-SERL) and Section 6.3 (real-robot BC + HG-Dagger).

Input encoding. The processing pipeline of the three input streams is drawn in full in Figure 4.2; here only the key points are stated: the simulation 25-dimensional (real-robot 14-dimensional) state observation is min-max normalized per dimension to $[-1, 1]$ and then projected by Linear + LN + Tanh to 256 dimensions; the 128×128 front-view and wrist RGB images are each normalized by the ImageNet mean and standard deviation and then fed into the *frozen* DINOv3-S backbone [23] (ViT-S/16, about 22 M frozen

4 Task Policy: Method Design and Shared Network Architecture

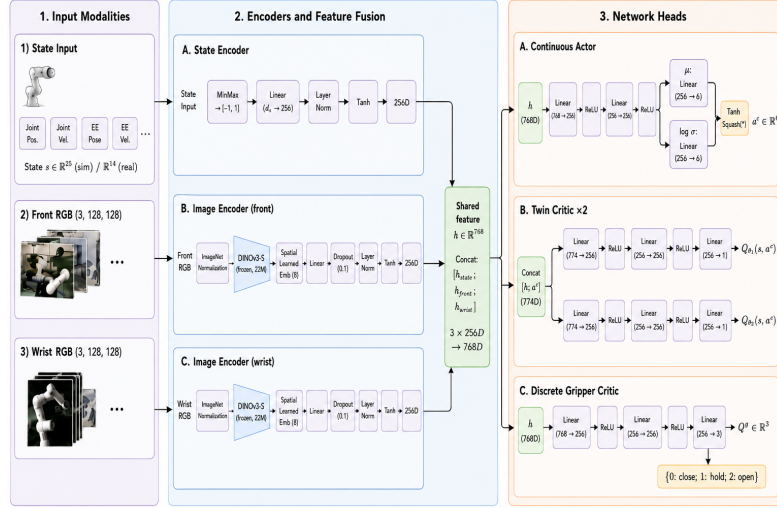


Figure 4.2: Network architecture of the task policy. The three input streams (simulation 25-dimensional / real-robot 14-dimensional state + front-view and wrist 128×128 RGB) are each processed by their own encoder and then concatenated channel-wise into a 768-dimensional shared feature h ; the three heads share h as input but have different outputs and training losses—the continuous Actor produces a 6-dimensional Cartesian end-effector action, the Twin Critic outputs two scalar Q-values (the minimum is taken to suppress overestimation), and the discrete Gripper Critic outputs a 3-dimensional Q-value to select the gripper action. The DINOv3-S backbone remains frozen during training, and only the subsequent projection layers, the state encoder, and the three heads are updated. Simulation HIL-SERL trains all four losses, whereas the real-robot BC stage trains only the continuous Actor (the critic and the temperature parameter serve as ckpt-schema placeholders).

parameters), and the output patch-token sequence is reshaped into a $384 \times 8 \times 8$ spatial feature map ($128/16 = 8$ patch grid), then passed through SpatialLearnedEmbeddings with $F = 8$ learnable spatial kernels for a channel-wise weighted summation—each (c, f) pair (384×8 in total) outputs one scalar $\sum_{h,w} \text{feat}_{c,h,w} \cdot \text{kernel}_{c,h,w,f}$ —yielding a $C \cdot F = 3072$ -dimensional vector, which is finally compressed to a 256-dimensional latent through Dropout(0.1) + Linear($3072 \rightarrow 256$) + LN + Tanh. Relative to global average pooling, this spatial-pooling design retains the discriminative power of F independent spatial patterns (avoiding merging “block in the upper left vs. lower right” into the same representation); relative to spatial softmax, it can learn a complex attention pattern rather than a single keypoint. The three 256-dimensional features are concatenated channel-wise into the 768-dimensional shared feature $\mathbf{h}$. The DINOv3-S backbone parameters are not updated during either the simulation or the real-robot training.

The three network heads. The three heads all use a two-layer MLP as their main body (hidden dimensions [256, 256], activation function ReLU), but their inputs, outputs, and training losses differ. The *continuous Actor* takes the shared feature $\mathbf{h}$ as input, and after a [768, 256, 256] MLP, attaches two Linear(256 $\rightarrow$ 6) heads that output the mean μ and the log standard deviation $\log \sigma$ respectively, and samples a 6-dimensional continuous action $\mathbf{a}^c \in \mathbb{R}^6$ (corresponding to a 3-dimensional end-effector position increment and a 3-dimensional end-effector orientation increment) through a TanhMultivariateNormalDiag distribution. The *Twin Critic* contains two independently initialized Q-networks $Q_{\theta_1}, Q_{\theta_2}$, each taking $[\mathbf{h}; \mathbf{a}^c]$ as input and outputting a scalar $Q \in \mathbb{R}$ through a [774, 256, 256, 1] MLP, with the minimum of the two taken during training to suppress value overestimation (Clipped Double Q-Learning). The *discrete Gripper Critic* takes the shared feature as input and outputs a 3-dimensional Q-value vector through a [768, 256, 256, 3] MLP, corresponding to the three discrete gripper actions numbered {0:close, 1:hold, 2:open} in the code, with arg max taken as the greedy policy during sampling. The temperature parameter α is parameterized by a learnable $\log \alpha$ and automatically tuned (initial value $\alpha_0 = 0.01$), and is assigned to an independent optimizer parameter group.

Training objective. The continuous branch follows the standard three losses of SAC (the background of entropy regularization and automatic temperature tuning is given in Chapter 2):

$$\begin{aligned} L_Q &= \mathbb{E}_{(s, \mathbf{a}, r, s') \sim \mathcal{B}} \left[(Q_{\theta}(s, \mathbf{a}) - y)^2 \right], \quad y = r + \gamma(1 - d) \left[\min_i Q_{\bar{\theta}_i}(s', \mathbf{a}') - \alpha \log \pi_{\phi}(\mathbf{a}' | s') \right], \\ L_{\pi} &= \mathbb{E}_{s \sim \mathcal{B}, \mathbf{a} \sim \pi_{\phi}} \left[\alpha \log \pi_{\phi}(\mathbf{a} | s) - \min_i Q_{\bar{\theta}_i}(s, \mathbf{a}) \right], \\ L_{\alpha} &= -\mathbb{E} \left[\alpha (\log \pi_{\phi}(\mathbf{a} | s) + \bar{\mathcal{H}}) \right], \end{aligned}$$

where $\mathbf{a}' \sim \pi_{\phi}(\cdot | s')$, $\bar{\mathcal{H}}$ is the target entropy, and $\bar{\theta}_i$ are the target-network parameters. The discrete gripper head is trained independently with an MSE Bellman loss in the form of Double DQN:

$$L_{Q^g} = \mathbb{E}_{(s, \mathbf{a}^g, r, s') \sim \mathcal{B}} \left[\left(Q_{\psi}^g(s, \mathbf{a}^g) - y^g \right)^2 \right], \quad y^g = r + \gamma(1 - d) Q_{\bar{\psi}}^g \left(s', \arg \max_{\mathbf{a}'} Q_{\bar{\psi}}^g(s', \mathbf{a}') \right),$$

where $\mathbf{a}^g \in \{\text{close, hold, open}\}$ (i.e., the three discrete gripper actions corresponding to the code numbers {0, 1, 2} in Figure 4.2 and the previous paragraph). During simulation HIL-SERL training, the gradients of all four losses are backpropagated to their respective head parameters and the trainable part of the shared encoder (the DINOv3-S backbone remains frozen), and the four parameter groups are updated synchronously by their respective Adam optimizers.

The real-robot BC pretraining stage uses only the single loss $L_{\text{BC}} = -\mathbb{E} [\log \pi_{\phi}(\mathbf{a} | s)]$, and the optimizer updates only the MLP head of the continuous Actor, the state encoder,

the SpatialLearnedEmbeddings, and the projection layers; the Twin Critic and the discrete Gripper Critic remain randomly initialized during BC, do not participate in gradient updates, and serve only as ckpt-schema placeholders so that a later path switch can directly -resume load them.

Pointers to the training protocols. The training protocols of sim and real (the concrete steps of demo warmup, online mixed sampling, HG-Dagger intervention iteration, etc.) are given respectively in Sections 5.2 and 6.3. The training hyperparameters shared by the two ends (learning rate, batch size, γ , τ , α_0 , image augmentation, etc.) are listed in a two-column comparison form in Table 4.3, and the protocol details are not repeated in this section.

Table 4.3: Task-policy training hyperparameters (simulation HIL-SERL vs. real-robot BC + HG-Dagger; “—” means not applicable)

Hyperparameter	Simulation HIL-SERL	Real-robot BC + HG-Dagger
Total training steps	10^5 online	20,000 per BC iter
Demo warmup steps (offline-only pretraining)	500	—
Environment steps before the first learning in the online stage	100	—
UTD ratio (gradient updates per environment step)	2	—
Actor weight push frequency (Learner $\rightarrow$ Actor)	every 4 gradient steps	—
Batch size $ \mathcal{B} $	256 (128 + 128 offline/online)	256
Learning rate (actor / critic / α)	3×10^{-4}	3×10^{-4} (actor only)
Gradient clipping norm	40.0	40.0
Discount factor γ	0.97	—
Target-network soft update τ	0.005	—
SAC initial temperature α_0	0.01	—
Latent dimension / MLP hidden layers	256 / [256, 256]	same as left
Buffer capacity (offline / online)	10^5 / 10^5	—
Image augmentation	—	DrQ random_shift, pad = 4
BC discrete-head weight λ	—	0.5
Tanh-Gaussian σ clamp range	—	$[10^{-5}, 10]$
Still-frame filtering threshold <code>action_norm_min</code>	—	10^{-3} (CLI overrides default 10^{-6})
HG-Dagger intervention trigger threshold	—	enter 0.05 / exit 0.02
HG-Dagger release tail <code>tail_k</code>	—	10 frames
Episodes per iteration	—	30
Stopping criterion (intervention rate $\leq 5\%$ sustained)	—	5 consecutive episodes

4.5 Reward Design

Reward function. The task policy of this thesis uses a sparse binary reward:

$$r_t = \begin{cases} 1 & \text{if the episode reaches the success criterion at step } t, \\ 0 & \text{otherwise.} \end{cases}$$

In simulation training, the success criterion is given by the MuJoCo ground truth: the projected distance of the block and the target plate on the horizontal plane $\|p_{\text{block}}^{xy} -$

$\|p_{\text{plate}}^{xy}\| < 5 \text{ cm}$, and the block z-coordinate $< 5 \text{ cm}$ (i.e., the block has been put down rather than suspended in mid-air). On the real robot, a *manual keyboard reward protocol* is used instead of the automatic distance criterion: the operator gives a binary reward and episode control signals for each episode through four keyboard keys—S starts the episode, Enter marks success ($r = 1$, *terminal*), Space marks failure ($r = 0$, *terminal*), and Backspace discards the entire episode (setting `discard=True` so that this episode does not enter the replay buffer or the demo pickle). This protocol simultaneously handles episode-boundary control (no automatic timeout detection needed) and replay filtering (the discard signal), simplifying the real-robot data loop; the concrete implementation is given in Chapter 6. Each episode terminates when any of success / failure / discard / step upper limit (300 steps @ 10 Hz, i.e., 30 s) is triggered; the success step produces $r_t = 1$, and the other steps are 0. Both mechanisms give a sparse binary reward signal, differing only in the trigger source (automatic criterion in simulation vs. manual adjudication on the real robot), and neither provides the policy with a dense signal at the distance level.

Feasibility of the sparse reward. Under a general setting, a sparse reward is often hard to converge due to exploration difficulty, but two concrete constraints of this thesis make a sparse reward in fact more suitable than a dense reward.

(i) The HIL-SERL training framework of Section 5.2 has already solved the most central exploration bottleneck of the sparse reward through offline demo guidance (50 successful episodes + warmup) and human-in-the-loop intervention throughout training—the policy does not have to “grope out the first success from scratch,” as the demos and the interventions have already provided the critic with sufficient positive samples and corrective gradients. *It should be pointed out that this argument applies only to simulation:* the real-robot SAC drift phenomenon reviewed in Section 6.3.5 shows that even when the exploration bottleneck is solved by demo + HIL, the critic may still be unable to calibrate the Q-surface due to the throughput limitation, and the actor drifts after being unfrozen; the exploration bottleneck and the accuracy of the Q-estimate are *two independent bottlenecks*, and the simulation only needs to satisfy the former, whereas the real robot needs to satisfy both. The switch on the real robot to BC + HG-Dagger, which does not depend on a critic, is precisely to bypass the Q-estimation bottleneck.

(ii) In addition to the exploration bottleneck, a dense reward itself has a second hidden danger under this setting: under encoder bias, a natural dense-reward candidate (such as a TCP-target distance of the form $-\|FK(\mathbf{q} + \mathbf{b}) - \mathbf{p}_{\text{block}}\|$) is itself a *corrupted signal*—it depends on $FK(\mathbf{q} + \mathbf{b})$ rather than the true TCP position, so directly optimizing along this dense signal would induce the policy to hack the geometric effect of the bias into the proxy metric and deviate from the true goal (i.e., typical reward hacking, the background of which is given in Chapter 2).

Combining (i) and (ii), a sparse binary reward is the most robust choice under the setting of this thesis.

5 Simulation: Policy Training and Observability Validation

5.1 Introduction: The Simulation-Side Paradigm and Chapter Organization

Chapter 4 gave the method design and the shared network architecture of the task policy (25-dimensional observation, randomized-bias training, the DINOv3-S encoder plus three network heads), but left open the execution-level question of “how exactly to train.” This chapter follows the method layer of Chapter 4 and gives the two main threads of the **simulation side** of this thesis and their experimental loops:

- *Simulation training of the task policy* (Section 5.2): the task policy is trained with HIL-SERL (RLPD + SAC + human-in-the-loop intervention), supported by an Actor-Learner distributed architecture. This training is used to validate the observability propositions raised in Chapter 3—forming an end-to-end loop composed of the three groups of experiments H1 (bias degradation baseline), H2/H3 (fixed/random bias comparison), and the observation-dimension ablation in Section 5.4. *The ablation experiment is conducted on PickCube 24D to isolate the encoder-bias factor; the deployment 25D described in Section 4.2 is an equivalent extension of 24D + the hand channel, and the hand channel does not participate in the identifiability argument, serving only the distance-triggered switching between the task policy and the backup policy.*
- *Simulation training of the backup policy* (Section 5.3): the backup avoidance policy is trained with purely online SAC + domain randomization, accompanied by the reward and geometry evolution $V1 \rightarrow V2 \rightarrow V3 \rightarrow V3b$. This thread serves the real-robot deployment of Chapter 6—the backup is not retrained on the real robot, and the simulation checkpoint is transferred directly.

The two main threads are mutually independent at the simulation level: the task policy is concerned with “how to complete grasping under encoder bias,” and the backup policy is concerned with “how to dodge safely when a hand approaches”; at runtime they are coordinated by a hard switch based on a distance criterion, detailed in the real-robot deployment of Chapter 6. Section 5.4 gives all the simulation experimental data corresponding to the two main threads; Section 5.5 is a brief bridge at the end of this

chapter, transitioning the simulation-validated “success signal path” to the real-robot deployment.

5.2 Simulation Training of the Task Policy (HIL-SERL)

Design choice 3 of Section 4.1 and Section 2.5 have already argued the basis for adopting HIL-SERL on the simulation side to validate the observability propositions and the randomized-bias robust training (high-throughput Actor-Learner, HIL intervention to compensate for the sparse reward, and an architecture cognate with the real-robot stack). This section gives the implementation framework: with the HIL-SERL proposed by Luo et al. [15] as the backbone, with human-in-the-loop intervention as a continuous corrective signal, and with high-throughput asynchronous sampling supported by an Actor-Learner distributed architecture. The framework described in this section *converges stably in the simulation loop*; the accompanying H1–H3 empirical evidence is given in Section 5.4, and the methodological post-mortem of the real-robot failure is given in Section 6.3.5.

Algorithm stack. The lowest layer of the framework is Soft Actor-Critic [14] over a continuous action space; on top of it, the RLPD proposed by Ball et al. [13] provides demo guidance for sparse-reward tasks through a 50/50 mixed sampling from two replay buffers—the *offline demonstration buffer* is preloaded with 50 successful episodes of human teleoperation, the *online buffer* collects the policy’s real-time interaction with the environment, every gradient update samples half from each side, and the SAC loss is not modified; this thesis follows RLPD’s high UTD ratio and LayerNorm design to stabilize critic training. HIL-SERL then, on top of RLPD, further makes human-in-the-loop intervention a first-class source of training data throughout the entire online training process (the concrete handling of the intervention data is given in the next paragraph).

Human-in-the-loop intervention. During online sampling, the trainer can intervene, through a teleoperation device (keyboard, SpaceMouse, etc.), in the episode that the policy is executing and directly give the desired action; this idea of “supplementing correction by human demonstration in the regions where the policy fails” can be traced back to the DAgger proposed by Ross et al. [11]. The transitions produced during an intervention are uniformly appended to the offline demonstration buffer. This thesis initially attempted to retain only the intervention episodes that “ultimately succeeded,” but found two problems during implementation and training: (i) the transitions are written into the buffer in units of steps rather than organized by episode, and tracing back and splitting them by success or failure would require introducing additional boundary-determination logic, and tends to discard a large amount of valid supervision wholesale because of the relatively high failure rate; (ii) even if an episode ultimately fails to complete the grasp, the action direction given by the human intervention is itself

a corrective gradient signal that the policy can learn from. Therefore, all intervention transitions are ultimately retained uniformly, no longer relying on ultimate success or failure for filtering.

Distributed training architecture. Figure 5.1 illustrates the two asynchronous processes of the training system. The Actor process holds a copy of the policy weights, and is responsible for interacting with the simulation environment, receiving HIL interventions, and writing transitions into the online buffer; the Learner process holds the two buffers and the optimizer, and is responsible for mixed sampling, computing the SAC loss, updating the actor/critic and the temperature parameter, and periodically synchronizing the latest weights to the Actor. The two sides communicate through gRPC, so that environment sampling and gradient updating are fully decoupled—the Learner executes gradient steps as fast as possible, the Actor only needs to collect transitions stably at a fixed physical control frequency, and the gradient computation does not block the environment step. This architecture also reserves a path for later real-robot training (although the real-robot stage of this thesis ultimately adopts BC and HG-Dagger, detailed in Section 6.3).

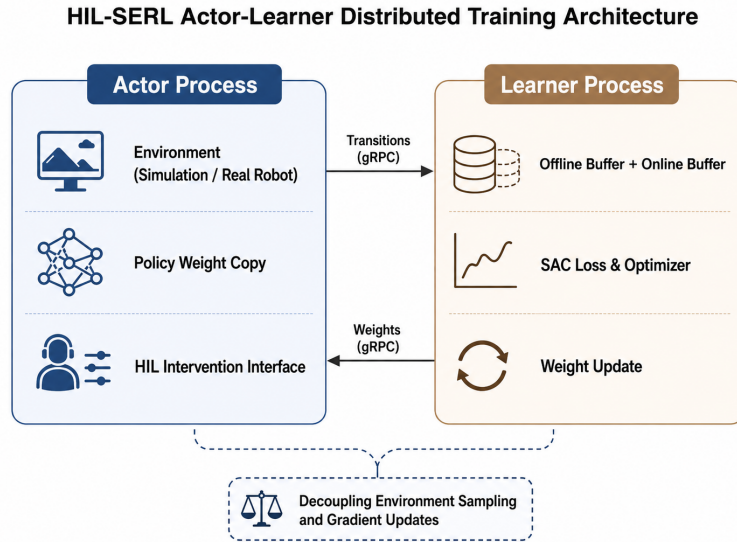


Figure 5.1: The Actor-Learner distributed architecture of HIL-SERL. The Actor is responsible for interacting with the environment, receiving HIL interventions, and writing into the online buffer; the Learner holds the buffers and the optimizer, and is responsible for mixed sampling, the SAC update, and pushing the weights back. The two sides communicate through gRPC, and environment sampling and gradient updating are fully decoupled.

Simulation training procedure. The complete training consists of two online stages. *Demo warmup stage:* the first 500 gradient updates at the start of training sample only from the offline buffer (the online buffer is still empty at this point), using the 50 demos to establish reasonable initial values for the two critics and the actor, avoiding unstable policy behavior caused by the randomly initialized value function at the start of online interaction. *Online mixed sampling stage:* after the warmup ends, the Actor process begins interacting with the simulation environment and receiving HIL interventions. Among these, the first 100 environment steps after entering the online stage are reserved for *pure data collection* (`online_step_before_learning = 100`, writing transitions into the online buffer only without triggering gradient updates), whose role is to let the online buffer first accumulate a small batch of samples matching the current policy distribution, avoiding destroying the initial values obtained during the warmup stage with an extremely biased sampling distribution when the buffer is almost empty. Afterwards it enters the steady-state training loop: each environment step triggers 2 gradient updates ($UTD = 2$), each update samples $|\mathcal{B}|/2$ transitions from each of the offline and online buffers; the transitions produced by interventions are reinjected into the offline buffer as described in the previous paragraph; every 4 gradient steps the Learner synchronously pushes the latest weights of the Actor and the discrete gripper critic to the Actor process through gRPC. All hyperparameters are given in Table 4.3 (the simulation HIL-SERL column).

5.3 Simulation Training of the Backup Policy (SAC + DR)

This section gives the full stack of the simulation training of the backup avoidance policy: problem definition, training-paradigm choice, scenario design, reward-function evolution, geometry upgrade, and domain randomization. The biggest difference between the backup policy and the task policy of Section 5.2 is that it adopts *pure-simulation* SAC + DR rather than HIL-SERL—Section 5.3.2 below gives the detailed argument for the paradigm choice.

5.3.1 Problem Definition and Safety Requirements

Applicable situation of the backup avoidance policy: during the process in which the arm executes a grasping/placing task, a human hand may intermittently enter the workspace—for example, an operator adjusting the material position, replenishing consumables, or manually intervening during HIL training. In such intermittent human-robot shared situations, the backup policy *takes over when a hand approaches the TCP, actively dodges, and hands control back to the task policy afterwards*. This policy covers two situations: (i) human-robot collaborative safety during production deployment (the main application); (ii) the safety guarantee when the operator intervenes during the HIL training of Chapter 4 (the derived application).

Safety requirements. The backup policy must simultaneously satisfy four requirements: (1) **collision avoidance**—the TCP-to-hand distance > 8 cm; (2) **maintaining the grasp**—never releasing an already-grasped block during avoidance; (3) **a hard workspace boundary**—the TCP does not exceed the preset Cartesian workspace (jointly defined by the simulation spawn box and the real-robot `abs_pose_limit`); (4) **dodging in place**—the cumulative TCP displacement during avoidance ≤ 15 cm, so as to guarantee that the task policy can resume execution near the original position. Violating any one of these terminates the episode with -10 , and the concrete reward form is given in Section 5.3.4.

Runtime policy-switching mechanism. The Actor side simultaneously holds the two sets of parameters of the task policy and the backup policy, and the hand–TCP Euclidean distance determines which set produces the action at the current step: when the distance ≥ 8 cm the task policy action takes effect, and when the distance < 8 cm it immediately switches to the backup policy; the backup policy executes for at most 10 consecutive steps, during which, if the hand leaves the workspace or the step count is used up, it switches back to the task policy. This “distance-triggered + time-bounded” hard switch simplifies the training objective of the backup policy to “dodge and stabilize within 10 steps,” fully decoupled from the task-level semantics. The concrete thresholds and geometric alignment of the real-robot deployment (Route A center-dist) are given in Section 6.4.

5.3.2 Choice of the Training Paradigm

Design evolution: from hard-coded freezing to separated learning. The current separated dual-policy design went through two failed iterations; the environment was always unified (the task policy and the backup policy share `PandaPickPlaceSafeEnv`), and what changed was how to “learn avoidance.” *Scheme one* is **hard-coded freezing**—when a hand approaches, the task policy stops outputting and the arm keeps its configuration. This scheme is statically safe but is not learning-based and is inefficient because it is purely passive (it cannot respond differentially to differences in motion patterns; when the hand approaches along the original direction, a still end-effector may still make contact). *Scheme two* is a **unified learning policy**—using the same policy to optimize the task and the avoidance simultaneously, with the reward following the sparse binary form (success = 1, any failure = 0). Typical reward hacking occurred during training: the policy learned to *accelerate toward the target plate* when the hand appeared, completing the placement before the collision; the reason is that this reward does not distinguish collision from timeout (the penalty is 0 for both), and “sprinting to completion” is superior in expected return to “completing after avoidance.”

The two failures together led to the final architecture of this section: **the environment is unchanged (still sharing the 25-dimensional unified observation), and the learning objective is split into two specialized policies**—the task policy retains the original

sparse 0/1 reward (Section 4.5), optimizes only “grasping the block and placing it on the target plate,” and produces no avoidance response to h_{active} or p_{hand} ; avoidance is handled by a separate backup policy, which adopts a non-negative immediate + large termination penalty reward (Section 5.3.4). The two are hard-switched by a runtime distance criterion—this both retains the safety lower bound of hard-coded freezing, and, through learning-based active avoidance, circumvents the two inefficient situations of “interrupting the task” and “passive contact.”

Architectural inspiration: Kiemel et al. 2024. The overall architecture of this section is inspired by Kiemel et al. [16]: they first train a backup policy with model-free RL to produce avoidance actions, and then use it as a safety layer when learning other tasks—before each decision the collision risk is estimated, and if the risk is too high the backup’s action replaces it. This thesis follows the idea of “pre-training the backup, switching at runtime,” and makes two adjustments: (i) the use case is changed from *a safety layer for general task learning* to collaborative safety under HIL training and a human-robot shared workspace; (ii) the online risk estimation is simplified to a hard switch based on the hand–TCP distance, reducing the computational and engineering complexity of the real-robot deployment. At the level of reward design, this thesis retains its core principle—*non-negative immediate rewards + a large termination penalty*.

Why pure simulation + domain randomization is adopted. In contrast to the HIL-SERL of the task policy in Section 5.2, the backup policy adopts *pure-simulation SAC + domain randomization* rather than HIL training. Four reasons: (i) *Observation modality*—the input is a low-dimensional state, with no RGB image, and there is no visual sim-to-real gap; the gap of the low-dimensional observation (position noise, velocity jitter, detection delay) is precisely covered by DR (Section 5.3.6). (ii) *Recursive safety*—if the backup itself also needed HIL training, then training it would in turn need a more low-level “backup of the backup” to guarantee the operator’s safety, an infinite regress; pure simulation fundamentally bypasses this problem. (iii) *Evaluability*—the simulation can construct obstacle-motion scenarios with complete coverage, and can directly evaluate the policy’s safety with collision ground truth. (iv) *Bias robustness*—the avoidance behavior mainly depends on the *relative geometry* of the obstacle and the TCP rather than the absolute position, and the DR observation noise makes the policy react to the approximate direction rather than the precise position, so the backup is naturally robust to encoder bias and needs no bias-aware training; empirically, running the backup checkpoint trained in a bias-free environment directly in an evaluation environment with a random joint-1 bias yields a survival rate basically consistent with that in the bias-free environment (detailed in Section 5.4).

5.3.3 Scenarios, Observation, and Motion Modes

This section gives the simulation environment design of the backup policy—two training scenarios, the composition of the single-frame observation vector, the frame-stacking strategy, and four obstacle motion modes.

Scenarios S1 / S2 and the observation vector. This section trains the backup policy on two progressive simulation scenarios: *S1* contains an intermittently appearing moving obstacle (simulating a single human hand); *S2* additionally places, on top of *S1*, a static obstacle at a random but fixed position (simulating a fixed object on the workbench). The two scenarios share the same single-frame observation structure—an 18-dimensional robot state (7 joint angles + 7 joint velocities + 1 gripper opening + 3 TCP positions) plus a 10-dimensional observation per obstacle (1-dimensional activation flag + 3-dimensional absolute position + 3-dimensional differential velocity + 3-dimensional position relative to the TCP). *S1* has 28 dimensions per frame, and *S2* has 38 dimensions per frame. The activation flag characterizes the intermittent appearance of an obstacle: the obstacle is 1 when within the workspace and becomes 0 after leaving, so that the policy can return to a normal state when the obstacle disappears and re-enter avoidance when it next appears.

Frame stacking. The single-frame observation already contains the differential velocity of the obstacle, but different motion modes exhibit significantly different features in the time dimension—the periodic pauses of STOP_GO mean that “the current velocity ≈ 0 ” may be a true stop or a brief pause; the accelerated crossing of PASSING and the constant-velocity straight line of LINEAR can be reliably distinguished only after observing several consecutive steps. To let the policy judge based on a short history rather than a single instantaneous observation, this section performs 3-frame stacking of the single-frame observation:

$$\mathbf{o}_t^{\text{stack}} = [\mathbf{o}_{t-2}^{\text{single}}; \mathbf{o}_{t-1}^{\text{single}}; \mathbf{o}_t^{\text{single}}],$$

so that the complete observation of *S1* is $28 \times 3 = 84$ dimensions and that of *S2* is $38 \times 3 = 114$ dimensions. This is equivalent to implicitly providing second-order motion information (acceleration and mode features) at the observation level, enabling the policy to recognize the motion mode and predict the short-term future motion.

Obstacle motion modes and episode setting. At the beginning of each episode, one of four motion modes is selected uniformly at random and kept unchanged within the episode (Table 5.1), so as to cover the typical modes of hand approach; the policy must remain robust to all four modes. The training episode length is fixed at 10 steps (@ 10 Hz, i.e., 1 s), and an *undiscounted* setting $\gamma = 1.0$ is adopted—under such a short episode the safety of every step is equally important, and any discount would artificially distort the weight of the avoidance behavior in the time dimension.

Table 5.1: The four obstacle motion modes in backup-policy training

Mode	Trajectory	Key parameter	Simulated scenario
LINEAR	Constant-velocity straight line toward the TCP	—	A person reaching directly for an object
ARC	Arc curving toward the TCP	Direction vector rotates $\omega \sim \mathcal{U}(-0.1, 0.1)$ rad per step	A person reaching around from the side
STOP_GO	Toward the TCP with random pauses	30% pause probability per step	A person hesitating before reaching
PASSING	Crossing perpendicular to the TCP direction	Speed $\times 1.3$	A hand passing by rather than targeting the robot

5.3.4 Evolution of the Reward Function Design

The reward function of the backup policy went through three iterations—V1 / V2 / V3, each failure exposing a typical reward-hacking mode, together leading to the final form of V3; V3 further evolves into the V3b geometry during the real-robot deployment stage (detailed in Section 5.3.5). This subsection states the three versions in chronological order and summarizes the design principles; the concrete training curves and quantitative comparison belong to the experimental part and are left to Section 5.4.

V1: the proximity reward and its failure. V1 adopts a “move away from the obstacle” immediate reward—driven by the distance between the TCP and the nearest obstacle:

$$r_t^{V1} = r_{\text{prox}}(d_t^{\text{obs}}) - 0.01 \|a_t\|, \quad r_{\text{prox}}(d) = \min(1, (d/d_{\text{danger}})^2),$$

where d_t^{obs} is the Euclidean distance between the TCP and the nearest obstacle; on collision $r_{\text{term}} = -1.0$. The problem is that r_{prox} *directly rewards moving away*—the optimal behavior the policy learns is to run as far away as possible, rather than staying in place and dodging slightly. This directly conflicts with the deployment requirement of this section (safety requirement (4) “dodging in place” of Section 5.3.1): the policy “avoided the hand” but “lost the task position.” *Lesson:* the reward must align with the deployment objective—the goal of avoidance is not “the farther from the obstacle the better,” but “neither colliding nor moving away.”

V2: the all-negative reward and its failure. V2 changes to an *all-negative* soft-constraint form, dissuading the policy from running far away with three penalties on displacement + action magnitude + action smoothness. For compactness of writing, two abbreviations are introduced: $\Delta p_t := p_t^{\text{TCP}} - p_0^{\text{TCP}}$ (the cumulative displacement relative to the episode start), and $\Delta a_t := a_t - a_{t-1}$ (the action increment); then the per-step reward is

$$r_t^{V2} = -0.5 \|\Delta p_t\| - 0.01 \|a_t\| - 0.01 \|\Delta a_t\|,$$

collision termination $r_{\text{term}} = -1.0$, $\gamma = 0.99$. The problem is that **every surviving step brings a negative reward**—the total return of fully surviving 10 steps is roughly between -0.5 and -0.8 , and once a collision occurs at step 1 it is -1.0 , so the gap between the two is only ~ 0.2 ; when the displacement accumulation is slightly larger, the total return of surviving is even worse than colliding immediately. The optimal behavior the policy learned became to *terminate as fast as possible* (“dying fast is better than dying slow”): in

the training log the average reward continuously deteriorated from -0.88 to -1.00 , and the episode length gradually shortened until the collision occurred on almost the first step of every episode (the complete curve is given in Section 5.4). *Lesson*: survival must be strictly superior to any termination—the immediate reward cannot be negative, and the termination penalty must be far larger than the cumulative return of any possible surviving trajectory.

V3: non-negative immediate reward + large termination penalty. V3 strictly follows the principle of “a non-negative per-step immediate reward, a large penalty on termination” of Kiemel et al. [16], and retains the displacement term in the soft reward to achieve the “dodging in place” goal. The immediate reward for every surviving step is

$$r_t^{\text{V3}} = 0.5 - 0.5\|\Delta \mathbf{p}_t\| - 0.01\|\mathbf{a}_t\| - 0.01\|\Delta \mathbf{a}_t\| \geq 0,$$

the episode ends with a large termination penalty $r_{\text{term}} = -10.0$ when any of collision, block drop, the TCP exceeding the hard workspace boundary, or the cumulative displacement $\|\Delta \mathbf{p}_t\| > 15$ cm is triggered; if the episode safely survives the full 10 steps, an additional $+5.0$ completion reward is given; the discount is $\gamma = 1.0$. Table 5.2 lists the total return of several typical scenarios: staying still $+10.0 >$ a small dodge $\approx +9.7 >$ a large dodge $\approx +8.5 \gg$ collision or over-reaction -10.0 . This ordering has three key properties: (i) *the per-step immediate reward is non-negative*, surviving is always superior to early termination, eliminating the “dying fast” loophole of V2; (ii) *the termination penalty is far larger than the cumulative upper bound of any surviving trajectory* ($|-10.0| \gg +10.0$), so actively terminating is not the optimal solution in expected return; (iii) the soft ordering of staying still $>$ a small dodge $>$ a large dodge makes the policy tend to stay still when it can (aligning with the “dodging in place” goal, eliminating the “run far away” loophole of V1), and tend toward a small-magnitude action when it has to move.

Table 5.2: Total expected return of typical scenarios under the V3 reward ($\gamma = 1.0$, 10-step episode)

Scenario	Behavior description	Total return
Perfect stillness	Staying in place; the hand passes by without contact	$+10.0$
Small dodge	Surviving after a cumulative displacement of ≈ 5 cm	$\approx +9.7$
Large dodge	Surviving after a cumulative displacement of ≈ 12 cm	$\approx +8.5$
Over-reaction	Cumulative displacement > 15 cm triggers the hard constraint	-10.0
Collision failure	Failing to dodge on the first step and colliding with the hand	-10.0

Summary of the design principles. The three iterations $V1 \rightarrow V2 \rightarrow V3$ yield four reward-design principles generally applicable to short-episode avoidance tasks: (1) the immediate reward must be *non-negative*, guaranteeing that survival is strictly superior to any termination; (2) the termination penalty must be *far larger than* the cumulative

upper bound of any surviving trajectory, preventing “active termination” from becoming the optimal solution; (3) a cross-episode hard constraint such as displacement must be expressed by a *termination penalty* rather than a soft penalty, avoiding circumventing termination through cumulative soft penalties; (4) a short-episode task should use an *undiscounted* setting $\gamma = 1.0$, avoiding an unnecessary trade-off of the safety weight in the time dimension.

5.3.5 V3→V3b Geometry Upgrade: from a Single Sphere to a 5-Sphere Whole Arm

The geometric model of the V3 reward only detects the distance between the single sphere of panda_hand and the obstacle. During the real-robot deployment stage (Section 6.4), it was found that this model has a failure mode: the policy learns to make panda_hand avoid the obstacle, but **the elbow and forearm may still collide with the obstacle**—single-sphere detection cannot capture the collision risk of mid-segment links such as link3/link4. Accordingly, this thesis upgrades V3 to the V3b geometry, comprising three changes:

1. *Multi-sphere whole-arm detection*: the single sphere (only panda_hand, radius 0.10 m) is extended to 5 spheres (link3, link4, link5, link6, hand, with radii 0.07, 0.07, 0.065, 0.065, 0.10 m respectively). The collision criterion takes the minimum distance over all spheres, so that the training reward directly covers the elbow/forearm collision risk.
2. *Adjustment of the obstacle radius and the TCP-obstacle distance*: the obstacle radius is increased from 0.035 m to 0.10 m (aligned with the equivalent radius of the real-robot hand bbox); the obstacle spawn distance is increased from $\mathcal{U}(0.21, 0.30)$ m to $\mathcal{U}(0.30, 0.40)$ m; the hand speed is increased from $\mathcal{U}(0.005, 0.015)$ m/step to $\mathcal{U}(0.015, 0.030)$ m/step; the maximum allowed displacement is increased from 0.30 m to 0.50 m.
3. *Saturating proximity bonus*: a saturating proximity bonus is added to the V3 reward (saturation value 0.20 @ clearance 0.10 m), giving the policy a positive gradient signal for the critical distance (close but not colliding), avoiding the situation in which V3, under multi-sphere detection, fails to train a fine avoidance because “moving away is enough.”

The FSM threshold upgrade accompanying V3b is given in Section 6.4 (Route A center-dist: $D_{\text{SAFE}} 0.30 \rightarrow 0.40$ m, $D_{\text{CLEAR}} 0.35 \rightarrow 0.45$ m). The trained V3b version (backup_policy_s1_v3b_300k) has a survival rate of 95.0% in the preferred real-robot ckpt, a significant improvement over the 71.5% of the V3 single-sphere version; the hand_collision rate drops from 17.5% to 3.0% (detailed in Section 5.4).

5.3.6 Training Protocol and Domain Randomization

Network architecture (the simplification relative to Section 4.4). The network main body of the backup policy is fully isomorphic to Section 4.4—the continuous Actor and the Twin Critic of SAC both use an MLP with hidden layers [256, 256] and ReLU activation as their main body, and the basic hyperparameters such as the learning rate, gradient clipping, and target-network soft update all reuse the settings of the task policy. The action space is a 6-dimensional continuous end-effector increment $(\Delta x, \Delta y, \Delta z, \Delta r_x, \Delta r_y, \Delta r_z)$ with a single-step scaling of 0.03 m, and the gripper is always locked during the activation of the backup policy and does not participate in the action. There are two simplifications: (i) the input is a low-dimensional state, with *no visual encoder*; (ii) *pure online SAC* is adopted, with no offline demo buffer and no RLPD-style demo bootstrap. The remaining nonzero differences are listed in Table 5.3. Among them, the UTD ratio is taken as 8 (significantly higher than the 2 of the task policy), as a compensation for “short episodes and scarce online data”; settings such as $\text{UTD} \in \{2, 4\}$ were also tried during training, and they have basically no influence on the final survival rate, affecting only the wall-clock speed of training convergence.

Table 5.3: Key hyperparameter differences of the backup policy relative to the task policy

Hyperparameter	Task policy (Chapter 4)	Backup policy (this section)
Offline demo buffer	50 episodes	None
Total online training steps	10^5	2×10^5 (V3b trains 3×10^5)
UTD ratio	2	8 (V3b reverts to 4 to prevent overfitting)
SAC initial temperature α_0	0.01	0.1
Online buffer capacity	10^5	2×10^5
Episode length	200 steps	10 steps (V3b 20 steps)
Discount factor γ	0.97	1.0

Domain-randomization parameters. Domain randomization *acts only on the observation layer*, and collision detection always uses the true positions—this is the bottom line of the safety boundary: DR makes the policy robust to perception noise, but it should not let the policy practice avoidance on “false collisions.” The concrete parameters are given in Table 5.4, and the real-robot source of each item is matched one to one in the sim-to-real alignment analysis of the next paragraph.

Sim-to-real alignment analysis. Table 5.5 compares the gap between simulation and the real robot for every stage on the observation and control chain. The core observation is: **the robot’s own state** ($q, \dot{q}$, TCP) is read directly from the encoders, with no gap between sim and real (this is precisely the main motivation for this section choosing a pure-state observation); whereas all the dimensions related to **obstacle perception** (position, velocity, delay) depend on the external vision system and must be covered by DR. There

Table 5.4: Domain-randomization parameters in backup-policy training

Parameter	Distribution	Modeled content
Obstacle position noise	$\mathcal{N}(\mathbf{0}, 0.03^2 \mathbf{I}_3)$ m	Cumulative localization error of vision + depth
Obstacle velocity noise	$\mathcal{N}(\mathbf{0}, 0.01^2 \mathbf{I}_3)$ m/step	Frame-to-frame differencing jitter
Observation delay	$\mathcal{U}\{0, 1, 2\}$ steps	Visual detection + inference delay
Obstacle spawn distance	$\mathcal{U}(0.30, 0.40)$ m (V3b)	Different entry distances
Obstacle movement speed	$\mathcal{U}(0.015, 0.030)$ m/step (V3b)	Different hand speeds
TCP position noise	$\mathcal{N}(\mathbf{0}, 0.005^2 \mathbf{I}_3)$ m	Single-frame error of external TCP localization

is a structural difference between the controller—from the OSC of the simulation to the Cartesian impedance control of the real robot—but they are approximately equivalent in the small-step TCP-increment action space, with the rigorous argument given in Chapter 6.

Table 5.5: Sim-to-real gap analysis of the backup policy

Stage	Simulation source	Real-robot source	Gap-bridging method
Robot state $q, \dot{q}$, TCP	MuJoCo ground truth	Joint encoders	No gap
Hand position	MuJoCo ground truth	Visual detection + D455 depth	DR position noise (see Section 6.4.1)
Hand velocity	Ground-truth differencing	Frame differencing of visual results	DR velocity noise
Observation delay	0	~ 10 ms detection delay	DR delay $\mathcal{U}\{0, 1, 2\}$ steps
Coordinate transform	None	$T_{\text{cam}} \rightarrow$ robot calibration	Included in the position-noise term
Controller	OSC	Cartesian impedance control	Approximately equivalent under small increments (Chapter 6)
Obstacle motion mode	Random over 4 preset modes	Real human-hand motion	4 modes cover typical motions

The concrete selection of the hand detector (the evolution from the initial choice of MediaPipe to the deployment of WiLoR) is a real-robot deployment decision, and is moved to Section 6.4.1 for a detailed account.

5.4 Simulation Experiments and Analysis

This section gives all the simulation experimental data corresponding to the two main threads of this chapter (the task policy HIL-SERL + the backup policy SAC + DR). All experiments are completed in the PandaPickCube / PandaPickPlaceSafe / PandaBackupPolicy series of MuJoCo environments, with 50 episodes evaluated per bias value, and 3 seeds trained per hyperparameter group.

5.4.1 Experimental Setup

Environment. A 7-DoF Franka Panda simulation based on MuJoCo 3.0, with OSC control at 10 Hz and physics simulation at 500 Hz. The task-policy evaluation is completed on PandaPickCube: grasping a tabletop block and lifting it by 20 cm is regarded as a success. The backup-policy evaluation is completed on PandaBackupPolicyS1 / S2: per episode, one obstacle motion is selected at random with equal probability from the four

modes of Table 5.1, and surviving the full 10 steps without triggering any termination condition is regarded as a success.

Statistical protocol. Each group of experiments uses 3 seeds, each seed evaluates 50 episodes, and the mean $\pm$ std is reported; in a bias sweep, each bias value is evaluated independently with 50 episodes. Unless otherwise specified, the training hyperparameters follow the default settings of Table 4.3 and Table 5.3.

5.4.2 H1: The Encoder-Bias Degradation Baseline

Hypothesis. A policy obtained by bias-free training degrades in performance under encoder bias.

Experiment. A bias-free policy is trained (PickCube 18D observation, without `real_tcp`), and evaluated respectively under $b \in \{0.0, 0.05, \dots, 0.30\}$ rad. The evaluation environment uses the dual-injection simulation step of Algorithm 1 that simultaneously satisfies condition (C1) of Proposition 3.3 (the control law uses $\mathbf{q}_{\text{true}} + \mathbf{b}$) and (C2) (the observation returns the biased state), ensuring that the bias injection faithfully reproduces the causal chain of Section 3.2.

Results (Table 5.6): the success rate drops significantly once the bias exceeds 0.10 rad, and approaches complete failure at ≥ 0.25 rad; the end-effector position shifts linearly with the bias (about 3 cm per 0.05 rad). This degradation curve directly corresponds to Proposition 3.1: under the basic observation the bias is non-identifiable, and the policy cannot make a differentiated compensation for different $\mathbf{b}$.

Table 5.6: H1: degradation of the bias-free policy under different J1 biases

Bias (rad)	Success rate	Average steps	TCP offset
0.00	100%	13.5	0 cm
0.05	100%	13.5	~ 3 cm
0.10	80%	55.9	~ 6.5 cm
0.15	62%	62.5	~ 10 cm
0.20	51%	99.8	~ 13 cm
0.25	5%	184.5	~ 16 cm
0.30	0%	191.1	~ 19 cm

5.4.3 H2 + H3: Fixed vs. Random Bias Training

Hypothesis. Fixed-bias training leads to single-point conditioning and zero-bias over-compensation; random-bias training achieves full-range robustness.

Experiment. Three groups of controlled training (PickCube 24D observation, with `real_tcp`): (a) no bias; (b) fixed $b = 0.2$ rad; (c) random $b \sim \mathcal{U}[0, 0.25]$ rad. After each is trained, it is evaluated on a full bias sweep.

Results (Table 5.7):

- *Bias-free policy*: perfect at $b = 0$ but degrading from $b > 0.10$ on (same as H1).
- *Fixed-bias policy*: 99%+ near the training point $b = 0.2$, but dropping to 83% at $b = 0$ —direct empirical evidence of **zero-bias over-compensation**, corresponding to the “single-point conditioning $\rightarrow$ over-compensation” assertion of Section 3.4.3.
- *Random-bias policy*: robust at 92–100% over the full range, and **generalizing beyond the training range** (99% still at $b = 0.30$ rad, with a training upper limit of 0.25).

Table 5.7: H2 + H3: success-rate comparison of the three policies under a bias sweep (PickCube 24D environment; the same bias grid as the H1 degradation curve of Table 5.6)

Bias (rad)	Bias-free policy	Fixed $b = 0.2$ policy	Random $b \in [0, 0.25]$ policy
0.00	100%	83%	92%
0.05	100%	87%	100%
0.10	80%	99%	99%
0.15	62%	100%	96%
0.20	51%	100%	97%
0.25	5%	100%	96%
0.30	0%	99%	99%

5.4.4 Observation-Space Ablation: from 18D to 24D

Hypothesis. The identifiability conclusion regarding the observation dimension can be directly validated by the ablation experiment—after removing the independent external TCP $\mathbf{o}_{\text{real}}$, the bias is non-identifiable and the training fails.

Experiment. A three-level observation-space comparison (PickCube, random-bias training): 18D (without `real_tcp` and `block_pos`) / 21D (adding only `block_pos`) / 24D (adding `real_tcp` as well).

Results (Table 5.8):

- *18D*: robot proprioception only, an average of 42% over the full bias sweep—under the basic observation the bias is non-identifiable, corresponding to Proposition 3.1.
- *21D*: adding the target position but still without an independent TCP, the training does not converge—“knowing the goal but not itself,” unable to compensate for the encoder bias.
- *24D*: after adding `real_tcp` as well, reaching 92–100%—the identifiability condition of Proposition 3.2 is satisfied, and the policy implicitly recovers $\mathbf{b}$ and compensates conditionally.

Table 5.8: Observation-space ablation: a one-to-one correspondence between theoretical prediction and measurement

Observation	Dim	Missing key signal	Result	Theoretical prediction
18D	18	<code>real_tcp</code> + <code>block_pos</code>	42% avg	Non-identifiable (Proposition 3.1)
21D	21	<code>real_tcp</code>	Training fails	Knows the goal but not itself
24D	24	(complete)	92–100%	Approximately identifiable (Proposition 3.2)

Remark. This group of ablations is done on PickCube 24D to isolate the encoder-bias factor; the deployment 25D observation described in Section 4.2 is an equivalent extension of 24D + the hand channel (the hand channel does not participate in the identifiability argument, serving only the runtime backup switching), and the ablation conclusion can be extended by analogy to the 25D deployment observation.

5.4.5 Backup S1 V3b Main Survival Rate

Experiment. Training for 3×10^5 steps on the PandaBackupPolicyS1 V3b geometry (Section 5.3.5), with `max_disp` = 0.50 m; evaluation on 200 episodes (50 episodes per mode).

Results. The trained V3b version (`backup_policy_s1_v3b_300k`):

- **Full survival rate:** 95.0% (vs. 71.5% of the V3 single-sphere version, vs. 100% of the V2 single-sphere version, which however suffered elbow collisions on the real robot)
- **Failure-mode distribution:** `hand_collision` 3.0% (V3 single-sphere version 17.5%, a $5\times$ drop); `excessive_displacement` 5.5%
- **Per-motion-mode breakdown:** LINEAR / ARC / STOP_GO / PASSING all $\geq 90\%$, with the weakest ARC at 90.5% (the residual 9.5% of failures is mainly concentrated in this mode, and the mechanism awaits empirical validation by the real-robot evaluation after Section 5.5)

The V3b geometry upgrade **significantly mitigates** (a $5\times$ drop in `hand_collision`) but *does not fully eliminate* the elbow/forearm collision failure mode observed in the real-robot deployment (Section 5.3.5 has elaborated on it); the V3 single-sphere version is retained as a comparison, proving that the direction of the geometry upgrade is necessary, but the breakdown of the residual $\sim 9.5\%$ of failures in the ARC mode (exactly which link sphere collides) is left as future work.

5.4.6 Backup Bias Robustness Zero-Shot

Hypothesis. The backup policy is naturally robust to encoder bias and needs no bias-aware training (the empirical evidence for the fourth reason of Section 5.3.2).

Experiment. Using the S1 baseline checkpoint of Section 5.4.5 (bias-free training), 200 episodes are run respectively in two evaluation environments: (a) PandaBackupPolicyS1 (no bias); (b) PandaBackupPolicyS1BiasJ1 (J1 random bias $b_1 \sim \mathcal{U}[0, 0.25]$ rad).

Results. The survival rate is 99.0% for the baseline vs. 98.5% for BiasJ1, and the gap of 0.5% is within the sampling-noise range of 200 ep ($\pm 1.5\%$ at 95% CI).

Interpretation caveat (a sanity check rather than a robustness result). This experiment serves as a *decoupling validation* (*sanity check*) rather than an additional robustness proof: the backup policy only reads `tcp_true` (an external independent channel), the encoder bias is physically invisible to its observation layer, so “zero degradation” is a design expectation rather than a learned bias robustness. What this result demonstrates is that “the backup observation path is already decoupled from the encoder path,” so the cost of bias-aware training can be saved; the genuine bias-robustness proof is still in Section 5.4.3 (the H2/H3 task-policy experiments).

5.4.7 Reward V1 / V2 / V3 Evolution Training Curves

Experiment. Three groups of training, in the same PandaBackupPolicyS1 environment and under the same SAC hyperparameters, use three rewards: V1 (proximity) / V2 (all-negative) / V3 (non-negative + large termination). Each group uses 3 seeds and 2×10^5 steps.

Recorded metrics. V1: the average cumulative TCP displacement (which should approach the 15 cm upper limit); V2: the average episode reward (which should continuously deteriorate from ~ -0.88 to ~ -1.00) + the average episode length (which should shorten); V3: the average episode reward (which should converge stably to $\sim +9$ to $+10$) + the survival rate (which should rise to $\sim 99\%$).

Results. The three groups of training curves confirm the design principles of Section 5.3.4:

- V1: the policy learns to run far away, the average displacement approaches the 15 cm upper limit, triggering a large number of `excessive_displacement` terminations—typical reward hacking.
- V2: the average reward deteriorates monotonically from -0.88 to -1.00 , and the episode length shortens until the collision occurs on almost the first step—“dying fast is better than dying slow.”
- V3: converges stably to the $+9$ to $+10$ range, the survival rate rises to $\sim 99\%$, corresponding to the final result of Section 5.4.5.

5.4.8 Backup S2 Multi-Obstacle Scenario Extension

Experiment. PandaBackupPolicyS2 (a moving + a static dual obstacle), 114D observation (38×3 frame stacking), UTD = 4 (UTD = 8 collapses after 1×10^5 steps of training), 2×10^5 steps.

Results. The survival rate is 83–91% (with a relatively large variance over the two evaluations); the `hand_collision` rate is 13.5% (vs. 0.5% for the S1 baseline, a $26\times$ rise); the weakest modes are ARC 75.0% / STOP_GO 81.8%, and the strongest mode is PASSING 91.0%. The drop from 99% for a single obstacle to $\sim 85\%$ for multiple obstacles is because **multi-obstacle geometry is itself more difficult** (the escape cone narrows, and the sample combinations rise from 50 to 2500), not because the policy cannot learn well. This proves that the architecture of this section is still extensible to multi-obstacle scenarios.

5.5 The Bridge from Simulation to the Real Robot

The simulation loop given in the previous section provides two pieces of evidence for the real-robot deployment of the next chapter, but their scope of applicability must be strictly delimited. **The identifiability proposition (Proposition 3.2) has been empirically validated in the 24D ablation of Section 5.4.4 as the information-theoretically “success signal path”**—but it must be made clear: the first real-robot run takes only the *privileged half* of that path (the external independent TCP `tcp_true`), and has not yet put the corrupted encoder path into use, so the success-rate matrix of Chapter 6 validates that “the success signal path proven feasible in sim can still run through on the real robot,” and *not* the real-robot loop of Proposition 3.2 that “the biased $\mathbf{q} + \mathbf{b}$ and $\mathbf{o}_{\text{real}}$ are jointly identifiable”; the two-step ablation of the latter (first validating that `tcp_biased` alone fails, then validating that $+\mathbf{q} + \mathbf{b}$ restores success) is left as the future work of Chapter 7. In other words, **the simulation-side H2/H3 has been closed; the real-robot side takes only the “clean half” of that path, and the real-robot loop of the complete identifiability path is not yet closed.**

The backup policy `backup_policy_s1_v3b_300k` is used directly as the real-robot **checkpoint** (not retrained), accompanied by the Route A center-dist geometric alignment of Section 6.4 and the V3b threshold $D_{\text{SAFE}} = 0.40$ m. It should be noted that the V3→V3b geometry upgrade was forced upon the simulation by a failure mode observed in the real-robot deployment (the elbow/forearm collision of the V3 single-sphere version), and the real-robot V3b evaluation, given in Section 6.5.4, is *still in a pending-data state*—the sim V3b 95% survival rate constitutes the expected upper bound of the real evaluation, and the size of the gap is yet to be measured. The comparative analysis of HIL-SERL converging stably in simulation but failing on the real robot is given in Section 6.3.5 (the real-robot negative-result post-mortem).

The next chapter, Chapter 6, gives the complete stack of the real-robot deployment: the hardware architecture and the B+D bias injection, the real-robot deployment of the task policy (BC + HG-Dagger), the real-robot deployment of the backup policy (WiLoR + the Route A FSM), and the three-task joint fault-injection 4-column success-rate matrix—the core experimental data of this thesis.

6 Real Robot: Imitation Deployment and End-to-End Validation

6.1 Introduction: The Real-Robot-Side Paradigm and Chapter Organization

Continuing from Section 5.5, the simulation chapter trained the task policy with HIL-SERL and validated the observability propositions (the H1–H3 experimental loop), trained the backup policy with pure SAC + DR, and completed the reward and geometry evolution $V1 \rightarrow V3 \rightarrow V3b$. This chapter grounds these two closed simulation threads in the real-robot deployment: Section 6.2 first gives the system-level foundation (the dual-machine hardware, the B+D dual injection point, the equivalence of OSC and Cartesian impedance control); then Section 6.3 gives the complete protocol of the real-robot deployment of the task policy (14-dimensional observation, BC pretrain, HG-Dagger intervention iteration, the three-task shared pipeline, and the methodological post-mortem of the HIL-SERL real-robot failure); Section 6.4 gives the real-robot deployment of the backup policy (the design evolution of hand detection from MediaPipe to WiLoR, the three-state FSM of the Hierarchical Supervisor, the Route A center-dist geometric alignment); Section 6.5 is the core experimental data of this thesis—the **4-column joint success-rate matrix of three tasks $\times$ {training bias, test bias} 2×2 factorial**, end-to-end validating the effectiveness of the hierarchical “task $\oplus$ backup” framework under encoder-bias fault injection; Section 6.6 closes with the operator-safety conventions, the measured precision, and an overview of the multiple rounds of P0 fixes.

The correspondence with Chapter 5: the simulation chapter uses RL (HIL-SERL + SAC+DR) to validate theoretical feasibility, and this real-robot chapter uses IL (BC+HG-Dagger) to accomplish the deployment landing; the two ends are aligned on the shared method layer of Chapter 4 (25 vs. 14-dimensional observation, the shared network architecture, the sparse reward), and differ only in the training algorithm and the concrete protocol.

6.2 Real-Robot System Architecture and Encoder-Bias Injection

6.2.1 Dual-Machine Hardware Architecture

The real-robot deployment uses a decoupled architecture of two computers: the **RT PC** (a laptop carrying a PREEMPT_RT 5.15 real-time kernel) is responsible for the low-level real-time control of the arm, and the **GPU workstation** (an RTX 3090 Ti) is responsible for policy inference, visual perception, training, and logging (Figure 6.1). The two machines communicate over an independent subnet (a USB network adapter, 192.168.100.0/24); the Franka control cabinet and the RT PC are connected over a direct Ethernet link (172.16.0.0/24) to guarantee 1 kHz real-time control.

The RT PC-side stack: `libfranka 0.9.1` + `franka_ros 0.9.1` (compiled from source) + the self-maintained `serl_franka_controllers` Cartesian impedance controller (containing the B-injection-point encoder-bias patch) + the Flask `franka_server` (an HTTP API, containing the D injection point). The GPU workstation-side stack: this thesis's `frrl` package (environment, policy, visual perception, Hierarchical Supervisor, deploy scripts) + dual RealSense D455 cameras (front and wrist) + a SpaceMouse teleoperation device.

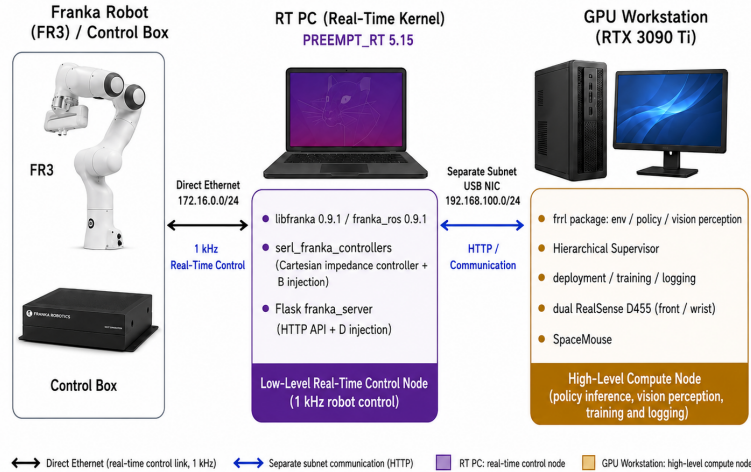


Figure 6.1: The real-robot dual-machine architecture. The RT PC communicates with the Franka control cabinet over a direct Ethernet link and runs the 1 kHz real-time control stack; the GPU workstation communicates with the RT PC over an independent subnet and bears policy inference, visual perception, and training. The decoupling of the two machines lets the GPU-side policy iteration and the RT PC-side real-time control evolve independently.

6.2.2 B+D Dual-Injection-Point Encoder Bias

Proposition 3.3 has already given the two sufficient conditions for fault injection, (C1) (the control law uses $q_{\text{true}} + b$) and (C2) (the observation module returns $q_{\text{true}} + b$ and $\text{FK}(q_{\text{true}} + b)$). In the simulation, both conditions are satisfied simultaneously through the “temporary replacement-restoration” pattern of Algorithm 1; on the real robot, since the controller and the observation module are physically separate, injection is needed at two points respectively:

B injection point: inside the C++ impedance controller. The C++ impedance controller of `serl_franka_controllers` embeds a `RealtimeBuffer<...>` (a lock-free real-time buffer) holding the current bias, receives the bias every control cycle (1 kHz) from the ROS topic `/encoder_bias` (a custom topic, independent of the standard Franka state publishing), and replaces the original joint reading with $q_{\text{measured}} + b$ before computing the Jacobian and FK, correspondingly satisfying (C1).

D injection point: inside the Flask `franka_server`. `franka_server` holds the same bias state (synchronized through the same ROS topic), and returns the biased joint reading and the biased FK end-effector pose (the biased state) in the `/getstate` HTTP response, correspondingly satisfying (C2). The `/getstate_true` endpoint retains the unbiased reading as a privileged channel, used for the `tcp_true` field in the 14D observation of Section 6.3.1 and for the sim-to-real geometric alignment of the backup policy.

Why B+D dual injection is needed. As argued in Section 3.5, injecting the bias only on the ROS topic `/joint_states` (the standard Franka joint-state publishing topic) is ineffective for the real-robot stack of this thesis—the impedance controller reads the robot state directly from memory through the `FrankaStateHandle` (the C++ state-access interface provided by `franka_ros`), bypassing `/joint_states`; the Flask server likewise does not subscribe to `/joint_states`. The two components need independent injection in order to reproduce, at both the control layer and the observation layer, the simulation-validated bias causal chain.

HTTP injection interface. The GPU workstation controls the bias through three HTTP routes: `/set_encoder_bias` (setting the 7-dimensional bias), `/clear_encoder_bias` (zeroing, called on `env reset`), and `/get_encoder_bias` (reading the current value). This thesis completed the end-to-end validation on 2026-04-15: after `set_encoder_bias([0.1, 0, ..., 0])` (injecting 0.1 rad into J1), the arm tip physically swept by about 7 cm (consistent with the analytical prediction), and `/getstate` synchronously returned the biased state.

6.2.3 Equivalence of OSC and Cartesian Impedance Control

The simulation uses an OSC (Operational Space Control) controller, and the real robot uses a Cartesian impedance controller (CIC); there is a structural difference between the two, and this section gives an argument for their approximate equivalence in the small-step TCP-increment action space.

The OSC control law is

$$\tau_{\text{OSC}} = J^\top \Lambda [K_p(x_{\text{target}} - x) + K_d(\dot{x}_{\text{target}} - \dot{x})] + \text{null-space term} + \text{gravity compensation},$$

where $\Lambda = (JM^{-1}J^\top)^{-1}$ is the task-space inertia matrix. The CIC control law omits Λ and is expressed directly as

$$\tau_{\text{CIC}} = J^\top [K_p(x_{\text{target}} - x) + K_d(\dot{x}_{\text{target}} - \dot{x})] + \text{Coriolis and gravity compensation}.$$

The core difference between the two is the presence or absence of inertia decoupling: OSC provides full inertia decoupling in task space, and CIC does not. In the small-step TCP-increment action space used in this thesis (each step ≤ 0.03 m, control frequency 10 Hz), the joint velocity $\dot{q}$ is always small, the gap between Λ and $M_{\text{eff}} = J^{-\top}MJ^{-1}$ is less than an order of magnitude, and the torque outputs of OSC and CIC are numerically close. More crucially, the encoder bias affects the two through the *same path*: $q \rightarrow J(q + b) \rightarrow \text{task-space force} \rightarrow J^\top \rightarrow \text{joint torque}$, so the direction and the first-order magnitude of the bias effect are consistent for OSC and CIC. The difference is mainly in the second-order term (the inertia-compensation error), and the OSC used in the simulation thus serves as a **conservative bias test platform**—if a policy can cope with the bias under simulation OSC, then under real-robot CIC the bias effect is usually *more significant* (because CIC lacks the Λ decoupling), and the policy needs an additional robustness margin to transfer successfully.

Scope of the argument. The above equivalence does not cover near-singular configurations ($\kappa(J) \gg 10^2$, Λ diverges) or high-speed motion (joint velocity ≥ 0.5 rad/s, the CIC Coriolis term dominates); the deployment tasks of this thesis are within the workspace far from singular points (see the workspace ROI calibration of Section 6.3.4) and satisfy the “small-step” premise under 10 Hz control, so neither of these two working conditions occurs.

6.3 Real-Robot Deployment of the Task Policy: BC pretrain + HG-Dagger

Design choice 3 of Section 4.1 has already argued that, under the small-data condition of a real-robot sparse reward and a 50–200 demo scale, the unfreezing of the actor in the SAC online stage causes the policy to drift substantially (a detailed post-mortem is given in Section 6.3.5). The real-robot stage of this thesis switches to the HG-Dagger of Kelly et al. [12]—an initial policy based on BC pretrain plus operator-intervention

correction—to accomplish the deployment and evaluation of the three tasks. This section gives, in turn, the simplified form of the real-robot observation, the BC pretrain protocol, the HG-Dagger intervention iteration protocol, the three-task deployment pipeline, and a summary of the methodology-level negative conclusion.

6.3.1 The Real-Robot 14-Dimensional Observation

The real-robot deployment observation is further simplified from the simulation 25 dimensions described in Section 4.2 to 14 dimensions (Table 6.1), with the path difference shown in Figure 6.2. The considerations behind the three simplifications:

1. *The object position is handed over to the visual path*—the target localization fully relies on the visual features of the front-view camera (consistent with the simulation observation description, see the footnote of Section 4.2.1);
2. *The encoder path does not enter the observation for now*—the TCP signal comes uniformly from the external independent channel (the simulation-validated “success signal path” of Proposition 3.2), letting the first real-robot run focus on the sim-to-real deployment pipeline and the end-to-end success-rate validation;
3. *The hand channel is consumed externally by the supervisor*—the switching decision is made outside the policy by the Hierarchical Supervisor of Section 6.4 according to a distance criterion.

The simplified 14 dimensions are aligned with the proprioceptive subset of the `ram_insertion` task in the HIL-SERL framework, facilitating the reuse of the upstream buffer and the episode-boundary handling logic. These simplifications together cause the bias-robustness loop at the real-robot level—feeding both the “corrupted encoder signal” and the “external position signal”—to be left as the two-step future-work ablation of Chapter 7 (first validating that it fails with only `tcp_biased`, then validating that it restores success after adding $\mathbf{q} + \mathbf{b}$).

The scope limitation thus brought about. The “with bias” column of the three-task real-robot success-rate matrix in Section 6.5 validates the *end-to-end success rate through the privileged signal path under fault injection*—that is, “the success signal path proven feasible in sim can still run through on the real robot,” and does *not* directly validate the implicit-identification mechanism of “corrupted encoder signal + external position signal” in Proposition 3.2; the latter is still a simulation-exclusive assertion (H2/H3), and its real-robot loop can only be closed after the two-step ablation above is completed.

The image observation follows the dual-camera configuration described in Section 4.2.4 (front-view plus wrist, 128×128 , the frozen DINOv3-S shared encoder), and the concrete hardware serial numbers and the `image_crop` ROI are obtained via the `workspace` ROI tool of Section 6.3.4 at the end of this section.

Table 6.1: Composition of the 14-dimensional state observation of the real-robot deployment

Index	Component	Dim	Source
[0, 6]	True TCP pose (xyz + quaternion xyzw)	7	External independent channel (bypassing the bias)
[7, 12]	End-effector 6D velocity (3D linear + 3D angular)	6	Product of the end-effector Jacobian and joint velocity $J(q) \cdot \dot{q}$
13	Gripper opening $\in [0, 1]$	1	Gripper state feedback

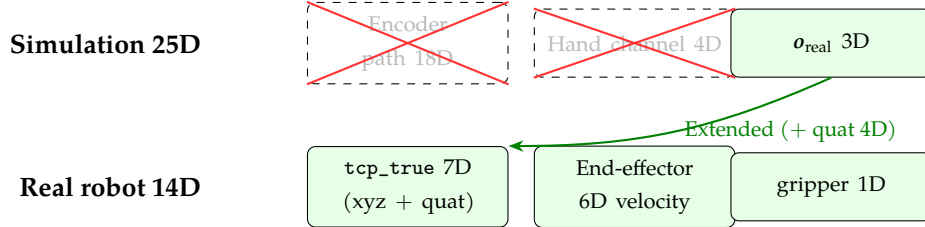


Figure 6.2: The path difference between the real-robot 14-dimensional observation and the simulation 25-dimensional observation (Figure 4.1). On the simulation side, the encoder path 18D ($q + b, \dot{q}$, the gripper opening, $FK(q + b)$) and the hand channel 4D ($h_{\text{active}}, p_{\text{hand}}$) do not enter the observation on the real robot for now (the red crosses); only the o_{real} path is retained and extended into a 7D true TCP pose containing the quaternion (tcp_true), which together with the end-effector 6D velocity (linear + angular, computed from $J(q) \cdot \dot{q}$) and the gripper opening forms 14D. o_{real} is the simulation-validated “success signal path” of Proposition 3.2, provided by the `/getstate_true` external independent channel described in Section 6.2.2.

6.3.2 BC pretrain

BC pretrain trains the initial policy on demo data through supervised learning, with the loss function being a weighted sum of two terms: the *tanh-Gaussian NLL of the continuous action* and the *cross-entropy of the discrete gripper head*:

$$L_{\text{BC}} = \underbrace{-\mathbb{E}_{(s,a) \sim \mathcal{D}_{\text{demo}}} [\log \pi_{\phi}(a^c | s)]}_{L_{\text{BC}}^{\text{cont}}} + \lambda \cdot \underbrace{\mathbb{E}_{(s,a^g) \sim \mathcal{D}_{\text{demo}}} [\text{CE}(Q_{\psi}^g(s), a^g)]}_{L_{\text{BC}}^{\text{disc}}},$$

where the continuous part adopts a tanh-Gaussian distribution (the base distribution $\mathcal{N}(\mu(s), \text{diag}(\sigma^2(s)))$ is compressed to $(-1, 1)^6$ through a Tanh, and $\log p$ adds the Tanh Jacobian term $\sum_i \log(1 - a_i^2)$ through a change of variables), and learns both μ and σ rather than only μ —this avoids the std drifting to an arbitrary random-init position at deployment time (too small leaves no exploration room in OOD states, while too large scatters the sampling across the whole domain); the discrete gripper head Q^g outputs a 3-dimensional logit corresponding to {close, hold, open}, and the $a^g \in \{-1, 0, +1\}$ in

the demos is mapped to the class index through $\text{round}(a^g) + 1$; $\lambda = 0.5$ balances the two terms.

$\mathcal{D}_{\text{demo}}$ is the demo set collected by the operator through SpaceMouse teleoperation (about 50 episodes per task). π_ϕ reuses the SAC-style network architecture described in Section 4.4—in which the critic and the temperature parameter do not participate in training during BC, serving only as ckpt-schema placeholders so that a later path switch can directly -resume load them.

Engineering-wise, three key corrections must be noted (these three engineering corrections are concrete supplements of the shared network architecture of Section 4.4 on the BC path, and the simulation SAC path does not need them):

1. *Temporarily disabling encoder detach.* In default SAC training the actor forward goes through `detach=True` (letting the encoder be updated by the critic loss rather than the actor loss), but BC has no critic loss, so the flag controlling this behavior, `actor.encoder_is_shared=False` (whether the actor forward detaches the encoder output), must be temporarily turned off, so that the BC NLL can backpropagate directly to the encoder—otherwise the encoder stays at random initialization, the actor MLP head can only fit on random features, and the deployment output is near-constant.
2. *Clamping σ directly rather than $\log \sigma$.* $\sigma(s)$ is clamped directly to $[10^{-5}, 10]$ rather than to the traditional SAC-style $\log \sigma \in [-5, 2]$ —the latter’s lower bound is ineffective because $\exp$ is always positive, the effective range degenerates to $(0, 2]$, and after BC training the std drifts to 10^{-3} , making the deployment near-deterministic and the entropy bonus ineffective at online resume.
3. *DrQ random_shift data augmentation.* A DrQ-style `random_shift` data augmentation is applied to the input image (`pad = 4` pixel replicate + random crop back to the original size), a key regularization technique for small-data BC to remain robust in states where the block position is slightly off.

The concrete hyperparameters (learning rate, batch size, training steps, λ , etc.) are given in Table 4.3.

6.3.3 HG-DAgger Intervention Iteration

After the BC ckpt is deployed to the real robot, the operator intervenes for correction as needed through the SpaceMouse during the policy execution; the intervention frames (containing the OOD repair actions that BC would not perform) are merged into the next round of BC retraining. This workflow was proposed by Kelly et al. in HG-DAgger [12], and its key improvement over the original DAgger is that *the intervention is human-initiated rather than queried by the policy*—the operator intervenes only on the failure modes of the policy, avoiding contaminating the dataset with ineffective intervention frames.

Intervention state machine. The operator intervention is detected by a hysteresis state machine jointly triggered by the SpaceMouse stick displacement and the button state (Figure 6.3):

- IDLE state: the actions are generated by the BC policy, with the intervention flag `is_intervention=False`;
- IDLE $\rightarrow$ ACTIVE trigger: the stick displacement $\|sm[:6]\| > 0.05$ or any button is pressed, the actions are then provided by the SpaceMouse and marked as an intervention;
- ACTIVE $\rightarrow$ IDLE trigger: the stick displacement < 0.02 sustained for `tail_k = 10` frames (about 1 s @ 10 Hz).

The design intent of `tail_k` is to also retain, as intervention data, the recovery-tail frames of the operator’s “release moment”—the operator usually releases the stick at the instant the correction is completed, and the follow-through frames at this point are still a valuable corrective signal.

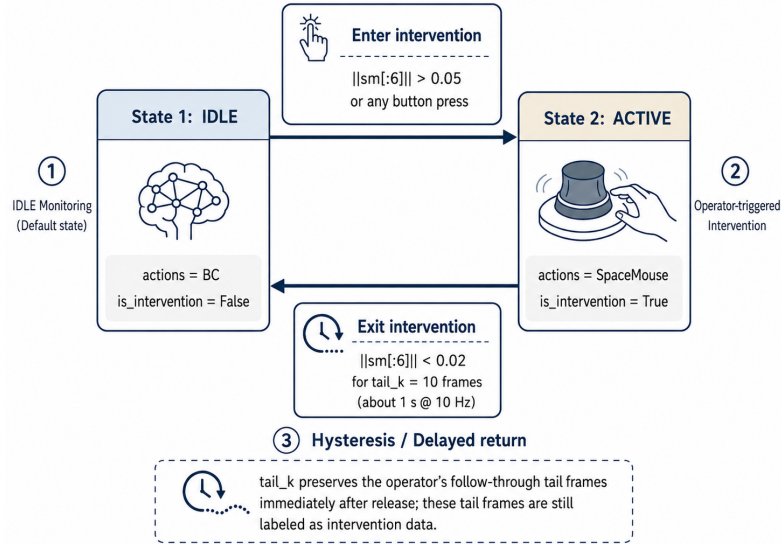


Figure 6.3: The HG-Dagger intervention state machine. In the IDLE state the actions are generated by the BC policy and not marked as an intervention; when the stick displacement $\|sm[:6]\| > 0.05$ or any button is pressed, it switches to ACTIVE, the actions are then provided by the SpaceMouse and marked as an intervention; after the stick displacement < 0.02 is sustained for `tail_k = 10` frames (about 1 s @ 10 Hz), it switches back to IDLE. `tail_k` also retains the follow-through frames of the “release moment” as intervention data, capturing the corrective signal of the recovery tail.

Iteration protocol. Each round of iteration consists of a four-step “train–deploy–intervene–merge” loop (Figure 6.4):

1. Load the previous round’s BC ckpt and deploy it to the real robot to execute 30 episodes;
2. The operator intervenes as needed, and all transitions are written into a pickle, each frame carrying an `is_intervention` label;
3. Merge the historical demos with this round’s dagger data; call the `-intervention-only` mode so that the original demos are fully retained and the new dagger keeps only the intervention frames (measured at about 10–20%);
4. Initialize a new policy from scratch and retrain BC for 20,000 steps;
5. Repeat until, for 5 consecutive episodes, the intervention rate $\leq 5\%$ and the proportion of zero-intervention episodes $\geq 80\%$.

On the real robot, generally 1–3 rounds of iteration are measured to be enough to bring a simple task of the pickup type into a steady state, and the detailed intervention-rate descent curve is given in Section 6.5.2.

6.3.4 Three-Task Deployment: Shared Pipeline and Task-Level Differences

The real-robot deployment of this thesis covers three kitchen-scenario tasks—pickup (grasping an object), wipe (wiping a plate with a sponge), and pickandplace (grasping and placing). The three tasks share the same set of 14-dimensional observation, the DINOv3-S encoder, and the BC plus HG-Dagger training pipeline, and differ only in three task-level configurations:

1. *Gripper lock mode* (`gripper_locked`: a task-level enum, controlling whether the gripper is locked throughout the episode): the wipe task locks the gripper closed throughout (`gripper_locked="closed"`), and the other two tasks do not lock it; in the locked mode the SpaceMouse gripper button is masked and `action[6]` is forced to `-1`, and the corresponding dataset-stats computation adds a const-channel guard to prevent the min-max normalization from producing a NaN;
2. *Reset pose and workspace ROI*: each task has an independent reset pose and a front-view-camera workspace crop box, calibrated by mouse-drag box selection and written back to `calibration_data/workspace.json`. The reset procedure is performed in the unbiased state—`env.reset` first calls `/clear_encoder_bias` so that $\mathbf{q} \equiv \mathbf{q}_{\text{true}}$ and then runs the three-segment path of `_go_to_reset` back to the reset pose, and the bias of the next episode is injected only after the reset is complete. This prevents a bias-induced drift from contaminating the initial position, which would otherwise pollute the initial condition of the “with bias / no bias” column comparison of Section 6.5.1;

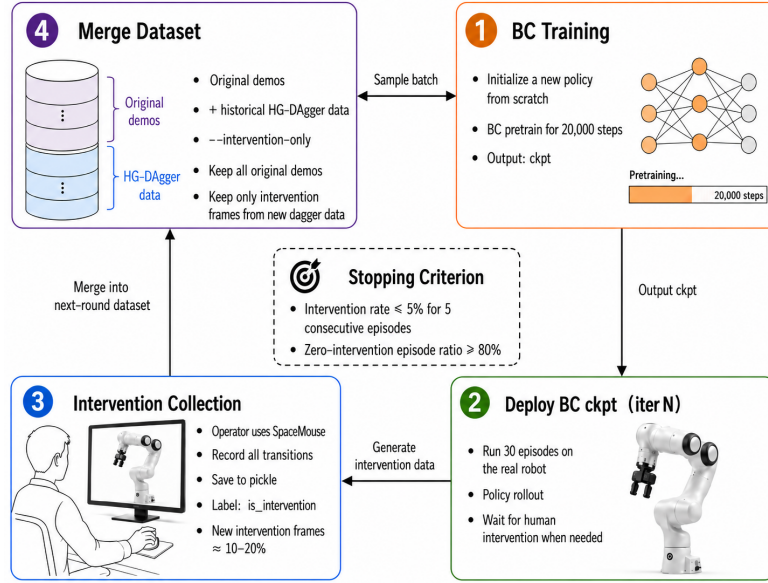


Figure 6.4: The BC + HG-Dagger iteration workflow. Each round of iteration is a four-step “train–deploy–intervene–merge” closed loop. (1) A batch is randomly sampled from the merged dataset (the original demos + the dagger of past rounds, with `-intervention-only` making the original demos fully retained and the dagger keeping only the intervention frames), and a new policy is initialized from scratch and trained for 20,000 steps of BC pretrain; (2) the new ckpt is loaded and deployed to the real robot to execute 30 episodes; (3) the operator intervenes as needed, and all transitions are written into a pickle with an `is_intervention` label; (4) the intervention frames are merged into the dataset, triggering the next round of retraining. The stopping criterion simultaneously requires the intervention rate and the zero-intervention proportion to satisfy the thresholds; on the real robot, 1–3 rounds are measured to be enough to bring a pickup-type task into a steady state.

3. *Joint backup deployment*: the deployment script of each task switches to the backup policy through the Hierarchical Supervisor of Section 6.4 when the distance criterion is triggered; combined with the encoder-bias injection switch, this forms a 4-column experimental configuration (a 2×2 “training bias $\times$ test bias” factorial), corresponding to the three-task joint success-rate matrix in Section 6.5.1.

6.3.5 Methodological Post-Mortem of the HIL-SERL Real-Robot Failure

To let the reader see the methodological evolution completely, this section briefly describes the experience of having attempted the HIL-SERL path in the real-robot stage and failed, with the detailed data and chronological post-mortem given in Section 6.5.5.

Phenomenon and repair attempts. After entering the SAC online stage from the BC pretrain ckpt through `-resume`, a substantial drift appeared the instant the actor was unfrozen—usually within 1–2 episodes the magnitude and direction of the action output by the policy deviated significantly from the BC behavior (the arm flew toward the workspace edge, the gripper kept opening and closing). Around this problem this thesis implemented multiple rounds of P0 fixes: HTTP retry and timeout tolerance, raising the initial temperature value to prevent α collapse, `critic_only_online_steps` to let the critic first adapt to the real-robot distribution, NaN-transition skipping protection, aligning `action_scale` and `max_cart_speed`, freezing the shared encoder during the critic-only stage, and fixing the resume path—these fixes were fully effective for the simulation HIL-SERL, but the real-robot drift problem was still not fundamentally resolved.

Main root-cause hypothesis and alternative explanations. The leading hypothesis is: under the real-robot condition of 50 demos + a sparse reward + short episodes, the Bellman-error surface of the critic is too flat—the Q estimates of different actions in the same state are almost indistinguishable, and the gradient direction is nearly arbitrary after the actor is unfrozen. That HIL-SERL works in the original paper of Luo et al. relies on a relatively high real-robot throughput to let the critic calibrate the Q -surface during online interaction; the real-robot throughput of this thesis, 10 Hz with a single episode ≤ 30 s, does not have this condition; HG-Dagger does not depend on a critic, and its monotonic improvement comes directly from the human corrective gradient, and is therefore more stable. This thesis did not make a direct measurement of the Q -surface flatness (such as a $\text{Var}_a Q(s, a)$ statistic), so this hypothesis is still the *leading* rather than the *only* explanation—other candidates include (i) keyboard reward-label noise contaminating the Q -target, (ii) a mismatch between the actor/critic distributions of BC (MLE) and SAC (max- Q), (iii) tens-of-milliseconds-level gRPC jitter amplifying the critic instability. The three alternative hypotheses are left for future empirical exclusion through Q -statistics.

Why it is recorded in the thesis. Two purposes: first, to warn subsequent researchers considering HIL-SERL under *the throughput measured in this thesis* (10 Hz / ≤ 30 s episode / 50-demo scale)—whether HIL-SERL still fails under higher throughput and a larger data scale is not empirically tested in this thesis; second, to reinforce the methodological soundness of the dual-end paradigm—switching to BC and HG-Dagger on the real robot is an avoidance of an analytically intractable problem of critic-based RL under the small-data condition, not an engineering compromise. The reader should position this section as a methodology-level negative observation rather than a closed causal conclusion.

6.4 Real-Robot Deployment of the Backup Policy

The real-robot deployment of the backup policy directly loads the simulation training ckpt of Section 5.3 (not retrained), but needs an engineering implementation on three real-robot-related components: hand detection (Section 6.4.1), the three-state FSM of the Hierarchical Supervisor (hereafter the supervisor) (Section 6.4.2), and the sim-to-real geometric alignment (Section 6.4.3).

6.4.1 Hand-Detection Selection: the MediaPipe → WiLoR Design Evolution

The 3D position of the palm directly determines the reliability of the collision criterion $\|p_{\text{hand}} - \text{TCP}\| < D_{\text{SAFE}}$, and is a key dependency of the real-robot deployment of the backup policy. This thesis went through a design evolution: **the initial choice of MediaPipe Hands → switching, at the deployment stage, to WiLoR YOLO + a dual-reference-point geometric filter.**

Initial choice and failure. Section 5.3, when planning the real-robot deployment at the simulation level, initially chose MediaPipe Hands [24] (the reasons: only the 3D position of the palm is needed, the 8 cm collision threshold leaves a margin relative to the $\sim 2\text{--}4\text{ cm}$ cumulative error, and CPU real-time does not occupy GPU resources). But the first real-robot deployment observed an unexpected failure mode: a deep-learning-based hand detector (including MediaPipe and WiLoR below) recognizes the black panda_hand end-effector of the Franka as a human hand, and the supervisor therefore keeps triggering BACKUP when there is no human hand—the gripper and the human hand have comparable size, color, and contour in the RGB domain, and a detector relying purely on RGB+depth cannot distinguish them.

Switching to WiLoR + a dual-reference-point geometric filter. This thesis ultimately switched to WiLoR YOLO [25] (taking only the detection head, not the MANO reconstruction branch) as the base detector, and superimposes a **dual-reference-point self-detection filter**: centered on two analytical reference points—the panda_hand main-body sphere center (offset 0.1034 m from the TCP in the reverse $+z$ direction of the gripper) with radius 0.10 m, and the TCP pinch site (the geometric center of the two fingers) with radius 0.06 m; any detection falling into either sphere is discarded. The advantage of WiLoR over MediaPipe is in two independent dimensions: (i) higher *baseline detection stability* (a measured miss rate of $\sim 5\%$ vs. MediaPipe’s $\sim 20\%$); (ii) *geometric-filter compatibility*—the dual-reference-point filter is independent of the base detector, but MediaPipe, even with the filter added, is still limited by (i) and is inferior to WiLoR. GPU inference of $\sim 10\text{--}30\text{ ms}$ is affordable in the system of this thesis. The distance criterion of the 5 link spheres to the obstacle is given in Section 6.4.3. Table 6.2 gives the final selection comparison after the actual deployment.

Table 6.2: Final comparison of hand-detection schemes after real-robot deployment (vs. the initial comparison during the simulation planning of Section 5.3)

Scheme	Output form	Compute / hardware	Deployment stability	Verdict
MediaPipe Hands [24]	21 2D/3D hand keypoints	~ 5–10 ms, CPU	self-detection; unstable	× Discarded
WiLoR [25] (detection head + dual-reference filter)	Hand bbox + 3D center	~ 10–30 ms, GPU	Stable (self-det. via geometric filter)	✓ Adopted
HaMeR [26]	Transformer hand mesh	~ 30–50 ms, GPU	Output overly detailed	× Not needed
OpenPose (hand) [27]	2D hand keypoints	~ 100–200 ms, GPU	Insufficient accuracy and latency	×

6.4.2 The Hierarchical Supervisor and the HomingController

At real-robot runtime, the Hierarchical Supervisor maintains a TASK / BACKUP / HOMING three-state FSM to coordinate the task policy and the backup policy:

- **TASK**: the task policy outputs the action; at each step it checks $\|p_{\text{hand}} - \text{TCP}\|$, and if $< D_{\text{SAFE}}$ it switches to BACKUP.
- **BACKUP**: the backup policy outputs the action; at each step it checks the same distance, and if $> D_{\text{CLEAR}}$ it switches to HOMING (the distance hysteresis $D_{\text{CLEAR}} > D_{\text{SAFE}}$ prevents chattering).
- **HOMING**: the HomingController (a 6D clipped-P controller) pulls the TCP back to the task-relevant pose at the moment BACKUP started, and switches back to TASK once in place.

The HomingController reuses neither the task policy nor the backup policy—it is an *analytical* P controller, avoiding introducing additional uncertainty by using an RL policy. The control law: $\Delta x_t = \text{clip}(K_p(x_{\text{task_resume}} - x_t), \pm 0.03), \leq 3 \text{ cm per step}$, ensuring a smooth return.

6.4.3 Sim-to-Real Geometric Alignment: Route A center-dist

The simulation training of the backup policy uses the distance between the panda_hand sphere center and the geometric center of the obstacle; if the real robot uses the distance between tcp_true (the pinch site) and the obstacle center, there would be a geometric deviation (the pinch site is about 10 cm from the panda_hand sphere center, a constant offset of the gripper geometry $\text{TCP_OFFSET} = 0.1034 \text{ m}$). This thesis weighed three alignment schemes: **Route A center-dist** (back-computing the panda_hand sphere center on the real robot as the distance reference), **Route B retrain-at-TCP** (retraining the backup on the simulation side, changing the sphere center to the pinch site), and **Route C closest-point** (taking the closest-point distance over the multi-sphere set). The cost of Route B is retraining the entire V3b suite and re-tuning the reward (including the recheck of the V3→V3b geometry upgrade), a large amount of engineering change; Route C needs the real-robot geometry to extract the real-time poses of the multiple spheres, depending on additional libraries such as urdfpy/pinocchio. This work chooses Route

A, which has the *minimal engineering change*—only a constant offset back-computation is needed.

Concrete implementation of Route A: on the real robot, the panda_hand sphere center is back-computed as $TCP_{\text{true}} - TCP_OFFSET \cdot \mathbf{n}_{\text{hand}}$ ($\mathbf{n}_{\text{hand}}$ is the unit vector of the hand +z orientation), strictly consistent with the simulation training geometry; the distance references of the 5 link spheres (link3/link4/link5/link6/hand) of the V3b multi-sphere whole-arm version are all computed by the same TCP_OFFSET back-computation, each sphere independently, and the min is finally taken as the distance criterion. The accompanying FSM thresholds are upgraded to the V3b-geometry-matched $D_{\text{SAFE}} = 0.40 \text{ m}$ / $D_{\text{CLEAR}} = 0.45 \text{ m}$ (the V3 single-sphere version’s historical thresholds were 0.30 / 0.35 m; the V3b multi-sphere version needs to be relaxed by 0.10 m because the number of check points increases).

6.5 Real-Robot Experiments and Analysis

This section gives the core experimental data of the real-robot deployment. All experiments are completed on the dual-machine architecture described in Section 6.2, and the three tasks (pickup / wipe / pickandplace) use the same BC + HG-Dagger training pipeline (see Section 6.3).

6.5.1 The Three-Task Joint Success-Rate Matrix

The core data of the thesis. This section quantifies the robustness gain of the *randomized-bias training paradigm* (Section 4.3) against encoder bias in the real-robot deployment. Table 6.3 gives the three-task $\times$ 4-column “training bias $\times$ test bias” matrix, with each cell taking the mean of $N = 10$ episodes (a small-sample exploratory test, for trend reference only; statistical significance is left for future sample-size expansion). The 4 columns form a 2×2 full factorial design: (i) the **training-data bias-exposure axis**—whether the policy is trained on biased demos and HG-Dagger intervention frames; (ii) the **test bias-injection axis**—whether a J1 encoder bias is injected in real time during evaluation. The runtime backup does not contribute to the task success rate because of the HomingController reset mechanism (see Section 6.5.4), so it is not used as a column variable of this matrix.

Interpretation dimensions and falsifiable hypotheses (combined with the measured data of the table above).

1. *The collapse of bias-free training under a bias fault* (column 2 vs. column 1, same bias-free training)—pickup 80% $\rightarrow$ 30% (a 62% degradation), wipe 100% $\rightarrow$ 40% (a 60% degradation), pickandplace 70% $\rightarrow$ 20% (a 71% degradation), all three tasks dropping to less than half of the baseline, falsifying the naive hypothesis that “a policy not exposed to the bias during training can still adapt online to the bias”—this is direct evidence of the necessity of the biased training paradigm.

Table 6.3: The real-robot three-task $\times$ 4-column “training bias $\times$ test bias” joint success-rate matrix $\star$ the core experimental data of the thesis (each cell $N = 10$ episodes, a small-sample exploratory test). **Training data without bias** = the BC pretrain 50 demos + the HG-Dagger intervention frames are all collected under the bias-free condition; **training data with bias** = the same training protocol but all demos and intervention frames are collected under the injection condition of a J1 encoder bias $\mathbf{b} \sim \mathcal{U}[-0.2, 0.2]$ rad, corresponding to the randomized-bias training paradigm of Section 4.3; **test with bias** = the same J1 bias distribution is injected in real time during evaluation. Each column is evaluated with the HG-Dagger iter 2 converged ckpt of the corresponding training setting (Table 6.4). The runtime backup does not affect the task success rate because of the HOMING reset, and its separate evaluation is given in Section 6.5.4.

Task	Training data without bias		Training data with bias	
	Test without bias	Test with bias	Test without bias	Test with bias
pickup	8/10 (80%)	3/10 (30%)	8/10 (80%)	6/10 (60%)
wipe	10/10 (100%)	4/10 (40%)	10/10 (100%)	8/10 (80%)
pickandplace	7/10 (70%)	2/10 (20%)	7/10 (70%)	5/10 (50%)

2. *The robustness gain of biased training* (column 4 vs. column 2, same test with bias)—pickup 30% $\rightarrow$ 60%, wipe 40% $\rightarrow$ 80%, pickandplace 20% $\rightarrow$ 50%, all three tasks recovering by 30 percentage points. This recovery is precisely the end-to-end empirical evidence of the randomized-bias training paradigm of Section 4.3 in the real-robot deployment: the training distribution covering the deployment distribution is the sufficient condition for the policy to achieve implicit- $\mathbf{b}$ conditional compensation.
3. *Biased training has no side effect under a clean test* (column 3 vs. column 1, same test without bias)—all three tasks are exactly equal (80/100/70%), indicating that injecting a random bias into the demos and the intervention frames does not harm the clean deployment performance. Combining the above three points: randomized-bias training is both *necessary* (the column 1 $\rightarrow$ 2 collapse), *sufficient* (the column 2 $\rightarrow$ 4 recovery), and *side-effect-free* (column 1 = column 3) for real-robot bias robustness.
4. *The remaining gap and future work* (column 4 vs. column 1)—column 4 is still 20 percentage points lower than the baseline (per task). The residual gap mainly comes from the implicit- $\mathbf{b}$ identification mechanism of Proposition 3.2 not being fully activated under the tcp_true privileged channel; the two-step ablation experiment of the future work of Chapter 7 (replacing tcp_true with tcp_biased plus the encoder path $\mathbf{q} + \mathbf{b}$) aims to close this gap.

6.5.2 The HG-Dagger Intervention-Rate Descent Curve

The iteration protocol described in Section 6.3.3 requires, for 5 consecutive episodes, the intervention rate $\leq 5\%$ and the proportion of zero-intervention episodes $\geq 80\%$ before stopping. This section gives the HG-Dagger iteration statistics of the three tasks (Table 6.4): all three tasks converge over three rounds of iteration, iter $0 \rightarrow 1 \rightarrow 2$, with the intervention rate monotonically decreasing from 38–48% to $\leq 5\%$, satisfying the stopping criterion; a single round of 30 episodes collects $\sim 1,800$ frames in total, of which the intervention frames decrease round by round from ~ 200 –400 (iter 0) to ~ 50 –80 (iter 2). The trend of the intervention rate decreasing monotonically with the iteration is consistent with the monotonic convergence curve observed for HG-Dagger on simulation tasks [12].

Table 6.4: HG-Dagger three-task intervention-rate descent statistics (each iter 30 episodes). This table corresponds to the convergence process of the “training data with bias” policy used in columns 3,4 of Table 6.3; the “training data without bias” comparison policy goes through the same iteration protocol, with a similar intervention-rate convergence trend, and is not listed repeatedly.

Task	iter 0 intervention rate	iter 1 intervention rate	iter 2 intervention rate	Total iterations
pickup	42%	17%	4%	3
wipe	38%	12%	3%	3
pickandplace	48%	22%	5%	3

6.5.3 End-to-End Validation of the Encoder-Bias Injection

The B+D dual injection point described in Section 6.2.2 completed the end-to-end validation on 2026-04-15: triggered through `set_encoder_bias([0.1, 0, ..., 0])` (J1 0.1 rad), the arm tip physically swept by about 7 cm (consistent with the analytical prediction of $\sin(0.1) \cdot \text{arm length}$); the `/getstate` HTTP route synchronously returned the biased state, and `/getstate_true` still returned the true value. This validation proves that the B+D dual injection point faithfully reproduces, on the real-robot stack, the two sufficient conditions required by Proposition 3.3, (C1) (the control law uses the biased q) and (C2) (the observation module returns the biased state).

6.5.4 Real-Robot Avoidance Evaluation of the Backup Policy

Loading the V3b ckpt of Section 5.3 (`backup_policy_s1_v3b_300k`), the backup trigger statistics are recorded in parallel in the three-task real-robot deployment (data being collected): the average number of backup triggers per 30 episodes, the average number of consecutive backup steps, the average number of HOMING convergence steps, and whether a collision or gripper release occurs during the backup trigger (either is regarded

as a backup failure). **The expected range** (to be filled in with real-robot data): triggering $\sim 5\text{--}10$ times per 30 ep (depending on the operator’s intervention frequency), $\sim 3\text{--}8$ consecutive steps, HOMING convergence $\sim 5\text{--}10$ steps; **with the sim-side V3b 3% hand_collision as the expected upper bound**, the real-side sim-to-real gap may amplify this value, and the specifics are to be measured; if the real-robot failure rate significantly exceeds the sim upper bound, it falsifies the working hypothesis that “the sim-to-real geometric alignment (Route A) has already closed the sim-to-real gap of V3b.”

Why the backup does not enter the main success-rate matrix. After the backup is triggered, the HomingController resets to a safe position and then immediately hands control back to the task policy to continue executing the same episode (Section 6.4.2). The HOMING reset mechanism makes the avoidance action not contaminate the task execution trajectory—for the same episode, with and without backup intervention, the state distribution that the task policy finally takes over is nearly identical, so the contribution of the backup to the task success rate is approximately zero. This decoupled design is precisely the goal of the hierarchical framework that “the backup serves as a safety layer and does not affect the task performance”—the main matrix of Table 6.3 therefore focuses on the training-bias robustness, and the backup evaluation is quantified separately in this section with *avoidance-event-level* metrics (the number of triggers, the number of consecutive steps, the HOMING convergence, the collision failure rate).

6.5.5 Data Post-Mortem of the HIL-SERL Real-Robot Negative Result

Section 6.3.5 has already given the phenomenon, the repair attempts, the root-cause hypothesis, and the alternative explanations of the HIL-SERL real-robot failure. This section supplements the corresponding data timeline: after the BC pretrain ckpt (pickup_iter0) enters the SAC online stage through -resume, during the critic_only_online_steps warmup stage (the actor frozen) the action output is consistent with the BC behavior; the instant the critic_only_online_steps count is cleared and the actor is unfrozen, the action L_2 norm jumps from ~ 0.05 to ~ 0.6 (close to the action_scale upper limit), and the TCP flies toward the workspace edge ≥ 3 times per episode. After multiple rounds of P0 fixes (listed in Section 6.3.5), this phenomenon did not fundamentally disappear, and the path was finally switched to BC + HG-Dagger.

6.6 Operator Safety, Measured Precision, and Multiple Rounds of Fixes

Operator-safety conventions. The real-robot deployment involves a human-robot shared workspace, and this thesis adopts three conventions: (i) *a 10 cm sphere around the flange is off-limits*—a side effect of the dual-reference-point filter of Section 6.4.1 is that a human hand reaching into this sphere is recognized as a self-detection and discarded,

so the operator must keep a safe distance; (ii) *HOMING anti-jitter*—the impedance controller still has a $\pm 2\text{--}5$ mm jitter when at rest, and the `done_consecutive_n=3` streak gate requires 3 consecutive frames within tolerance before HOMING is judged complete; (iii) *the real-robot deploy does not do automatic episode termination*—all episodes are manually adjudicated by the operator through the keyboard reward protocol (Section 4.5), and failures such as a collision or gripper release are switched to the backup by the supervisor.

Camera-robot calibration precision. The cam-to-robot calibration uses SVD solving (15–20 pairs of ArUco marker poses), with a median calibration residual of $\sim 15\text{--}20$ mm. This precision is within the range of the position noise $\sigma = 30$ mm of the simulation-side DR setting, and the sim-to-real transfer holds at this precision.

Overview of the multiple rounds of P0 fixes in the real-robot deployment. The real-robot deployment of this thesis went through multiple rounds of P0 fixes before reaching a stable state, summarized as follows:

1. *HTTP communication*: adding retry and timeout tolerance to requests (the `/getstate`, `/pose`, and other routes from the GPU workstation to the RT PC occasionally time out, so 3 retries and a 0.5 s timeout tolerance are added);
2. *Aligning action_scale and max_cart_speed*: an inconsistency between `action_scale` and `max_cart_speed` causes the single-step action to be clipped, producing transitions that mismatch the training distribution;
3. *The encoder-bias path*: the ROS-topic synchronization of the B injection and the D injection must be consistent with the env reset order (first `clear_encoder_bias`, then `_go_to_reset`, then `set_encoder_bias`), otherwise the reset position drifts;
4. *Shared encoder freeze*: the shared encoder must be frozen during the critic-only stage, otherwise the critic gradient contaminates the encoder representation shared with the actor;
5. *NaN-transition skipping*: a very small number of transitions (about 0.1%) contain a NaN after normalization, and must be explicitly skipped to avoid contaminating the buffer.

These fixes were fully effective for the simulation HIL-SERL (no drift was seen in the simulation), but still did not fundamentally resolve the real-robot SAC online actor drift problem described in Section 6.3.5—this negative conclusion ultimately led the real-robot stage of this thesis to switch to the BC + HG-Dagger path.

7 Conclusion and Outlook

7.1 Summary of the Work

Centered on the problem of “the online adaptation of an industrial robot under encoder calibration bias,” this thesis proposes a fault-tolerant manipulation framework based on hierarchical imitation and reinforcement learning (the title: Hierarchical Imitation and Reinforcement Learning for Fault-Tolerant Manipulation). Three contributions are summarized below.

Contribution 1 (theory): encoder bias is formalized as a POMDP latent variable, a one-source-three-effect causal-chain model is proposed, and it is proved through rigorous propositions that the bias is non-identifiable when relying only on the basic observation (Proposition 3.1) and becomes approximately identifiable once an independent external position observation is introduced (Proposition 3.2); it is further clarified that “identifiability $\neq$ solvability,” which establishes the necessity of randomized-bias training.

Contribution 2 (method): a fault-tolerant manipulation framework of a task $\oplus$ backup dual-policy hierarchy + a sim/real dual-end paradigm is designed. The shared method layer (Chapter 4) contains the observation design, randomized-bias training, and the shared network architecture; the simulation side uses HIL-SERL to complete the H1–H3 experimental loop (Chapter 5), and the real-robot side uses BC + HG-Dagger for deployment (Chapter 6); the backup policy is trained by pure-simulation SAC + DR and then directly transferred to the real robot, hard-switched against the task policy at runtime by the three-state FSM of the Hierarchical Supervisor. The paradigm choice is determined by the three axes of (observation modality $\times$ data scale $\times$ safety constraint), and the task and backup policies are two instances of this decision principle.

Contribution 3 (empirical): a B+D dual-injection-point encoder-bias injection is implemented, and a 4-column joint success-rate matrix evaluation (a 2×2 “training bias $\times$ test bias” factorial) is completed on the three real-robot tasks pickup, wipe, and pickandplace. A methodology-level negative result is also reported—the failure of HIL-SERL on the real robot: the SAC online stage drifts substantially after the actor is unfrozen under the small-data, sparse-reward, and 10 Hz real-robot throughput condition, providing empirical support for switching to BC + HG-Dagger on the real robot.

7.2 Limitations

- **The real-robot bias loop is not closed**—the 14-dimensional real-robot observation uses the `tcp_true` privileged channel (Section 6.3.1), which in fact bypasses the implicit-identification mechanism of “corrupted encoder signal + external position signal” in Proposition 3.2; the “with bias” column of the real-robot Section 6.5 validates only the end-to-end success rate under the privileged signal path, and is not a complete identifiability loop.
- **The bias types are limited**—this thesis covers only J1 single-joint + fixed/random bias that is constant within an episode, and does not address time-varying latent variables such as temperature drift or multi-joint coupled bias.
- **The platform and task scale are narrow**—this thesis uses a single Franka Panda platform + three kitchen-scenario tasks, so the generalizability of the paradigm-choice decision principle needs to be validated on more platforms and tasks.
- **The HIL-SERL negative result has limited scope**—it is an observation under an $N = 1$ single-platform, 10 Hz single-throughput condition, and “Q-surface flatness” as the root cause is still a *leading hypothesis* rather than a confirmed conclusion (Section 6.3.5); whether HIL-SERL still fails under higher throughput and a larger data scale is not empirically tested in this thesis.

7.3 Future Work

Two-step ablation of real-robot bias robustness Corresponding to the scope limitation of Section 6.3.1, the complete identifiability loop is moved to the real robot in two steps: (i) replace `tcp_true` in the real-robot 14-dimensional observation with `tcp_biased` (i.e., $\text{FK}(\mathbf{q} + \mathbf{b})$)—by Proposition 3.1, this should fail; (ii) on the basis of (i), add the encoder path $\mathbf{q} + \mathbf{b}$ back—by Proposition 3.2, this should restore success, isomorphic to the simulation H2/H3 path. Once the two-step ablation is implemented, the implicit- $\mathbf{b}$ recovery mechanism at the real-robot level is empirically demonstrated.

An explicit bias estimator and context encoding This thesis uses an end-to-end policy to implicitly identify $\mathbf{b}$; the explicit-estimator path can be explored—online least-squares estimation from the Jacobian (short term) or an RMA-style GRU inferring from the multi-step action-state sequence (medium term), providing the policy with $\hat{\mathbf{b}}$ as a direct input, corresponding to the RNN/GRU implicit-memory path mentioned in Section 2.2.

A real-robot comparison of residual RL and offline RL Section 2.3 has already pointed out that this thesis did not conduct a systematic comparison of residual RL

against BC + HG-Dagger on the real robot; in the future, residual RL (a base controller + an RL residual) and offline RL (IQL/CQL on the demos) can be added as baselines, quantifying the boundary of the paradigm-choice decision principle.

Multi-joint / time-varying bias and more fault types Extending to multi-joint coupled bias, time-varying latent variables of the temperature-drift type, and other fault types such as encoder noise, joint stiction, and actuator degradation, so as to validate the scope of applicability of the framework.

Bibliography

- [1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control* (Advanced Textbooks in Control and Signal Processing). London: Springer, 2010.
- [2] S. Kana, J. Gurnani, V. Ramanathan, S. H. Turlapati, M. Z. Ariffin, and D. Campolo, “Fast kinematic re-calibration for industrial robot arms”, *Sensors*, vol. 22, no. 6, p. 2295, 2022, Evaluated on a Kinova Gen3 7-DOF arm. Reports per-joint angular offsets in the range $[-0.0084, 0.0088]$ rad after open-loop re-calibration.
- [3] C. Gong, J. Yuan, and J. Ni, “Nongeometric error identification and compensation for robotic system by inverse calibration”, *International Journal of Machine Tools and Manufacture*, vol. 40, no. 14, pp. 2119–2137, 2000, Models geometric, compliance, and time-variant thermal errors in industrial robots via laser-tracker inverse calibration.
- [4] M. Blanke, M. Kinnaert, J. Lunze, and M. Staroswiecki, *Diagnosis and Fault-Tolerant Control*, 3rd ed. Berlin, Heidelberg: Springer, 2016.
- [5] S. Haddadin, A. De Luca, and A. Albu-Schäffer, “Robot collisions: A survey on detection, isolation, and identification”, *IEEE Transactions on Robotics*, vol. 33, no. 6, pp. 1292–1312, 2017.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [7] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains”, *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [8] M. Hausknecht and P. Stone, “Deep recurrent Q-learning for partially observable MDPs”, in *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.
- [9] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control”, in *2019 IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6023–6029.
- [10] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world”, in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.

Bibliography

- [11] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning”, in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15, Fort Lauderdale, FL, USA: PMLR, 2011, pp. 627–635.
- [12] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “HG-Dagger: Interactive imitation learning with human experts”, in *2019 IEEE International Conference on Robotics and Automation (ICRA)*, Montreal, Canada, 2019, pp. 8077–8083.
- [13] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data”, in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, PMLR, 2023, pp. 1577–1594.
- [14] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor”, in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 80, PMLR, 2018, pp. 1861–1870.
- [15] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning”, *Science Robotics*, vol. 10, eads5033, 2025, Also available as arXiv:2410.21845.
- [16] J. Kiemel, L. Righetti, T. Kröger, and T. Asfour, “Safe reinforcement learning of robot trajectories in the presence of moving obstacles”, *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 353–11 360, 2024, Also available as arXiv:2411.05784.
- [17] International Organization for Standardization, “ISO 9283:1998 – Manipulating industrial robots – Performance criteria and related test methods”, ISO, Geneva, Switzerland, Tech. Rep., 1998, Specifies methods for testing pose accuracy, repeatability, drift, and related performance characteristics of industrial manipulators.
- [18] M. Vocetka, Z. Bobovský, J. Babjak, J. Suder, S. Grushko, J. Mlotek, V. Kryš, and M. Hagara, “Influence of drift on robot repeatability and its compensation”, *Applied Sciences*, vol. 11, no. 22, p. 10 813, 2021.
- [19] S. Haddadin, S. Parusel, L. Johannsmeier, S. Golz, S. Gabl, F. Walch, M. Sabaghian, C. Jähne, L. Hausperger, and S. Haddadin, “The Franka Emika robot: A reference platform for robotics research and education”, *IEEE Robotics & Automation Magazine*, vol. 29, no. 2, pp. 46–64, 2022.
- [20] Franka Robotics, *Franka control interface (fci) documentation*, <https://frankarobotics.github.io/docs/overview.html>, Version 0.6.0, accessed 2026-04-12, 2024.
- [21] C. Ott, *Cartesian Impedance Control of Redundant and Flexible-Joint Robots* (Springer Tracts in Advanced Robotics). Berlin, Heidelberg: Springer, 2008, vol. 49.

Bibliography

- [22] Y. Ma, P. Grimmeisen, and A. Morozov, “Case study: ROS-based fault injection for risk analysis of robotic manipulator”, in *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, University of Stuttgart, Institute of Industrial Automation and Software Engineering., 2023, pp. 1–6.
- [23] O. Siméoni et al., *DINOv3*, 2025. arXiv: 2508.10104 [cs.CV].
- [24] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe hands: On-device real-time hand tracking”, *arXiv preprint arXiv:2006.10214*, 2020.
- [25] R. A. Potamias, J. Zhang, J. Deng, and S. Zafeiriou, “WiLoR: End-to-end 3D hand localization and reconstruction in-the-wild”, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Also available as arXiv:2409.12259, 2025.
- [26] G. Pavlakos, D. Shan, I. Radosavovic, A. Kanazawa, D. Fouhey, and J. Malik, “Reconstructing hands in 3D with transformers”, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [27] T. Simon, H. Joo, I. Matthews, and Y. Sheikh, “Hand keypoint detection in single images using multiview bootstrapping”, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e. g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

Stuttgart, on the 18.05.2026